RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 Fundamentals of Data Science

Laboratory Record Note Book

Name :	Aditya.P
Year / Branch / Section :	II / CSE / M
University Register No. :	2116250701026
College Roll No. :	250701026
Semester :	III
Academic Year :	2026-27

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 Fundamentals of Data Science

NAME	Aditya.P
ROLL NO	250701026
YEAR	II
SEC	M

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ADITYA.P

Academic Year: 2026-27 Semester: III Branch: CSE

Register No. 2116250701026

Certified that this is the bonafide record of work done by the above student in the CS23334

FUNDAMENTALS OF DATA SCIENCE Laboratory

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner External Examiner

INDEX

SlNO	Experiment	Date	GitHub
1	a) Analyze the trend of data science job postings over the last decade for the given data. b) Analyze and visualize the distribution of various data science roles. c) Differentiate Structured, Un-structured and Semi-structured data. d) Encrypt and decrypt given sensitive data using cryptography library fernet.	20/07/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_01.ipynb
2	Upload and Analyze the data set given in csv format and perform data preprocessing and visualization.	20/07/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_02.ipynb
3	a) Data visualization using line plot. b) Data visualization using bar chart. c) Data visualization using pie chart.	27/07/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_03.ipynb
4	Count the frequency of occurrence of a word in a body of text.	27/07/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_04.ipynb
5	Data Collection and Initial Exploration.	03/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_05.ipynb
6	Time Series Analysis of Website Traffic.	03/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_06.ipynb
7	Random Sampling and Sampling Distribution.	10/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_07.ipynb
8	Conduct Z test for the Data given using Python.	10/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_08.ipynb
9	Experiment to understand Matplotlib library use cases in Data Science through visualization.	17/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_09.ipynb
10	Experiment to understand array function in Data science.	17/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_10.ipynb
11	Experiment to understand pandas library use cases in Data science.	24/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/

			Experiment_11.ipynb
12	Experiment to detect outliers in a given data set.	24/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_12.ipynb
13	Experiment to handle missing values and inappropriate data in a given data set.	31/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_13.ipynb
14	Experiment to understand feature scaling.	31/08/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_14.ipynb
15	Experiment to understand the data preprocessing in Data science.	14/09/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_15.ipynb
16	Experiment to understand EDA - Quantitative and Qualitative analysis.	14/09/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_16.ipynb
17	Experiment to understand Linear Regression for a given data set.	28/09/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_17.ipynb
18	Experiment to understand KNN algorithm for a given data set.	28/09/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_18.ipynb
19	Experiment to understand Logistic Regression for a given data set.	12/10/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_19.ipynb
20	Experiment to understand K-means clustering algorithm for a given data set.	12/10/2026	https://github.com/adityaparthiban200729/FDS-Lab-Notebooks/blob/main/Experiment_20.ipynb

EXPERIMENT NO: 1

A) ANALYZE THE TREND OF DATA SCIENCE JOB POSTINGS OVER THE LAST DECADE FOR THE GIVEN DATA. B) ANALYZE AND VISUALIZE THE DISTRIBUTION OF VARIOUS DATA SCIENCE ROLES. C) DIFFERENTIATE STRUCTURED, UN-STRUCTURED AND SEMI-STRUCTURED DATA. D) ENCRYPT AND DECRYPT GIVEN SENSITIVE DATA USING CRYPTOGRAPHY LIBRARY FERNET.

AIM:

To analyze the trend of data science job postings over the last decade and the distribution of various data science roles, to differentiate structured, un-structured and semi-structured data, and to encrypt and decrypt sensitive data using the cryptography fernet library.

PROGRAM:

Experiment No: 1

1.a Analyze the trend of data science job postings over the last decade for the given data on the number of data science job postings each year. Use pandas for data manipulation and matplotlib/seaborn for visualization.

```
%matplotlib inline
import pandas as pd
import matplotlib.pyplot as plt

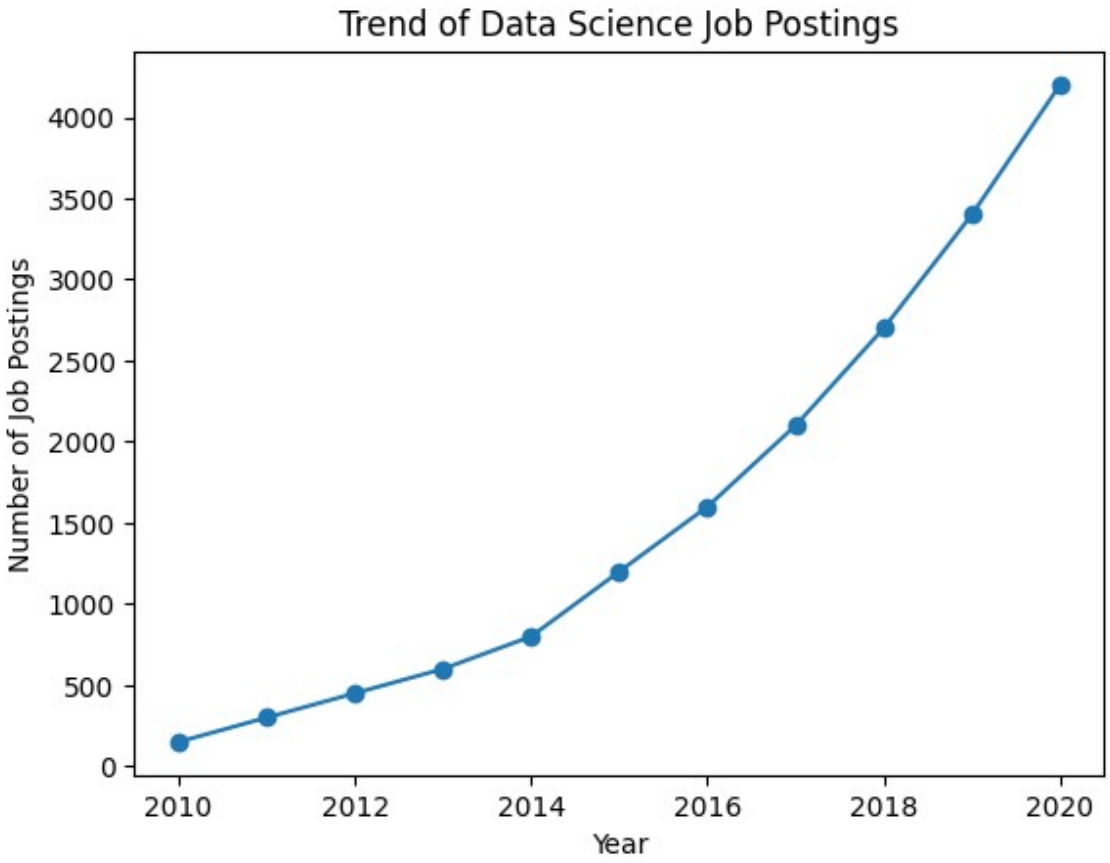
data = {'Year': list(range(2010, 2021)),
        'Job Postings': [150, 300, 450, 600, 800, 1200, 1600, 2100, 2700, 3400,
4200]}

df = pd.DataFrame(data)
print(df)

plt.plot(df['Year'], df['Job Postings'], marker='o')
plt.title('Trend of Data Science Job Postings')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.show()
```

OUTPUT:

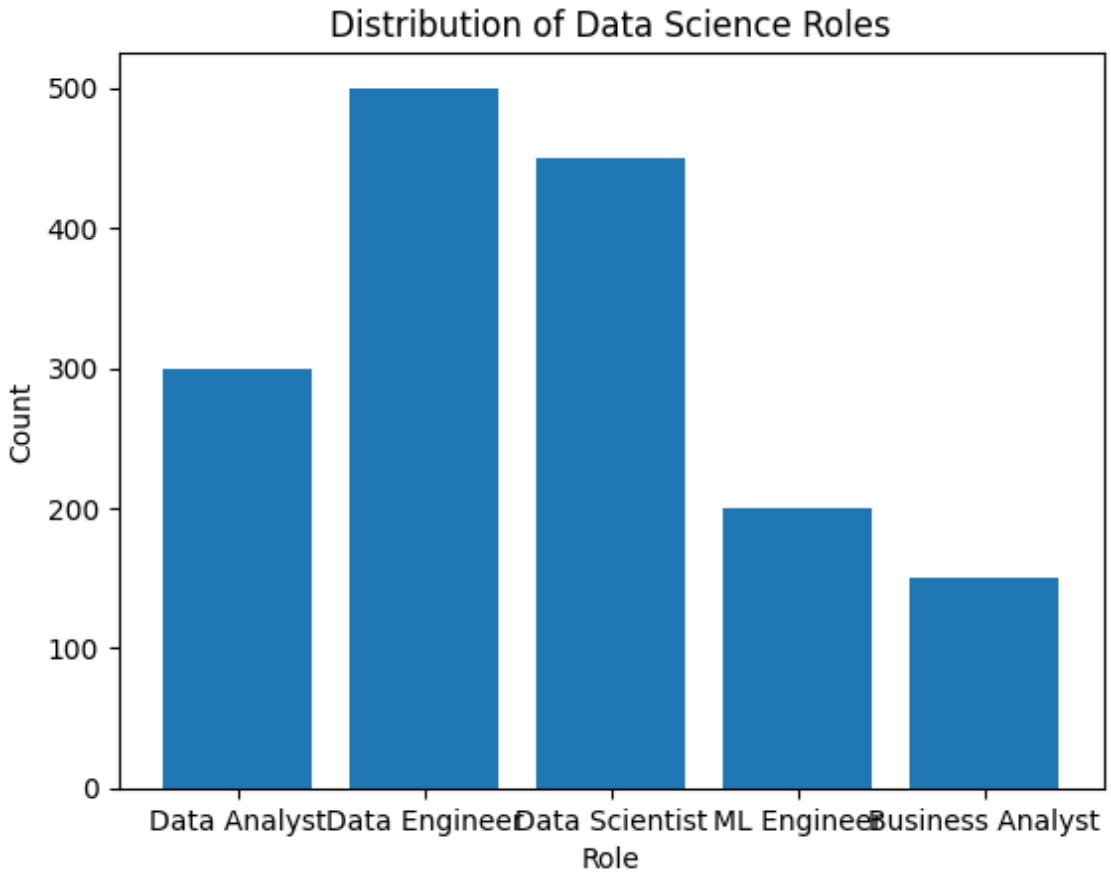
```
    Year  Job Postings
0   2010           150
1   2011           300
2   2012           450
3   2013           600
4   2014           800
5   2015          1200
6   2016          1600
7   2017          2100
8   2018          2700
9   2019          3400
10  2020          4200
```



1.b Analyze and visualize the distribution of various data science roles (Data Analyst, Data Engineer, Data Scientist, etc.) from a dataset.

```
roles = ['Data Analyst', 'Data Engineer', 'Data Scientist', 'ML Engineer',
         'Business Analyst']
counts = [300, 500, 450, 200, 150]

plt.bar(roles, counts)
plt.title('Distribution of Data Science Roles')
plt.xlabel('Role')
plt.ylabel('Count')
plt.show()
```



1.c Conduct an experiment to differentiate Structured, Un-structured and Semi-structured data based on data sets given.

```
# Structured data example
structured_data = pd.DataFrame({
    'ID': [1, 2, 3],
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35]
})
print("Structured Data:\n", structured_data)

# Unstructured data example
unstructured_data = ("This is an example of unstructured data. It can be a
piece "
                     "of text, an image, or a video file.")
print("\nUnstructured Data:\n", unstructured_data)

# Semi-structured data example (JSON)
semi_structured_data = {'ID': 1, 'Name': 'Alice',
                        'Attributes': {'Height': 165, 'Weight': 68}}
print("\nSemi-structured Data:\n", semi_structured_data)

Structured Data:
    ID     Name  Age
0   1    Alice   25
1   2      Bob   30
2   3  Charlie   35

Unstructured Data:
 This is an example of unstructured data. It can be a piece of text, an image,
or a video file.
```

```
Semi-structured Data:
 {'ID': 1, 'Name': 'Alice', 'Attributes': {'Height': 165, 'Weight': 68}}
```

1.d Conduct an experiment to encrypt and decrypt given sensitive data. Use the cryptography library fernet.

```
%pip install -q cryptography
from cryptography.fernet import Fernet

# Generate key and encrypt data
key = Fernet.generate_key()
cipher_suite = Fernet(key)
plain_text = b"Rajalakshmi Engineering College."
cipher_text = cipher_suite.encrypt(plain_text)

# Decrypt data
decrypted_text = cipher_suite.decrypt(cipher_text)

print("Original Data:", plain_text)
print("Encrypted Data:", cipher_text)
print("Decrypted Data:", decrypted_text)
```

```
Note: you may need to restart the kernel to use updated packages.
Original Data: b'Rajalakshmi Engineering College.'
Encrypted Data:
b'gAAAAABq6DHRgcCnNDfNhsPgiHm3YQOpaEBTnFDLgJkxZnGUEbg9VhcWJJL0KnXaU7kqsHEVGAXXU
PWxRA0SjAb0rBGFklgn8SAGYl7Axai6HcayrrFJCfoblg1MGLNG7mngtOKvwRdv'
Decrypted Data: b'Rajalakshmi Engineering College.'
```

RESULT:

Thus the trend of data science job postings and the distribution of data science roles were analyzed and visualized, the three types of data were differentiated, and sensitive data was encrypted and decrypted using the fernet library.

EXPERIMENT NO: 2

UPLOAD AND ANALYZE THE DATA SET GIVEN IN CSV FORMAT AND PERFORM DATA PREPROCESSING AND VISUALIZATION.

AIM:

To upload and analyze the data set given in csv format and perform data preprocessing and visualization.

PROGRAM:

```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

data = {
    'Date': ['2023-01-01', '2023-01-02', '2023-01-03', '2023-01-04',
             '2023-01-05', '2023-01-06', '2023-01-07', '2023-01-08',
             '2023-01-09', '2023-01-10', '2023-01-11', '2023-01-12',
             '2023-01-13', '2023-01-14', '2023-01-15', '2023-01-16'],
    'Product': ['Product A', 'Product B', 'Product A', 'Product C',
                'Product B', 'Product C', 'Product A', 'Product B',
                'Product C', 'Product A', 'Product B', 'Product C',
                'Product A', 'Product B', 'Product C', 'Product C'],
    'Sales': [200, 150, 220, 300, 180, 250, 300, 170,
              270, 290, 160, 260, 340, 190, 280, 240],
    'Quantity': [8, 3, 7, 6, 4, 6, 6, 4,
                 6, 6, 3, 6, 6, 3, 6, 6],
    'Region': ['North', 'South', 'North', 'East', 'West', 'East', 'North',
               'South', 'East', 'North', 'South', 'East', 'North', 'West',
               'East', 'East']
}
df = pd.DataFrame(data)
df.to_csv('sales_data.csv', index=False)
print("sales_data.csv created successfully")
```

OUTPUT:

```
sales_data.csv created successfully

# Load the data into a pandas DataFrame
df = pd.read_csv('sales_data.csv')

# Display the first few rows of the DataFrame
print(df.head())

# Check for missing values
print(df.isnull().sum())

         Date    Product  Sales  Quantity Region
0  2023-01-01  Product A    200         8  North
1  2023-01-02  Product B    150         3  South
2  2023-01-03  Product A    220         7  North
3  2023-01-04  Product C    300         6   East
4  2023-01-05  Product B    180         4   West
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64
```

```
# Summary statistics
print(df.describe())

# Group by product and calculate the total sales and quantity
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()

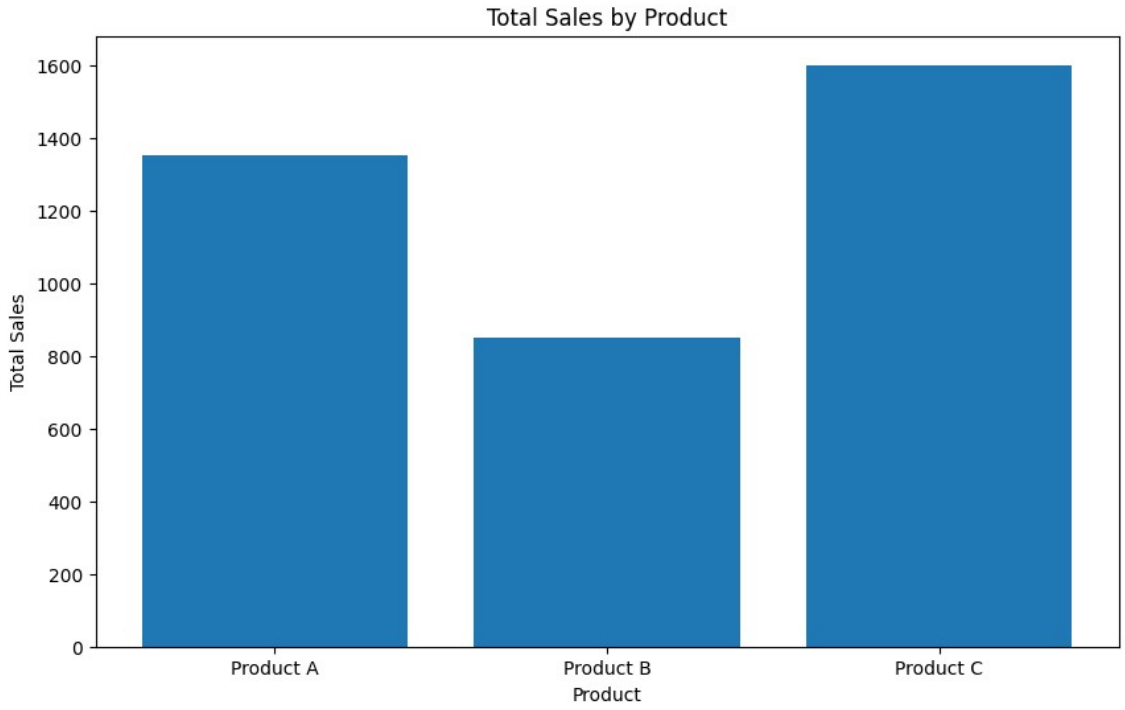
print(product_summary)
```

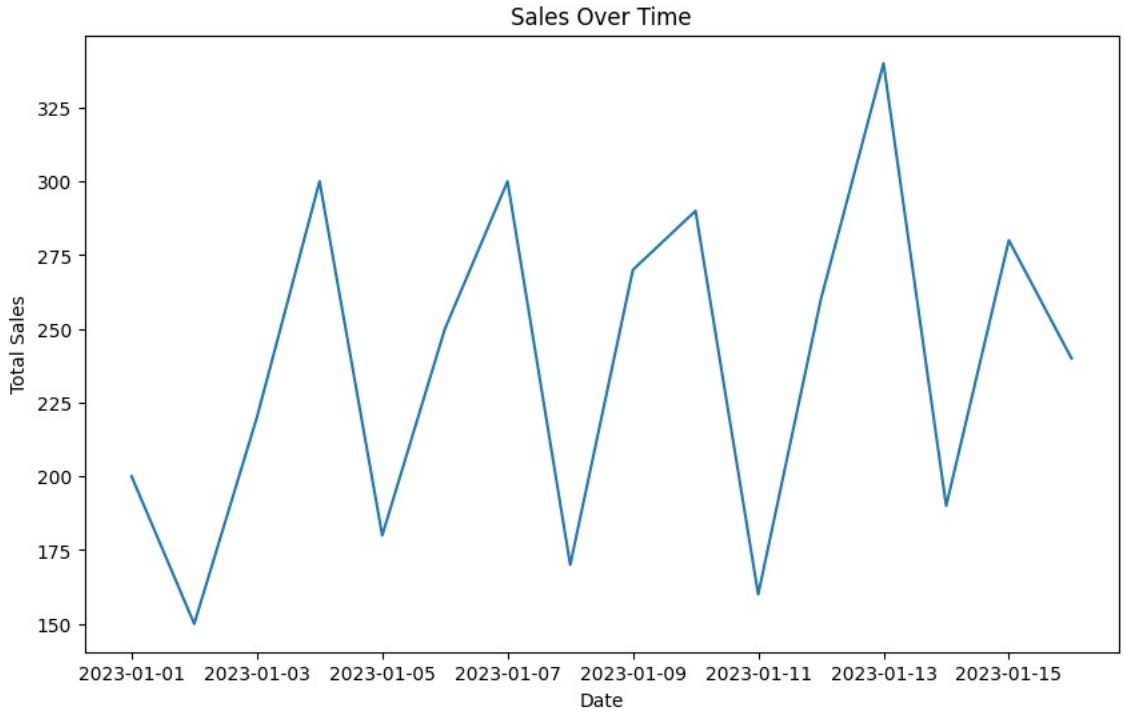
```
            Sales  Quantity
count   16.000000    16.000
mean   237.500000     5.375
std     57.792733     1.500
min    150.000000     3.000
25%    187.500000     4.000
50%    245.000000     6.000
75%    282.500000     6.000
max    340.000000     8.000
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```

```
# Bar plot of total sales by product
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()

# Line plot of sales over time
df['Date'] = pd.to_datetime(df['Date'])
sales_over_time = df.groupby('Date').agg({'Sales': 'sum'}).reset_index()

plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.show()
```





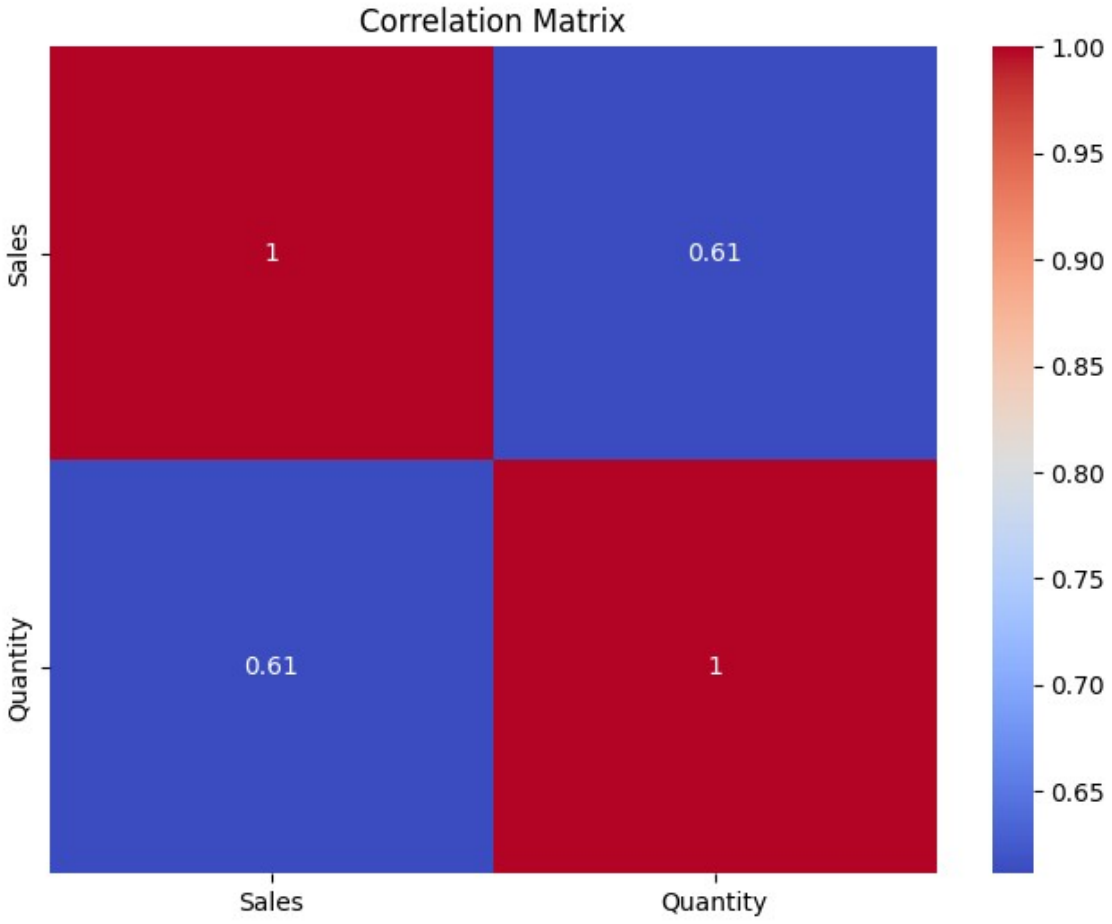

```
# Pivot table to analyze sales by region and product
pivot_table = df.pivot_table(values='Sales', index='Region', columns='Product',
                             aggfunc=np.sum, fill_value=0)
print(pivot_table)

# Correlation matrix
correlation_matrix = df.corr(numeric_only=True)
print(correlation_matrix)

# Heatmap of the correlation matrix
import seaborn as sns
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

```
Product  Product A  Product B  Product C
Region
East             0          0       1600
North         1350          0          0
South            0        480          0
West             0        370          0
            Sales  Quantity
Sales     1.00000   0.61138
Quantity  0.61138   1.00000
/tmp/ipykernel_2731/1413451212.py:2: FutureWarning: The provided callable
<function sum at 0x7f7410719750> is currently using DataFrameGroupBy.sum. In a
future version of pandas, the provided callable will be used directly. To keep
current behavior pass the string "sum" instead.
  pivot_table = df.pivot_table(values='Sales', index='Region',
columns='Product',
```



RESULT:

Thus the given csv data set was uploaded, analyzed, preprocessed and visualized using pandas, matplotlib and seaborn.

EXPERIMENT NO: 3

A) DATA VISUALIZATION USING LINE PLOT. B) DATA VISUALIZATION USING BAR CHART. C) DATA VISUALIZATION USING PIE CHART.

AIM:

To conduct experiments to show data visualization using line plot, bar chart and pie chart.

PROGRAM:

Experiment No: 3

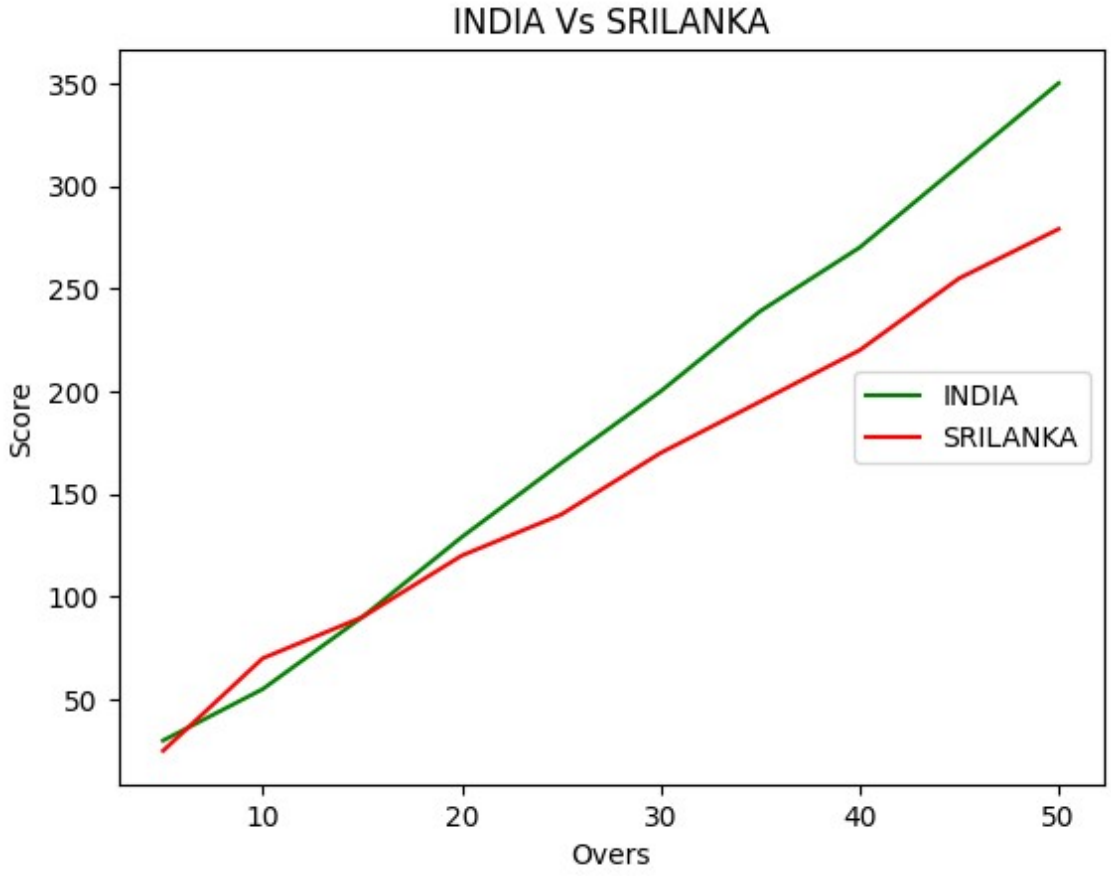
3.a Conduct an experiment to show data visualization using line plot

```
%matplotlib inline
import matplotlib.pyplot as cricket

Overs = list(range(5, 51, 5))
Indian_Score = [30, 55, 90, 129, 165, 200, 239, 270, 310, 350]
Srilankan_Score = [25, 70, 90, 120, 140, 170, 195, 220, 255, 279]

cricket.plot(Overs, Indian_Score, color="green", label="INDIA")
cricket.plot(Overs, Srilankan_Score, color="red", label="SRILANKA")
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend(loc="center right")
cricket.show()
```

OUTPUT:

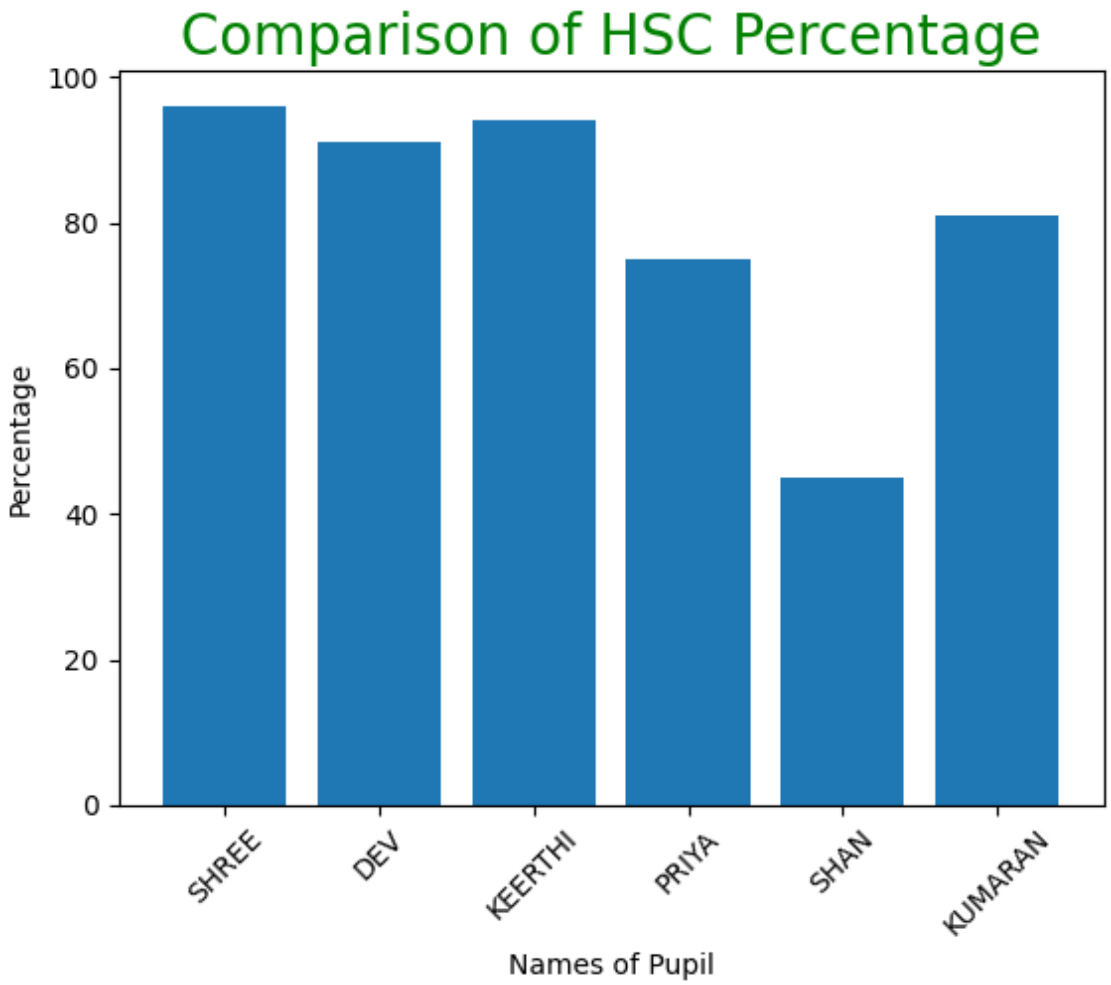


3.b Conduct an experiment to show data visualization using bar chart

```
import matplotlib.pyplot as hscmark
import numpy as np

Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]

hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20, color="green")
hscmark.show()
```



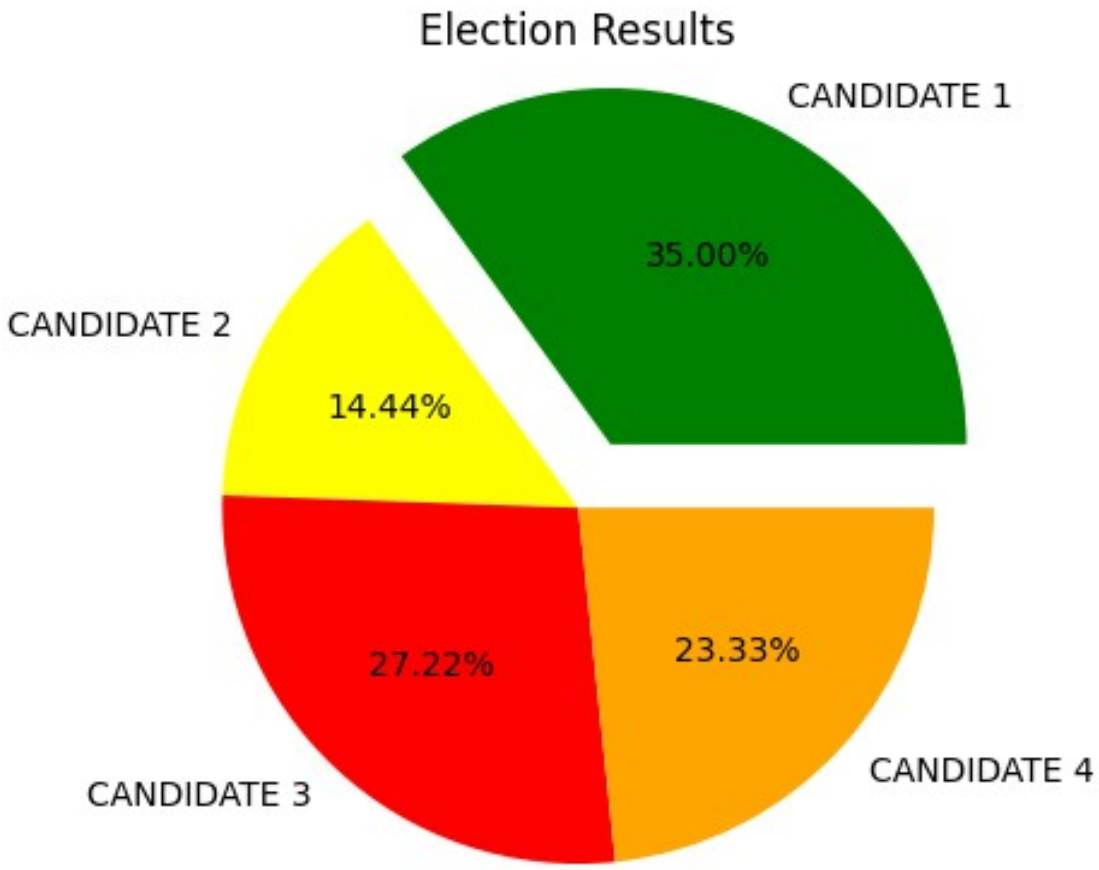
3.c Conduct an experiment to show data visualization using pie chart

```
import matplotlib.pyplot as election

# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0)

# Plotting the pie chart
election.pie(Votes, labels=labels, colors=colors, explode=explode,
             autopct='%0.2f%%')
election.title('Election Results')
```

```
election.show()
```



RESULT:
Thus data visualization was performed using line plot, bar chart and pie chart.

EXPERIMENT NO: 4

COUNT THE FREQUENCY OF OCCURRENCE OF A WORD IN A BODY OF TEXT.

AIM:

To count the frequency of occurrence of a word in a body of text, which is often needed during text processing.

PROGRAM:

```
%pip install -q nltk
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg

# Download the gutenberg corpus if not already installed
nltk.download('gutenberg')
nltk.download('punkt')  # Ensure the punkt tokenizer models are also downloaded
nltk.download('punkt_tab')

# Load the text from the gutenberg corpus
sample = gutenberg.raw("austen-emma.txt")

# Tokenize the sample text
token = word_tokenize(sample)

# Create a list of the first 50 tokens
wlist = []
for i in range(50):
    wlist.append(token[i])

# Calculate the frequency of each word in the list
wordfreq = [wlist.count(w) for w in wlist]

# Print the word-frequency pairs
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

OUTPUT:

```
Note: you may need to restart the kernel to use updated packages.
[nltk_data] Downloading package gutenberg to
[nltk_data]     /home/sandbox/nltk_data...
[nltk_data]   Package gutenberg is already up-to-date!
[nltk_data] Downloading package punkt to /home/sandbox/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     /home/sandbox/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package punkt_tab to
[nltk_data]     /home/sandbox/nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816', 1),
(']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma', 2),
('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5), ('clever', 1), (',', 5),
('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a', 1), ('comfortable', 1),
('home', 1), ('and', 3), ('happy', 1), ('disposition', 1), (',', 5), ('seemed',
1), ('to', 1), ('unite', 1), ('some', 1), ('of', 2), ('the', 2), ('best', 1),
('blessings', 1), ('of', 2), ('existence', 1), (';', 1), ('and', 3), ('had',
1), ('lived', 1), ('nearly', 1), ('twenty-one', 1), ('years', 1), ('in', 1),
('the', 2), ('world', 1), ('with', 2)]
```

RESULT:
Thus the frequency of occurrence of each word in the given body of text was counted using nltk.

EXPERIMENT NO: 5
DATA COLLECTION AND INITIAL EXPLORATION.

AIM:
To collect, load and perform initial exploration of the diabetes dataset.

PROGRAM:

```
%matplotlib inline
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'Pregnancies': [
        6, 1, 8, 1, 0, 5, 3, 10,
        2, 8, 4, 10, 10, 1, 5, 7,
        0, 7, 1, 1, 3, 8, 7, 9,
        11, 10, 7, 1, 13, 5, 5, 3,
        3, 6, 10, 4, 11, 9, 2, 4,
        3, 7, 7, 9, 7, 0, 1, 2,
        7, 7, 1, 1, 5, 8, 7, 1,
        7, 0, 0, 0
    ],
    'Glucose': [
        148, 85, 183, 89, 137, 116, 78, 115,
        197, 125, 110, 168, 139, 189, 166, 100,
        118, 107, 103, 115, 126, 99, 196, 119,
        143, 125, 147, 97, 145, 117, 109, 158,
        88, 92, 122, 103, 138, 102, 90, 111,
        180, 133, 106, 171, 159, 180, 146, 71,
        103, 105, 103, 101, 88, 176, 150, 73,
        187, 100, 146, 105
    ],
    'BloodPressure': [
        72, 66, 64, 66, 40, 74, 50, 0,
        70, 96, 92, 74, 80, 60, 72, 0,
        84, 74, 30, 70, 88, 84, 90, 80,
        94, 70, 76, 66, 82, 92, 75, 76,
        58, 92, 78, 60, 76, 76, 68, 72,
        64, 84, 92, 110, 64, 66, 56, 70,
        66, 0, 80, 50, 66, 90, 66, 50,
        68, 88, 82, 64
    ],
    'SkinThickness': [
        35, 29, 0, 23, 35, 0, 32, 0,
        45, 0, 0, 0, 0, 23, 19, 0,
        47, 0, 38, 30, 41, 0, 0, 35,
        33, 26, 0, 15, 19, 0, 26, 36,
        11, 0, 31, 33, 0, 37, 42, 47,
        25, 0, 18, 24, 0, 39, 0, 27,
        32, 0, 11, 15, 21, 34, 42, 10,
        39, 60, 0, 41
    ],
    'Insulin': [
        0, 0, 0, 94, 168, 0, 88, 0,
        543, 0, 0, 0, 0, 846, 175, 0,
        230, 0, 83, 96, 235, 0, 0, 0,
        146, 115, 0, 140, 110, 0, 0, 245,
        54, 0, 0, 192, 0, 0, 0, 207,
        70, 0, 0, 240, 0, 0, 0, 0,
        0, 0, 82, 36, 23, 300, 342, 0,
        304, 110, 0, 142
```

```
        ],
        'BMI': [
            33.6, 26.6, 23.3, 28.1, 43.1, 25.6, 31.0, 35.3,
            30.5, 0.0, 37.6, 38.0, 27.1, 30.1, 25.8, 30.0,
            45.8, 29.6, 43.3, 34.6, 39.3, 35.4, 39.8, 29.0,
            36.6, 31.1, 39.4, 23.2, 22.2, 34.1, 36.0, 31.6,
            24.8, 19.9, 27.6, 24.0, 33.2, 32.9, 38.2, 37.1,
            34.0, 40.2, 22.7, 45.4, 27.4, 42.0, 29.7, 28.0,
            39.1, 0.0, 19.4, 24.2, 24.4, 33.7, 34.7, 23.0,
            37.7, 46.8, 40.5, 41.5
        ],
        'DiabetesPedigreeFunction': [
            0.627, 0.351, 0.672, 0.167, 2.288, 0.201, 0.248, 0.134,
            0.158, 0.232, 0.191, 0.537, 1.441, 0.398, 0.587, 0.484,
            0.551, 0.254, 0.183, 0.529, 0.704, 0.388, 0.451, 0.263,
            0.254, 0.205, 0.257, 0.487, 0.245, 0.337, 0.546, 0.851,
            0.267, 0.188, 0.512, 0.966, 0.42, 0.665, 0.503, 1.39,
            0.271, 0.696, 0.235, 0.721, 0.294, 1.893, 0.564, 0.586,
            0.344, 0.305, 0.491, 0.526, 0.342, 0.467, 0.718, 0.248,
            0.254, 0.962, 1.781, 0.173
        ],
        'Age': [
            50, 31, 32, 21, 33, 30, 26, 29,
            53, 54, 30, 34, 57, 59, 51, 32,
            31, 31, 33, 32, 27, 50, 41, 29,
            51, 41, 43, 22, 57, 38, 60, 28,
            22, 28, 45, 33, 35, 46, 27, 56,
            26, 37, 48, 54, 40, 25, 29, 22,
            31, 24, 22, 26, 30, 58, 42, 21,
            41, 31, 44, 22
        ],
        'Outcome': [
            1, 0, 1, 0, 1, 0, 1, 0,
            1, 1, 0, 1, 0, 1, 1, 1,
            1, 1, 0, 1, 0, 0, 1, 1,
            1, 1, 1, 0, 0, 0, 0, 1,
            0, 0, 0, 0, 0, 1, 1, 1,
            0, 0, 0, 1, 0, 1, 0, 0,
            1, 0, 0, 0, 0, 1, 0, 0,
            1, 0, 0, 0
        ],
    }
    df = pd.DataFrame(data)
    df.to_csv('diabetes.csv', index=False)
    print("diabetes.csv created successfully")
```

OUTPUT:

```
    diabetes.csv created successfully

    # Load the dataset
    diabetes_df = pd.read_csv('diabetes.csv')

    # Display the first few rows
    print(diabetes_df.head())
    print("\nShape of the data set:", diabetes_df.shape)
    print("\nColumn information:")
    print(diabetes_df.info())

       Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
    0            6      148             72             35        0  33.6
    1            1       85             66             29        0  26.6
    2            8      183             64              0        0  23.3
    3            1       89             66             23       94  28.1
    4            0      137             40             35      168  43.1
```

```
   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Shape of the data set: (60, 9)

Column information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 60 entries, 0 to 59
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               60 non-null     int64
 1   Glucose                   60 non-null     int64
 2   BloodPressure             60 non-null     int64
 3   SkinThickness             60 non-null     int64
 4   Insulin                   60 non-null     int64
 5   BMI                       60 non-null     float64
 6   DiabetesPedigreeFunction  60 non-null     float64
 7   Age                       60 non-null     int64
 8   Outcome                   60 non-null     int64
dtypes: float64(2), int64(7)
memory usage: 4.3 KB
None

# Statistical summary
print(diabetes_df.describe())
print("\nMissing values in each column:")
print(diabetes_df.isnull().sum())

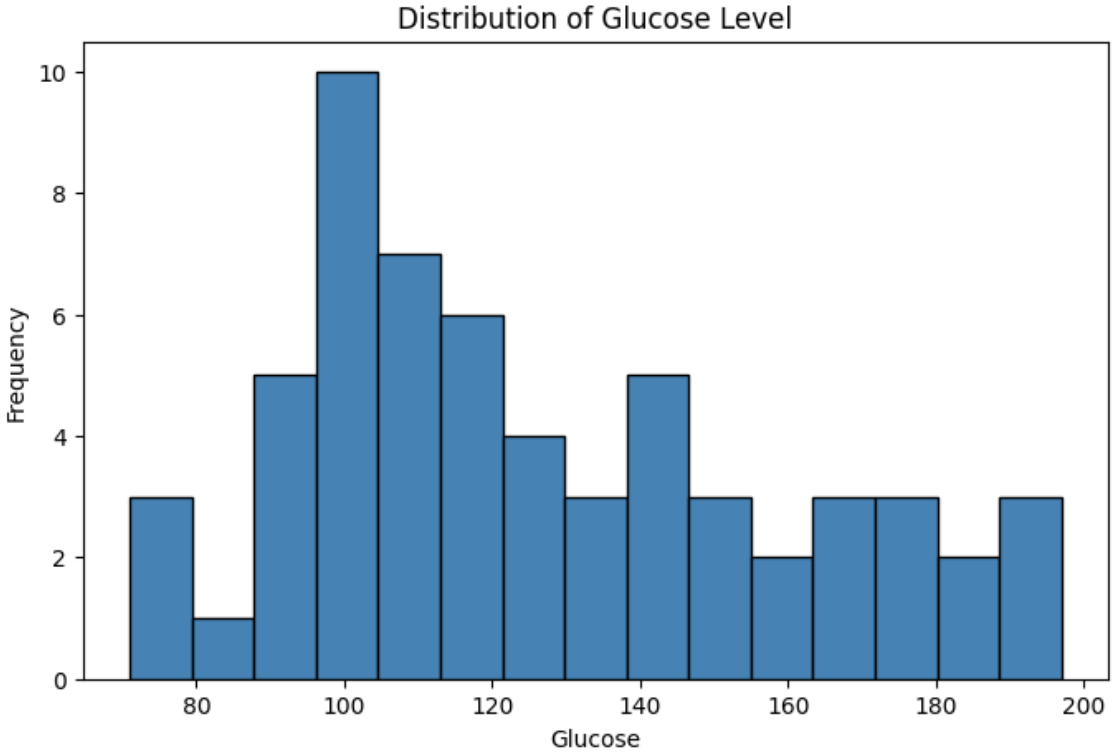
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count    60.000000   60.000000      60.000000      60.000000   60.000000
mean      5.016667  126.633333      69.383333      20.433333   90.266667
std       3.567707   33.410121      21.347085      17.178393  150.306339
min       0.000000   71.000000       0.000000       0.000000    0.000000
25%       1.000000  102.750000      64.000000       0.000000    0.000000
50%       5.000000  117.500000      72.000000      23.000000    0.000000
75%       7.250000  147.250000      82.000000      35.000000  140.500000
max      13.000000  197.000000     110.000000      60.000000  846.000000

             BMI  DiabetesPedigreeFunction        Age    Outcome
count  60.000000                 60.000000  60.000000  60.000000
mean   31.480000                  0.533383  36.683333   0.450000
std     9.082742                  0.433882  11.598497   0.501692
min     0.000000                  0.134000  21.000000   0.000000
25%    26.400000                  0.254000  28.000000   0.000000
50%    32.250000                  0.435500  32.500000   0.000000
75%    37.775000                  0.597000  45.250000   1.000000
max    46.800000                  2.288000  60.000000   1.000000

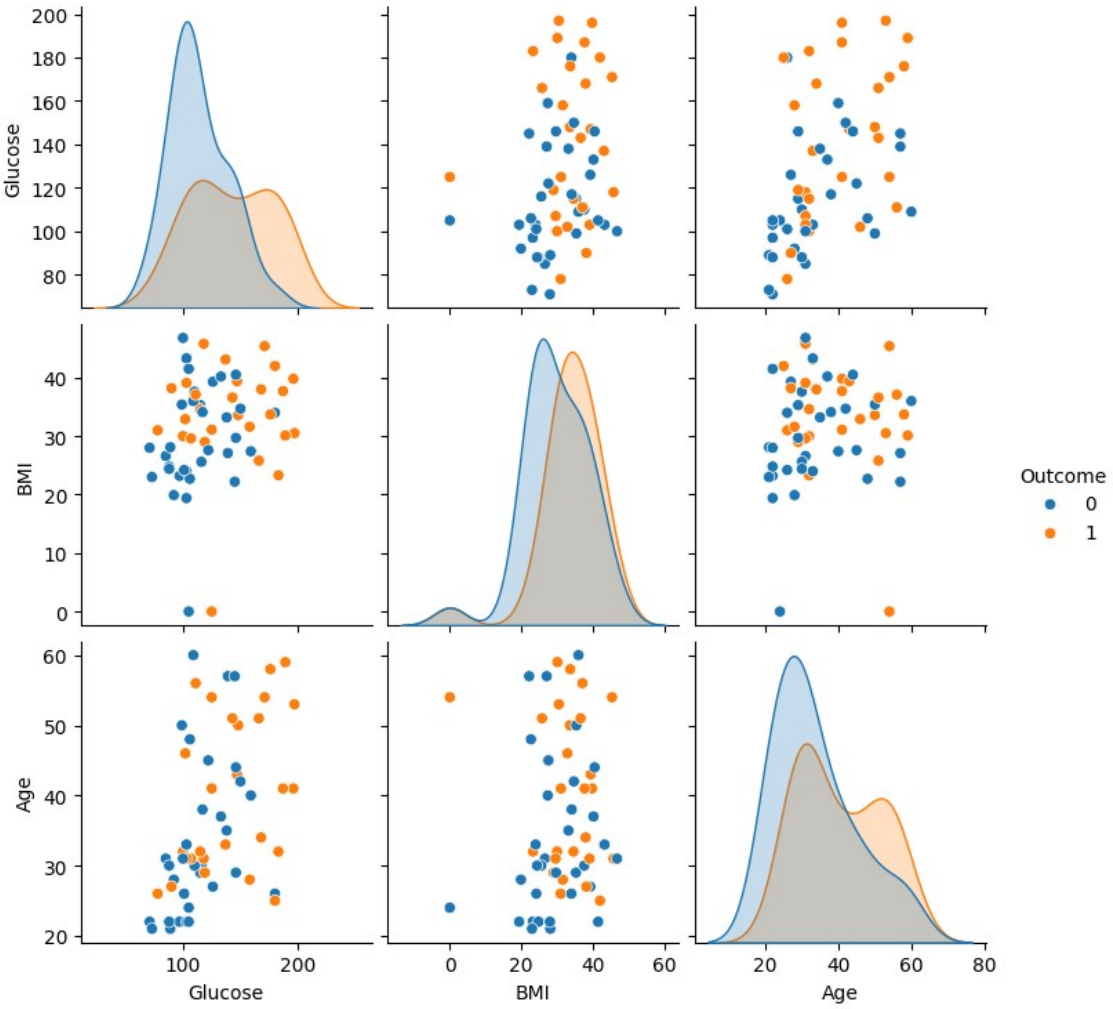
Missing values in each column:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
```

```
dtype: int64
```

```
# Distribution of Glucose
plt.figure(figsize=(8, 5))
plt.hist(diabetes_df['Glucose'], bins=15, color='steelblue', edgecolor='black')
plt.title('Distribution of Glucose Level')
plt.xlabel('Glucose')
plt.ylabel('Frequency')
plt.show()
```




```
# Pairplot of important features with the outcome
sns.pairplot(diabetes_df[['Glucose', 'BMI', 'Age', 'Outcome']], hue='Outcome')
plt.show()
```



RESULT:

Thus the diabetes data set was collected and initial exploration was performed to understand its structure, statistics and relationships.

EXPERIMENT NO: 6
TIME SERIES ANALYSIS OF WEBSITE TRAFFIC.

AIM:
To analyze the time series data of website traffic to identify trends, seasonality and anomalies.

PROGRAM:

```
%pip install -q statsmodels
%matplotlib inline
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

page_views = [
    120, 135, 128, 140, 150, 145, 160, 155, 170, 165,
    180, 175, 190, 185, 200, 195, 210, 205, 220, 215,
    230, 225, 240, 235, 250, 245, 260, 255, 270, 265,
    280, 275, 290, 285, 300, 295, 310, 305, 320, 315,
    330, 325, 340, 335, 750, 345, 360, 355, 370, 365,
    380, 375, 390, 385, 400, 395, 410, 405, 420, 415,
    430, 425, 440, 435, 450, 445, 460, 455, 470, 465,
    480, 475, 490, 485, 500, 495, 510, 505, 520, 515,
    530, 525, 540, 535, 550, 545, 560, 555, 570, 565
]

data = pd.DataFrame({
    'Date': pd.date_range(start='2025-01-01', periods=90, freq='D'),
    'Page_Views': page_views,
    'Unique_Visitors': [int(v * 0.7) for v in page_views],
    'Bounce_Rate': [round(35 + (v % 10), 1) for v in page_views]
})
data.to_csv('website_traffic.csv', index=False)
print("website_traffic.csv created successfully")
```

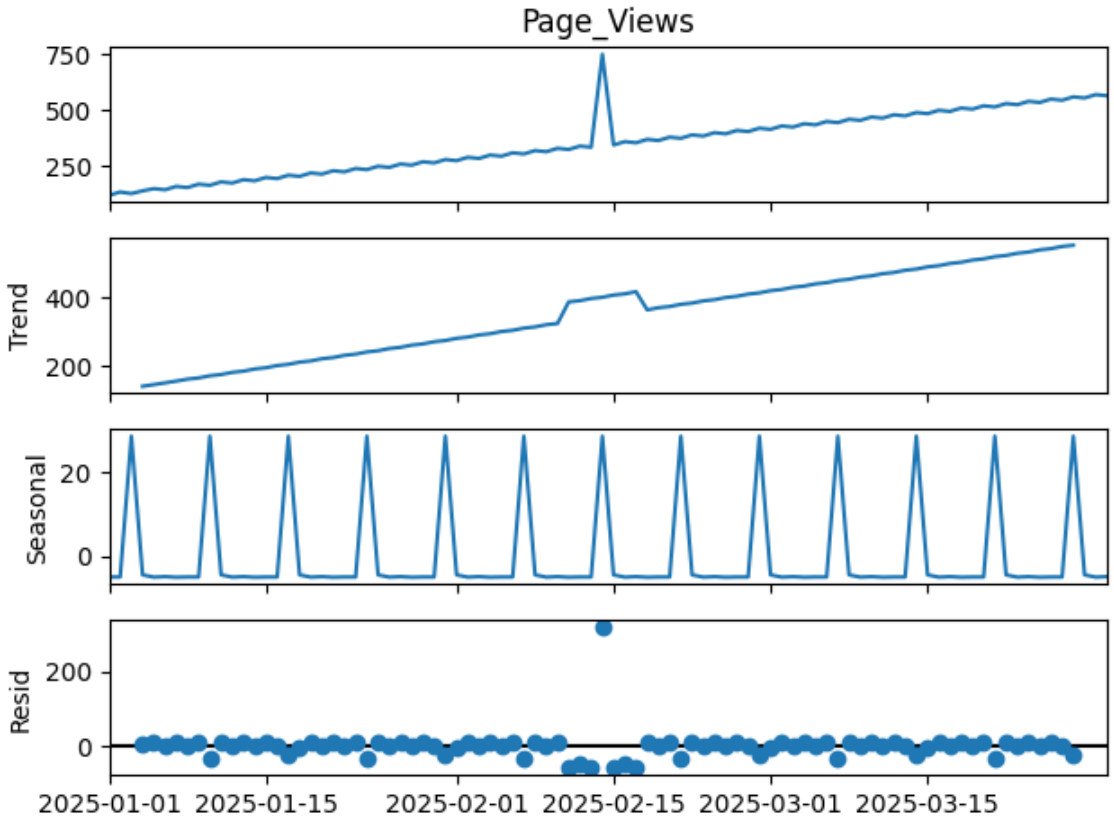
OUTPUT:

```
Note: you may need to restart the kernel to use updated packages.
website_traffic.csv created successfully

# Load dataset
data = pd.read_csv('website_traffic.csv', parse_dates=['Date'],
index_col='Date')
print(data.head())

# Time Series Decomposition
decomposition = seasonal_decompose(data['Page_Views'], model='additive',
period=7)
decomposition.plot()
plt.show()

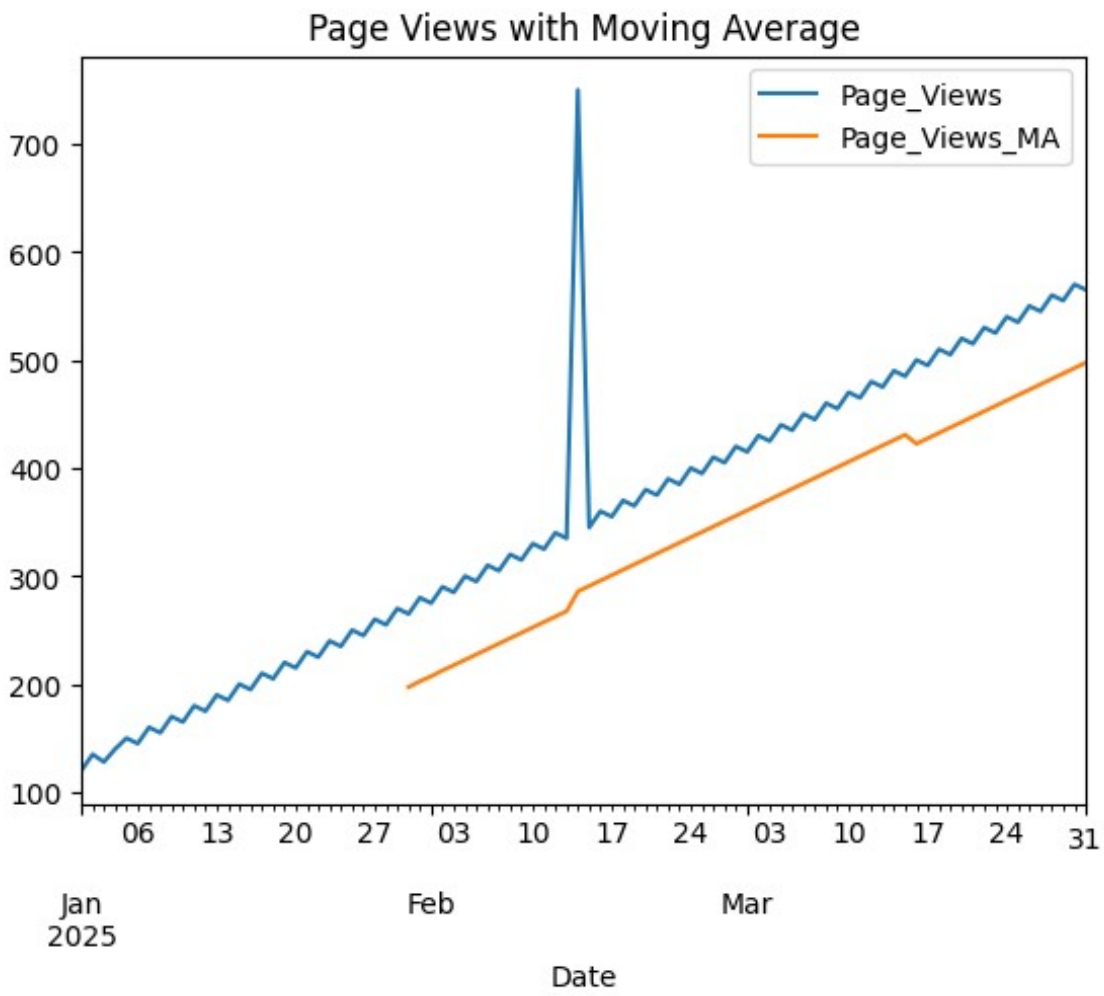
            Page_Views  Unique_Visitors  Bounce_Rate
Date
2025-01-01         120               84           35
2025-01-02         135               94           40
2025-01-03         128               89           43
2025-01-04         140               98           35
2025-01-05         150              105           35
```

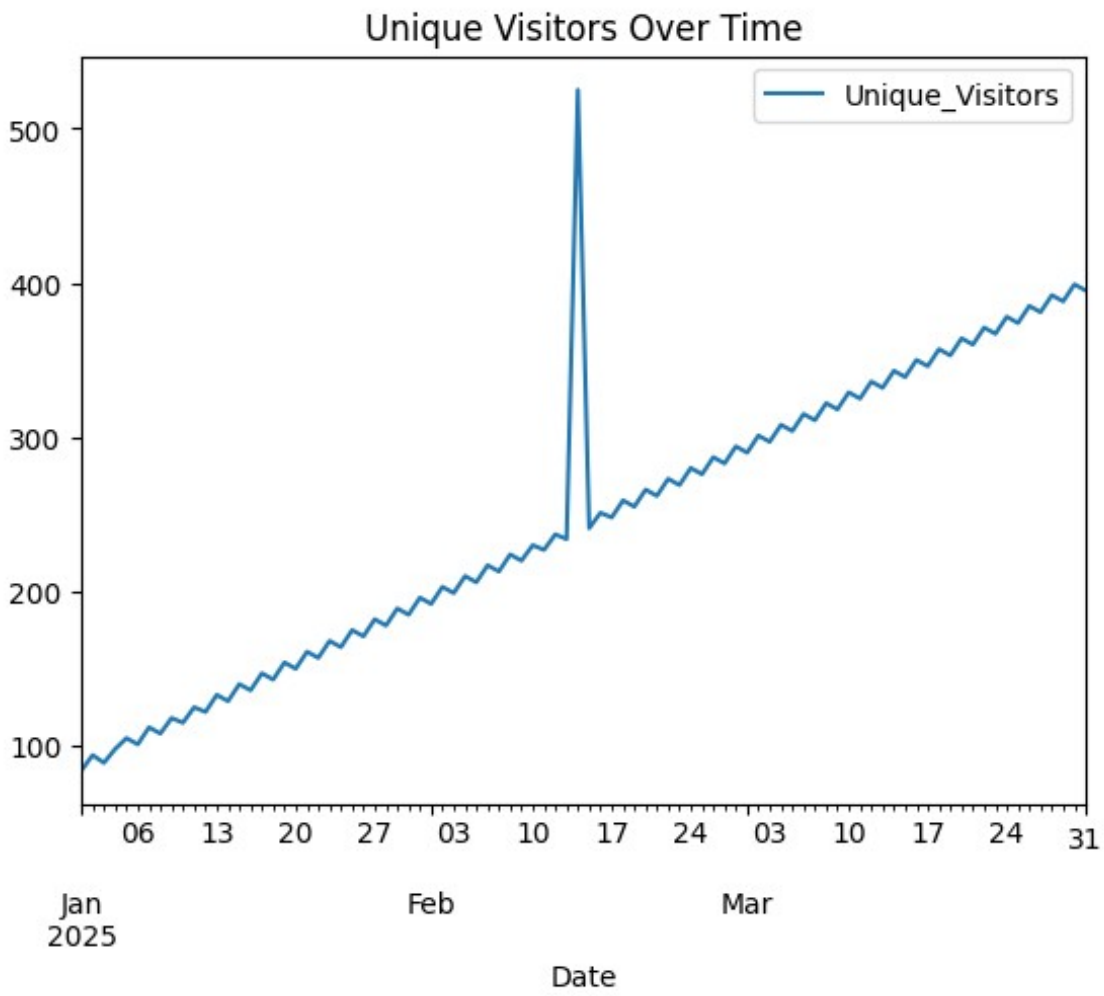


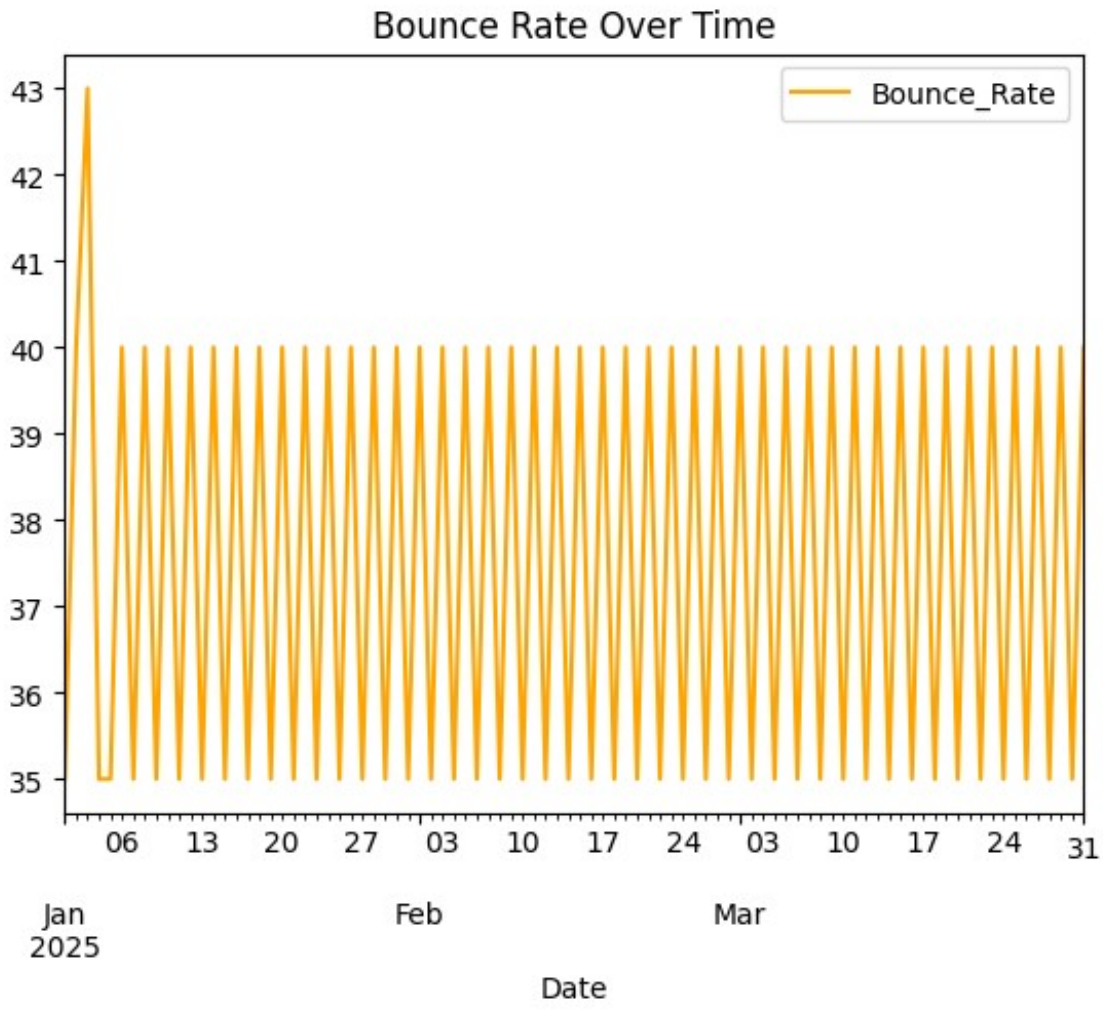

```
# Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(window=30).mean()
data[['Page_Views', 'Page_Views_MA']].plot()
plt.title('Page Views with Moving Average')
plt.show()

# Plot Unique_Visitors and Bounce_Rate
data[['Unique_Visitors']].plot()
plt.title('Unique Visitors Over Time')
plt.show()

data[['Bounce_Rate']].plot(color='orange')
plt.title('Bounce Rate Over Time')
plt.show()
```


Page Views with Moving Average
Page_Views
Page_Views_MA
700
600
500
400
300
200
100
06
13
20
27
03
10
17
24
03
10
17
24
31
Jan
2025
Feb
Mar
Date

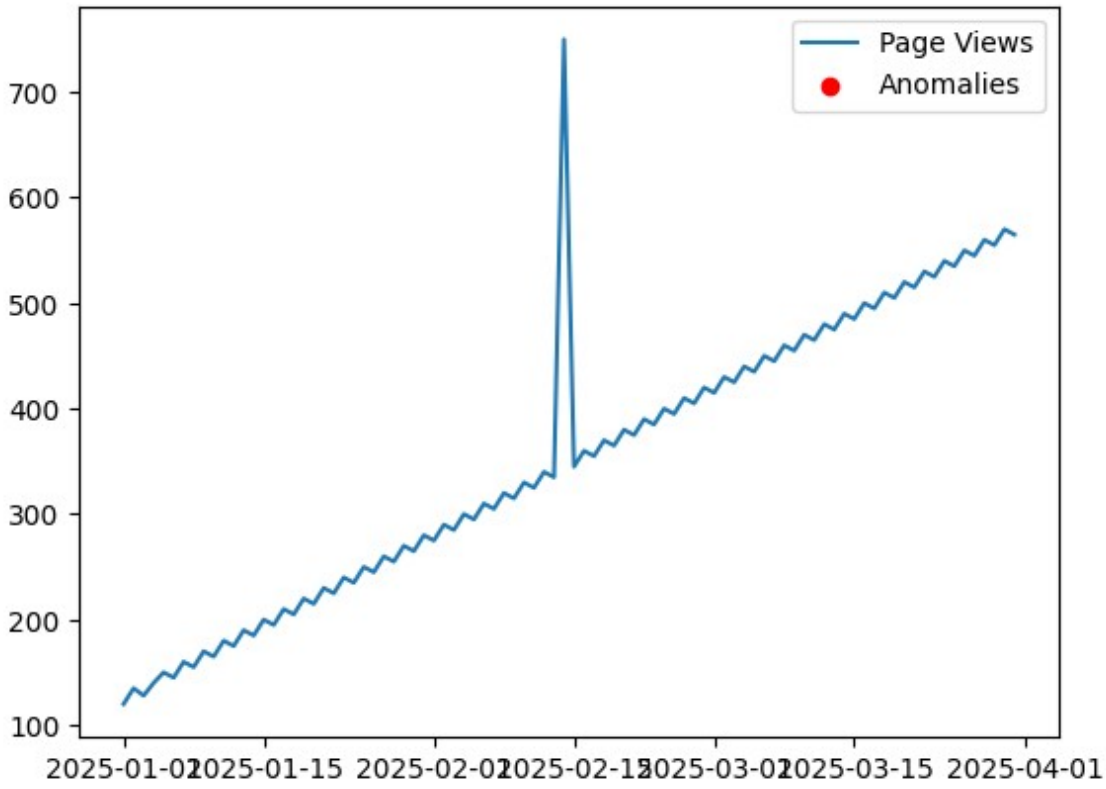

Unique Visitors Over Time
Unique_Visitors
500
400
300
200
100
06
13
20
27
03
10
17
24
03
10
17
24
31
Jan
2025
Feb
Mar
Date




```
# Anomaly Detection using Z-score
data['Page_Views_Z'] = (data['Page_Views'] - data['Page_Views'].mean()) /
data['Page_Views'].std()
anomalies = data[data['Page_Views_Z'].abs() > 3]
print("Anomalies detected:")
print(anomalies[['Page_Views', 'Page_Views_Z']])

plt.plot(data.index, data['Page_Views'], label='Page Views')
plt.scatter(anomalies.index, anomalies['Page_Views'], color='red',
label='Anomalies')
plt.legend()
plt.show()
```

```
Anomalies detected:
Empty DataFrame
Columns: [Page_Views, Page_Views_Z]
Index: []
```



RESULT:

Thus the website traffic time series was analyzed; trend and seasonality were identified using decomposition and moving average, and anomalies were detected using the Z-score method.

EXPERIMENT NO: 7
RANDOM SAMPLING AND SAMPLING DISTRIBUTION.

AIM:
To explore random sampling from a population and understand the concept of sampling distribution using Python.

PROGRAM:

```
%matplotlib inline
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)

# Step 1: Generate a population (e.g., normal distribution)
population_mean = 50
population_std = 10
population_size = 100000
population = np.random.normal(population_mean, population_std, population_size)
print("Population size:", len(population))
print("Population mean:", round(np.mean(population), 2))
print("Population standard deviation:", round(np.std(population), 2))
```

OUTPUT:

```
Population size: 100000
Population mean: 50.01
Population standard deviation: 10.01

# Step 2: Random sampling
sample_sizes = [30, 50, 100]  # different sample sizes to consider
num_samples = 1000  # number of samples for each sample size

sample_means = {}
for size in sample_sizes:
    sample_means[size] = []
    for _ in range(num_samples):
        sample = np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

print("First 5 sample means for sample size 30:", np.round(sample_means[30]
[:5], 2))

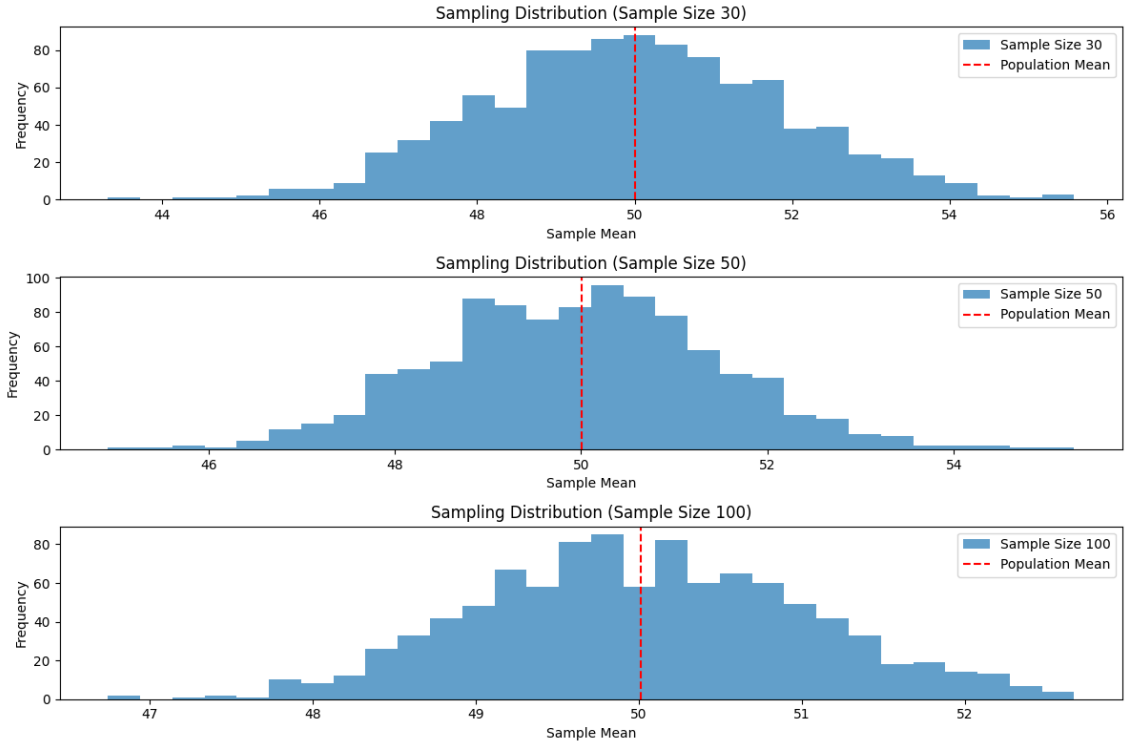
First 5 sample means for sample size 30: [52.15 49.65 50.2  49.58 46.36]

# Step 3: Plotting sampling distributions
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
    plt.subplot(len(sample_sizes), 1, i + 1)
    plt.hist(sample_means[size], bins=30, alpha=0.7, label=f'Sample Size
{size}')
    plt.axvline(np.mean(population), color='red', linestyle='dashed',
                linewidth=1.5, label='Population Mean')
    plt.title(f'Sampling Distribution (Sample Size {size})')
    plt.xlabel('Sample Mean')
    plt.ylabel('Frequency')
    plt.legend()

plt.tight_layout()
```

```
plt.show()
```



RESULT:

Thus random sampling was performed and the sampling distribution of the sample mean was studied; as the sample size increases the sampling distribution approaches a normal distribution centred at the population mean (Central Limit Theorem).

EXPERIMENT NO: 8
CONDUCT Z TEST FOR THE DATA GIVEN USING PYTHON.

AIM:

To test whether the average weight of a species of birds differs from 150 grams using a Z test.

PROGRAM:

Hypotheses

- Null Hypothesis (H0): The average weight of the birds is 150 grams.
- Alternative Hypothesis (H1): The average weight of the birds is not 150 grams.
- Sample: Weights of 30 birds randomly selected from the population.
- Decision rule: significance level alpha = 0.05.

```
%pip install -q scipy
import numpy as np
import scipy.stats as stats

# Define the sample data (hypothetical weights in grams)
sample_data = np.array([152, 148, 151, 149, 147, 153, 150, 148, 152, 149,
                        151, 150, 149, 152, 151, 148, 150, 152, 149, 150,
                        148, 153, 151, 150, 149, 152, 148, 151, 150, 153])

# Population mean under the null hypothesis
population_mean = 150

# Calculate sample statistics
sample_mean = np.mean(sample_data)
sample_std = np.std(sample_data, ddof=1)  # Using sample standard deviation

# Number of observations
n = len(sample_data)

# Calculate the Z-statistic
z_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))

# Calculate the p-value
p_value = 2 * (1 - stats.norm.cdf(np.abs(z_statistic)))  # Two-tailed test

# Print results
print(f"Sample Mean: {sample_mean:.2f}")
print(f"Z-Statistic: {z_statistic:.4f}")
print(f"P-Value: {p_value:.4f}")

# Decision based on the significance level
alpha = 0.05
if p_value < alpha:
    print("Reject the null hypothesis: The average weight is significantly
different from 150 grams.")
else:
    print("Fail to reject the null hypothesis: There is no significant
difference in average weight from 150 grams.")
```

OUTPUT:

```
Note: you may need to restart the kernel to use updated packages.
Sample Mean: 150.20
Z-Statistic: 0.6406
P-Value: 0.5218
Fail to reject the null hypothesis: There is no significant difference in
average weight from 150 grams.
```

RESULT:

Thus the Z test was conducted for the given data; since the p-value (0.52) is greater than 0.05, we fail to reject the null hypothesis - there is no significant difference in average weight from 150 grams.

EXPERIMENT NO: 9

EXPERIMENT TO UNDERSTAND MATPLOTLIB LIBRARY USE CASES IN DATA SCIENCE THROUGH VISUALIZATION.

AIM:

To use the Iris data set to understand the Matplotlib library use cases in Data Science through visualization.

PROGRAM:

```
%pip install -q scikit-learn seaborn
%matplotlib inline
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

iris = load_iris()
data = pd.DataFrame(iris.data, columns=['SepalLengthCm', 'SepalWidthCm',
                                        'PetalLengthCm', 'PetalWidthCm'])
data.insert(0, 'Id', range(1, len(data) + 1))
data['variety'] = ['Iris-setosa' if t == 0 else
                   'Iris-versicolor' if t == 1 else 'Iris-virginica'
                   for t in iris.target]
data
```

OUTPUT:

```
Note: you may need to restart the kernel to use updated packages.
      Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
0      1            5.1           3.5            1.4           0.2
1      2            4.9           3.0            1.4           0.2
2      3            4.7           3.2            1.3           0.2
3      4            4.6           3.1            1.5           0.2
4      5            5.0           3.6            1.4           0.2
..   ...            ...           ...            ...           ...
145  146            6.7           3.0            5.2           2.3
146  147            6.3           2.5            5.0           1.9
147  148            6.5           3.0            5.2           2.0
148  149            6.2           3.4            5.4           2.3
149  150            5.9           3.0            5.1           1.8

            variety
0       Iris-setosa
1       Iris-setosa
2       Iris-setosa
3       Iris-setosa
4       Iris-setosa
..              ...
145  Iris-virginica
146  Iris-virginica
147  Iris-virginica
148  Iris-virginica
149  Iris-virginica

[150 rows x 6 columns]

data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
```

```
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   Id             150 non-null    int64
 1   SepalLengthCm  150 non-null    float64
 2   SepalWidthCm   150 non-null    float64
 3   PetalLengthCm  150 non-null    float64
 4   PetalWidthCm   150 non-null    float64
 5   variety        150 non-null    object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

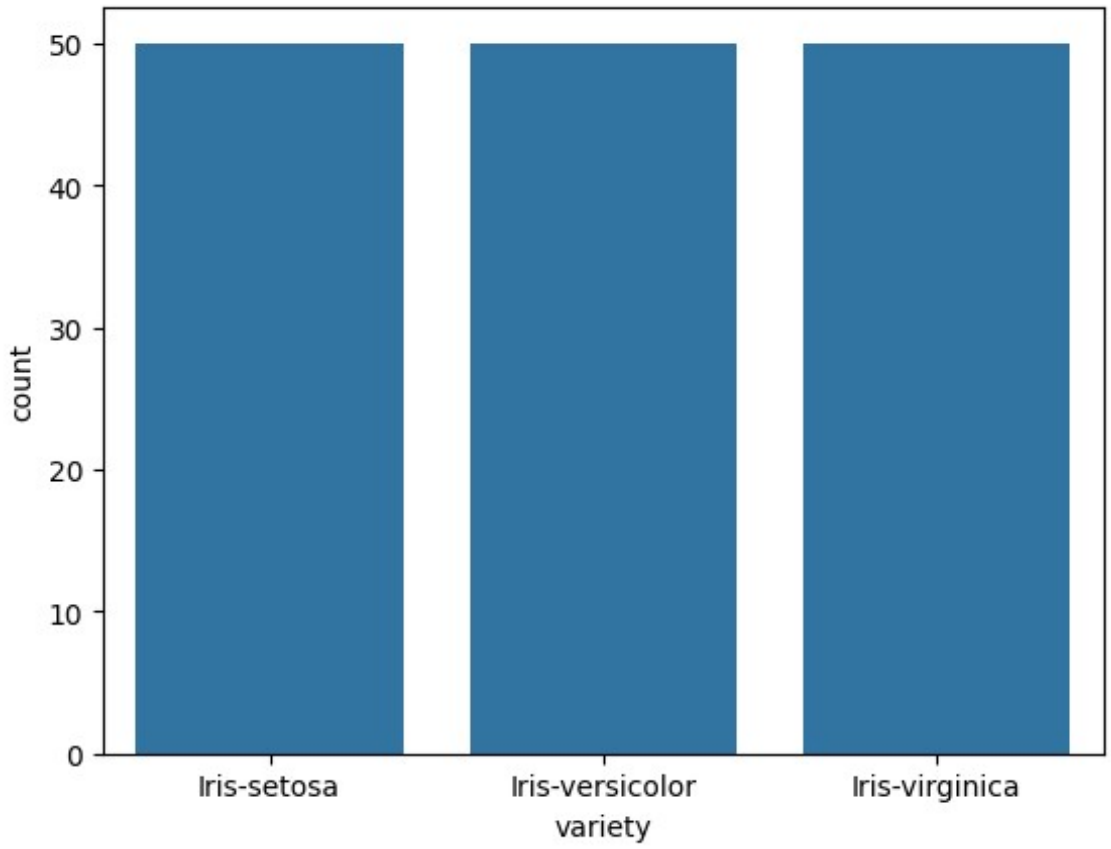
```
data.describe()
```

	Id	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	75.500000	5.843333	3.057333	3.758000	1.199333
std	43.445368	0.828066	0.435866	1.765298	0.762238
min	1.000000	4.300000	2.000000	1.000000	0.100000
25%	38.250000	5.100000	2.800000	1.600000	0.300000
50%	75.500000	5.800000	3.000000	4.350000	1.300000
75%	112.750000	6.400000	3.300000	5.100000	1.800000
max	150.000000	7.900000	4.400000	6.900000	2.500000

```
data.value_counts('variety')
```

```
variety
Iris-setosa        50
Iris-versicolor    50
Iris-virginica     50
Name: count, dtype: int64
```

```
sns.countplot(x='variety', data=data)
plt.show()
```

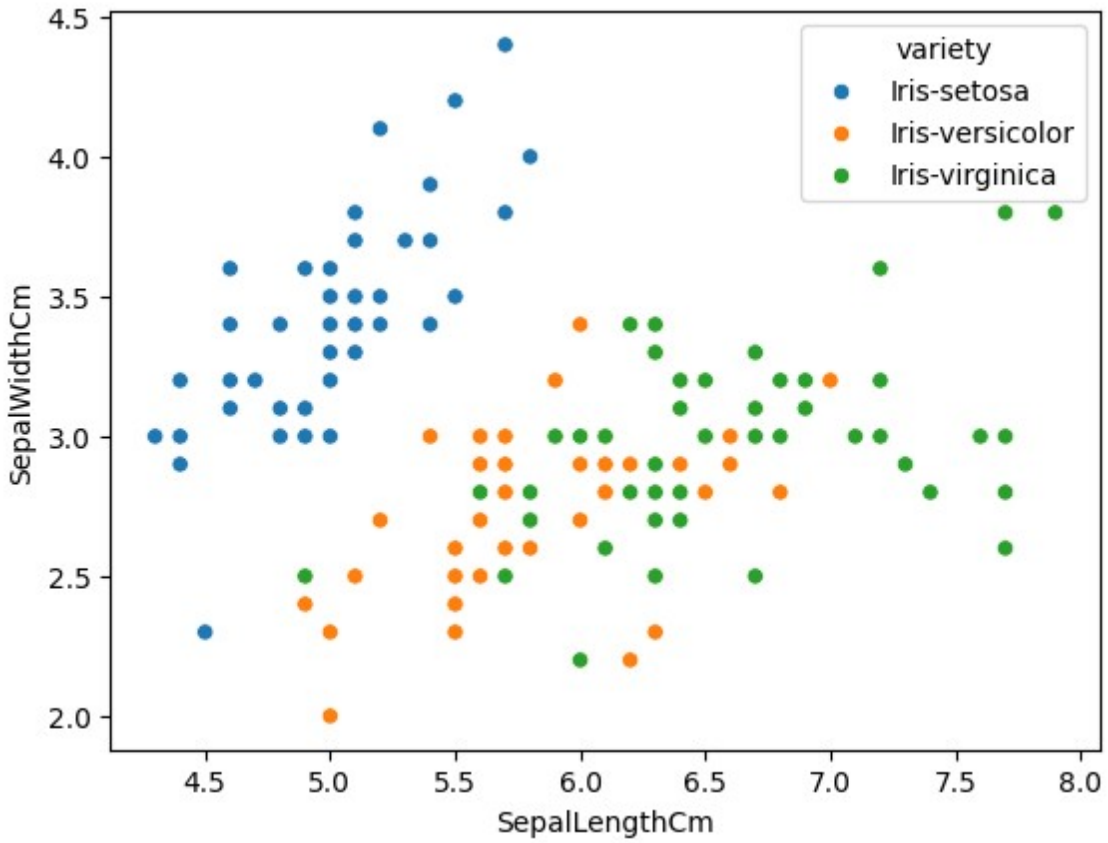



```
dummies = pd.get_dummies(data.variety)
FinalDataset = pd.concat([pd.get_dummies(data.variety), data.iloc[:, [0, 1, 2,
3]]], axis=1)
FinalDataset.head()
```

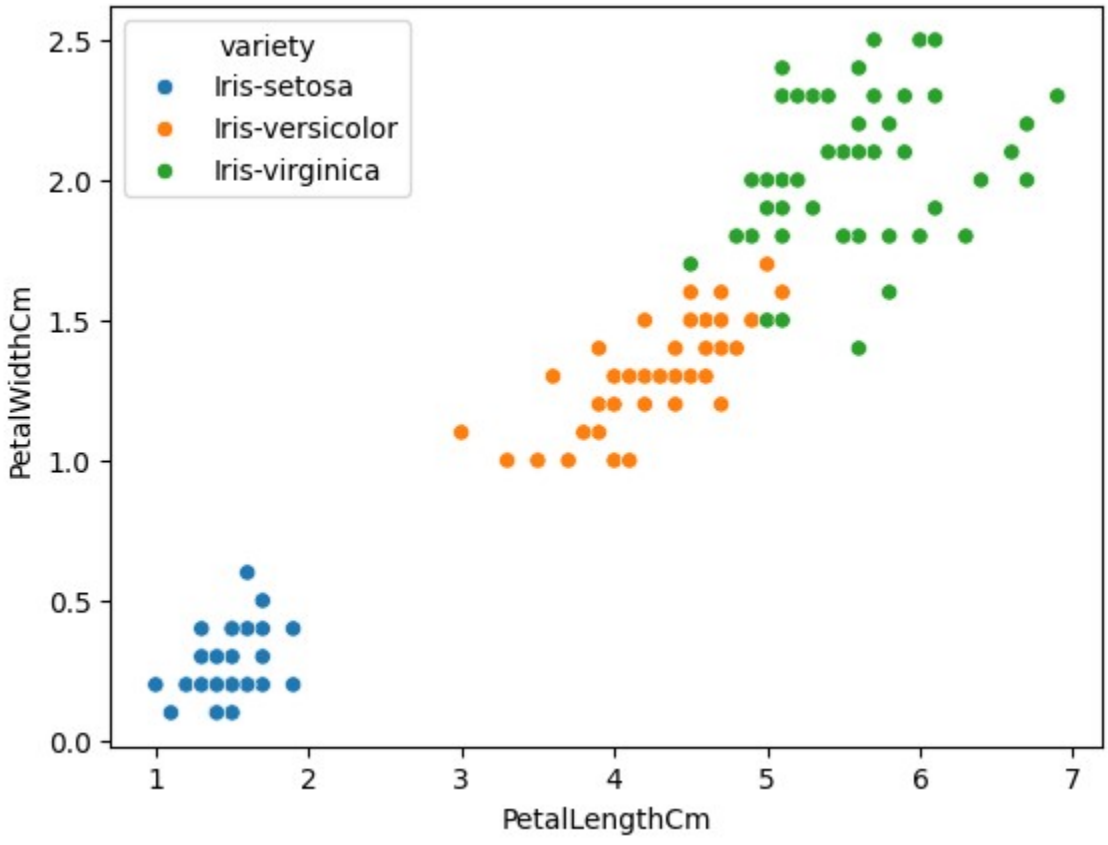
```
   Iris-setosa  Iris-versicolor  Iris-virginica  Id  SepalLengthCm  \
0         True            False           False   1            5.1
1         True            False           False   2            4.9
2         True            False           False   3            4.7
3         True            False           False   4            4.6
4         True            False           False   5            5.0

   SepalWidthCm  PetalLengthCm
0           3.5            1.4
1           3.0            1.4
2           3.2            1.3
3           3.1            1.5
4           3.6            1.4
```

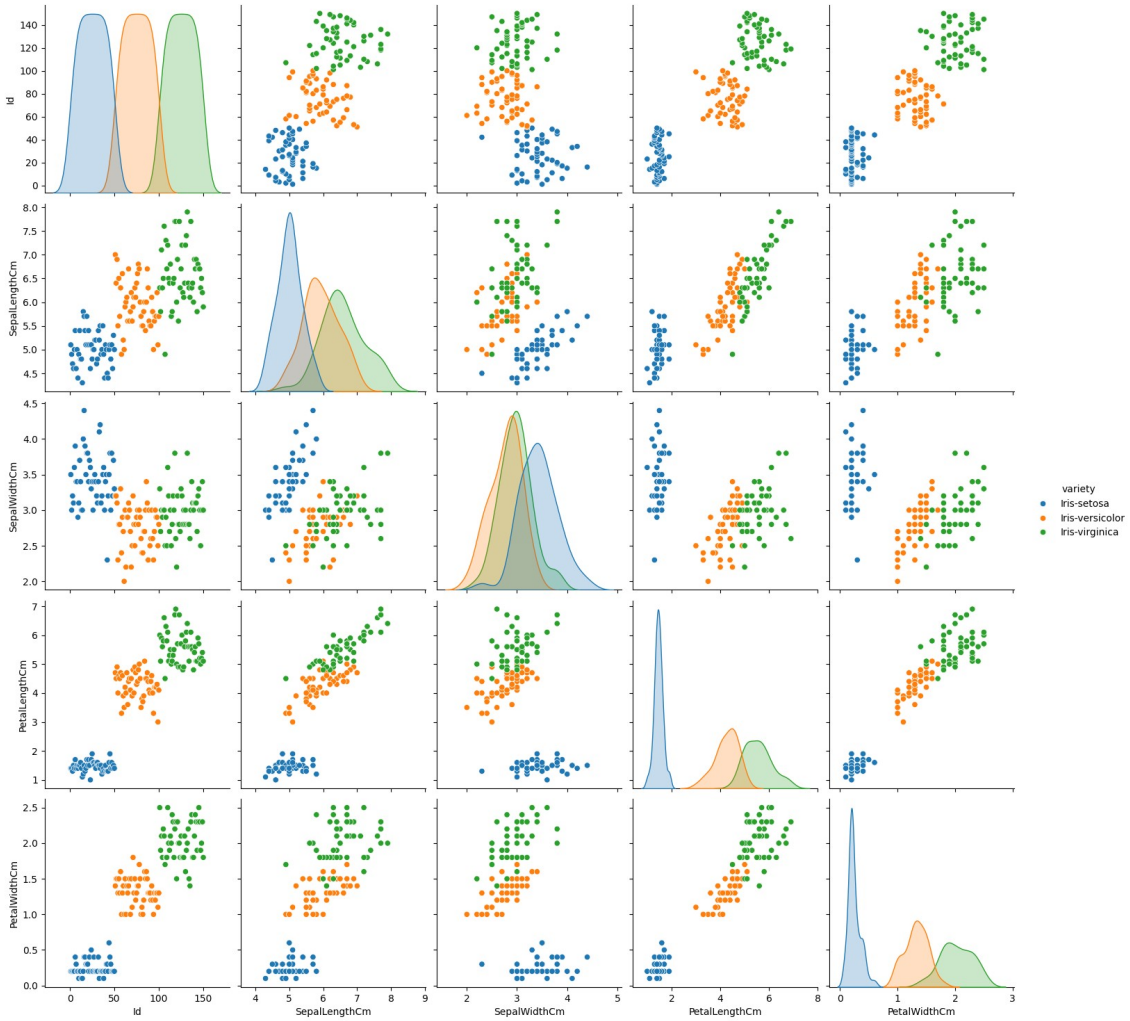
```
sns.scatterplot(x='SepalLengthCm', y='SepalWidthCm', hue='variety', data=data)
plt.show()
```




```
sns.scatterplot(x='PetalLengthCm', y='PetalWidthCm', hue='variety', data=data)
plt.show()
```



```
sns.pairplot(data, hue='variety', height=3)
plt.show()
```

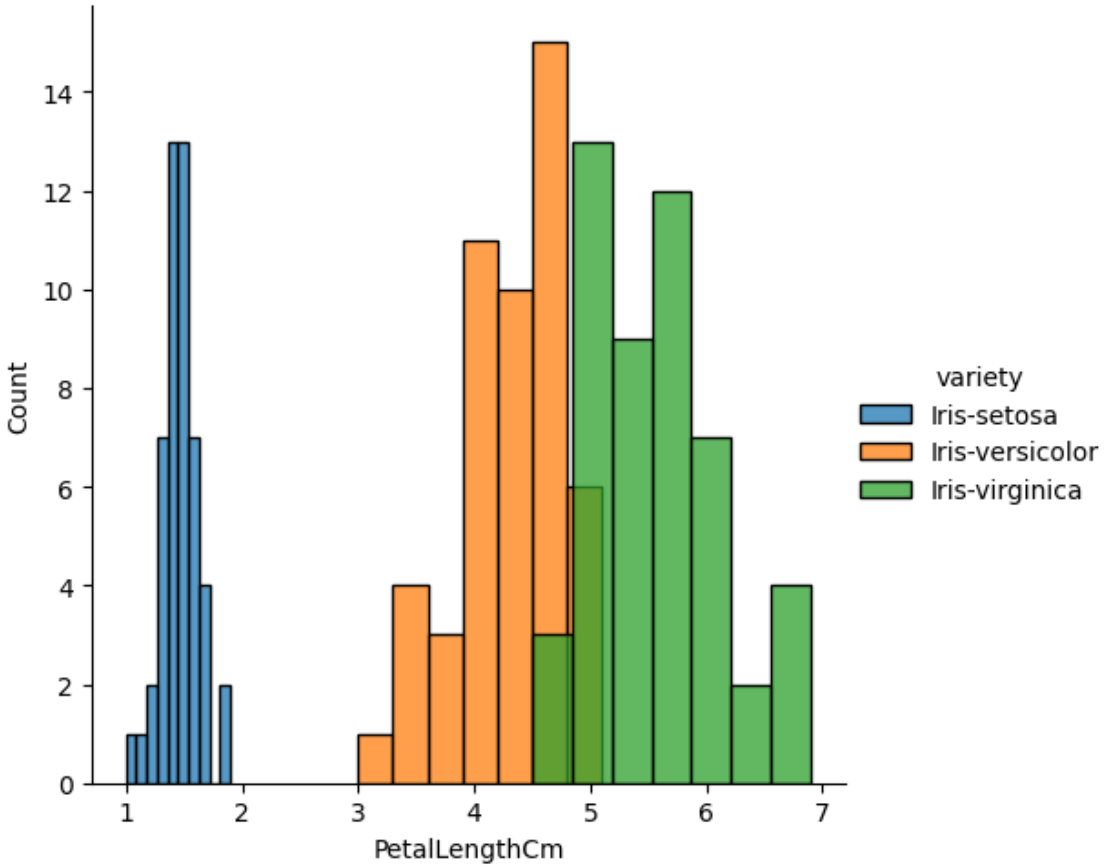


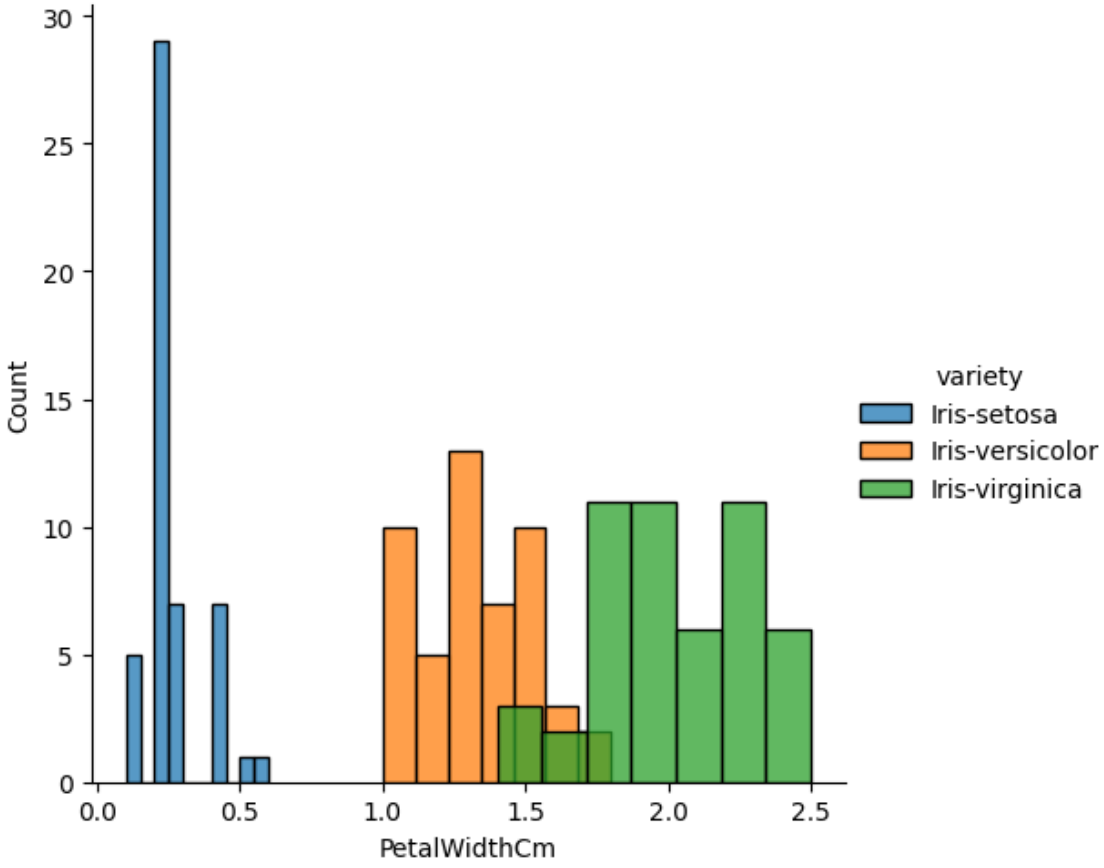

```
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'PetalLengthCm').add_legend()
plt.show()

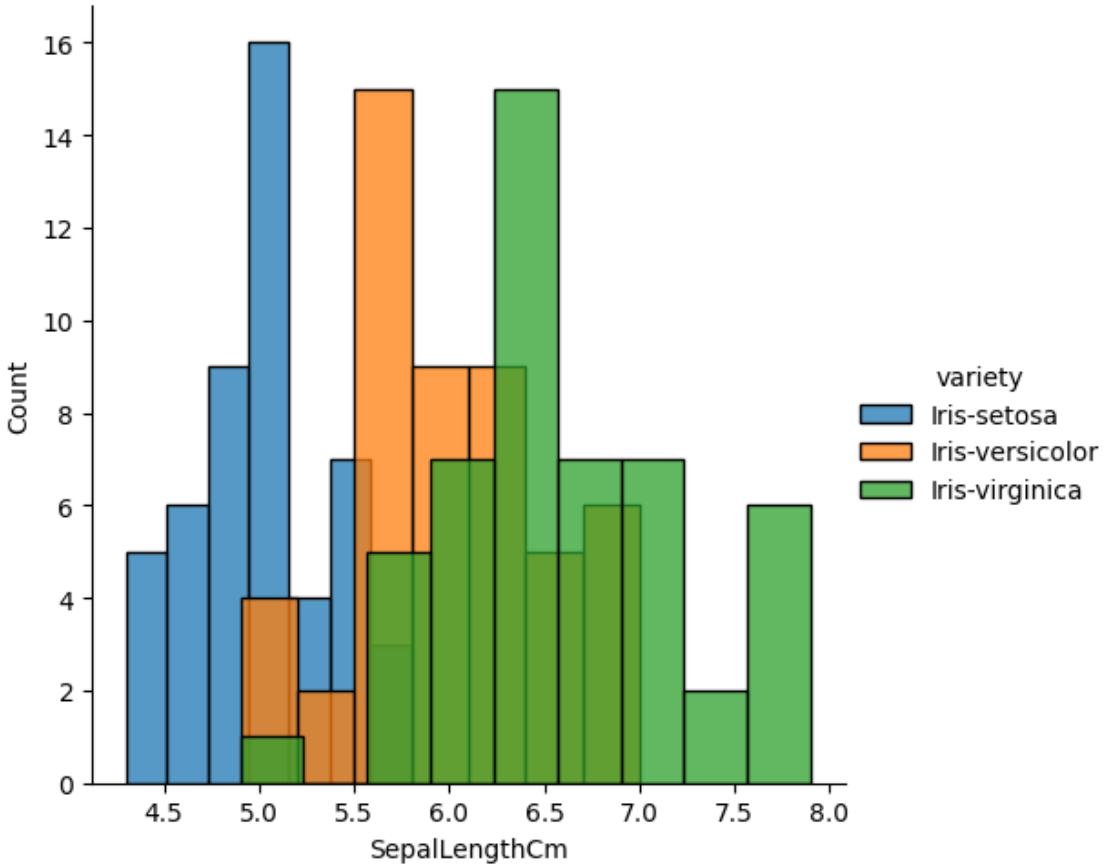
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'PetalWidthCm').add_legend()
plt.show()

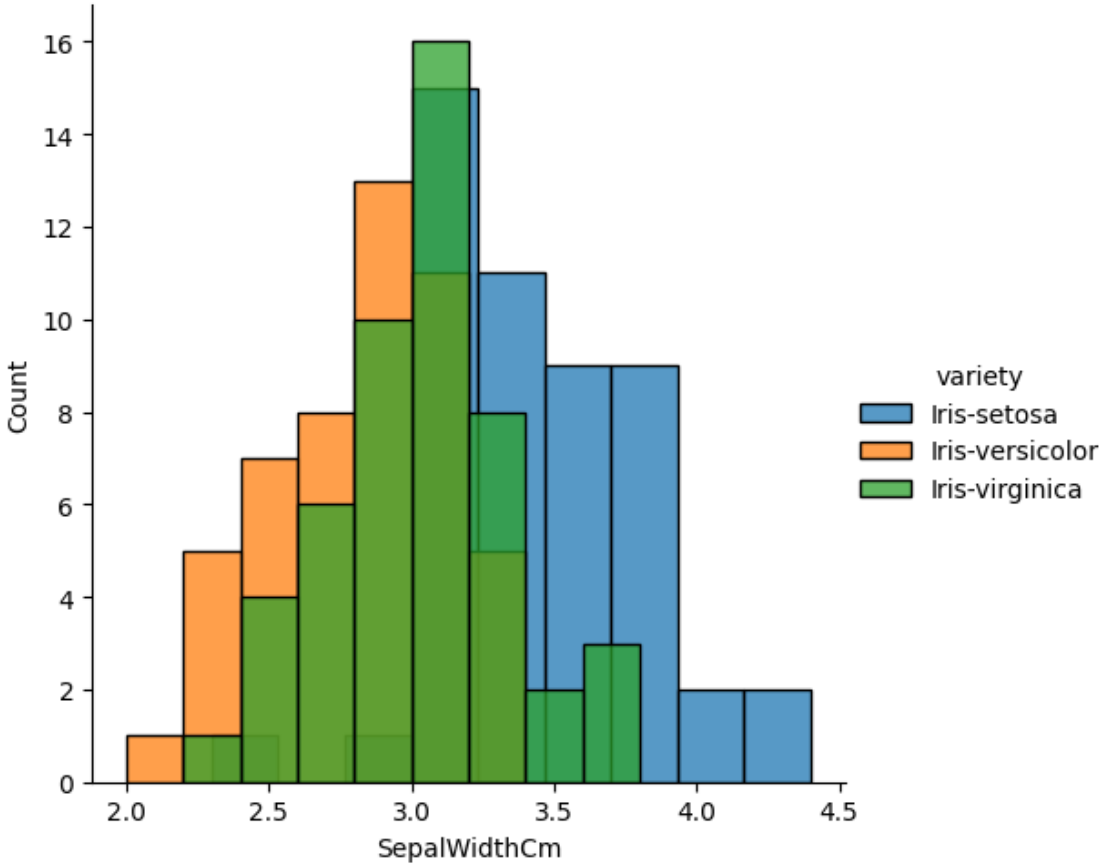
sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'SepalLengthCm').add_legend()
plt.show()

sns.FacetGrid(data, hue='variety', height=5).map(sns.histplot,
'SepalWidthCm').add_legend()
plt.show()
```


Count
PetalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
PetalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
SepalLengthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica


Count
SepalWidthCm
variety
Iris-setosa
Iris-versicolor
Iris-virginica

RESULT:

Thus the Matplotlib and Seaborn library use cases in Data Science were understood through count plot, scatter plot, pair plot and histogram visualizations of the Iris data set.

EXPERIMENT NO: 10

EXPERIMENT TO UNDERSTAND ARRAY FUNCTION IN DATA SCIENCE.

AIM:

To understand array functions in Data science using the NumPy library.

PROGRAM:

```
import numpy as np

np.random.seed(42)
array = np.random.randint(1, 100, 9)  # randomly generate 9 numbers between 1
to 100
array
```

OUTPUT:

```
array([52, 93, 15, 72, 61, 21, 83, 87, 75])

np.sqrt(array)

array([7.21110255, 9.64365076, 3.87298335, 8.48528137, 7.81024968,
       4.58257569, 9.11043358, 9.32737905, 8.66025404])

array.ndim

1

new_array = array.reshape(3, 3)
new_array

array([[52, 93, 15],
       [72, 61, 21],
       [83, 87, 75]])

new_array.ndim

2

new_array.ravel()

array([52, 93, 15, 72, 61, 21, 83, 87, 75])

newm = new_array.reshape(3, 3)
newm

array([[52, 93, 15],
       [72, 61, 21],
       [83, 87, 75]])

newm[2, 1:3]

array([87, 75])

newm[1:2, 1:3]

array([[61, 21]])

new_array[0:3, 0:0]
```

```
array([], shape=(3, 0), dtype=int64)

new_array[0:2, 0:1]

array([[52],
       [72]])

new_array[0:3, 0:1]

array([[52],
       [72],
       [83]])

new_array[1:3]

array([[72, 61, 21],
       [83, 87, 75]])
```

RESULT:

Thus array functions such as sqrt, ndim, reshape, ravel and array slicing were understood using the NumPy library.

EXPERIMENT NO: 11

EXPERIMENT TO UNDERSTAND PANDAS LIBRARY USE CASES IN DATA SCIENCE.

AIM:

To understand DataFrame use cases in Data science using the pandas library.

PROGRAM:

```
import numpy as np
import pandas as pd

list1 = [[1, 'Smith', 50000], [2, 'Jones', 60000]]
df = pd.DataFrame(list1)
df
```

OUTPUT:

```
   0      1      2
0  1  Smith  50000
1  2  Jones  60000
```

```
df.columns = ['Empd', 'Name', 'Salary']
df
```

```
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      object
 2   Salary  2 non-null      int64
dtypes: int64(2), object(1)
memory usage: 176.0+ bytes
```

```
data = {
    'R&D Spend': [
        165349.2, 162597.7, 153441.51, 144372.41, 142107.34, 131876.9,
134615.46, 130298.13,
        120542.52, 123334.88, 101913.08, 100671.96, 93863.75, 91992.39,
119943.24, 114523.61,
        78013.11, 94657.16, 91749.16, 86419.7, 76253.86, 78389.47, 73994.56,
67532.53,
        77044.01, 64664.71, 75328.87, 72107.6, 66051.52, 65605.48, 61994.48,
61136.38,
        63408.86, 55493.95, 46426.07, 46014.02, 28663.76, 44069.95, 20229.59,
38558.51,
        28754.33, 27892.92, 23640.93, 15505.73, 22177.74, 1000.23, 1315.46,
0.0,
        542.05, 0.0
    ],
    'Administration': [
        136897.8, 151377.59, 101145.55, 118671.85, 91391.77, 99814.71,
147198.87, 145530.06,
```

```
        148718.95, 108679.17, 110594.11, 91790.61, 127320.38, 135495.07,
156547.42, 122616.84,
        121597.55, 145077.58, 114175.79, 153514.11, 113867.3, 153773.43,
122782.75, 105751.03,
        99281.34, 139553.16, 144135.98, 127864.55, 182645.56, 153032.06,
115641.28, 152701.92,
        129219.61, 103057.49, 157693.92, 85047.44, 127056.21, 51283.14,
65947.93, 82982.09,
        118546.05, 84710.77, 96189.63, 127382.3, 154806.14, 124153.04,
115816.21, 135426.92,
        51743.15, 116983.8
    ],
    'Marketing Spend': [
        471784.1, 443898.53, 407934.54, 383199.62, 366168.42, 362861.36,
127716.82, 323876.68,
        311613.29, 304981.62, 229160.95, 249744.55, 249839.44, 252664.93,
256512.92, 261776.23,
        264346.06, 282574.31, 294919.57, 0.0, 298664.47, 299737.29, 303319.26,
304768.73,
        140574.81, 137962.62, 134050.07, 353183.81, 118148.2, 107138.38,
91131.24, 88218.23,
        46085.25, 214634.81, 210797.67, 205517.64, 201126.82, 197029.42,
185265.1, 174999.3,
        172795.67, 164470.71, 148001.11, 35534.17, 28334.72, 1903.93,
297114.46, 0.0,
        0.0, 45173.06
    ],
    'State': [
        'New York', 'California', 'Florida', 'New York', 'Florida', 'New York',
'California', 'Florida',
        'New York', 'California', 'Florida', 'California', 'Florida',
'California', 'Florida', 'New York',
        'California', 'New York', 'Florida', 'New York', 'California', 'New
York', 'Florida', 'Florida',
        'New York', 'California', 'Florida', 'New York', 'Florida', 'New York',
'Florida', 'New York',
        'California', 'Florida', 'California', 'New York', 'Florida',
'California', 'New York', 'California',
        'California', 'Florida', 'California', 'New York', 'California', 'New
York', 'Florida', 'California',
        'New York', 'California'
    ],
    'Profit': [
        192261.83, 191792.06, 191050.39, 182901.99, 166187.94, 156991.12,
156122.51, 155752.6,
        152211.77, 149759.96, 146121.95, 144259.4, 141585.52, 134307.35,
132602.65, 129917.04,
        126992.93, 125370.37, 124266.9, 122776.86, 118474.03, 111313.02,
110352.25, 108733.99,
        108552.04, 107404.34, 105733.54, 105008.31, 103282.38, 101004.64,
99937.59, 97483.56,
        97427.84, 96778.92, 96712.8, 96479.51, 90708.19, 89949.14, 81229.06,
81005.76,
        78239.91, 77798.83, 71498.49, 69758.98, 65200.33, 64926.08, 49490.75,
42559.73,
        35673.41, 14681.4
    ],
}
df = pd.DataFrame(data)
df.to_csv('50_Startups.csv', index=False)

df = pd.read_csv('50_Startups.csv')
df.info()

<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype  
---  ------           --------------  -----  
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object 
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
```

```
df.head()
```

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

```
df.tail()
```

	R&D Spend	Administration	Marketing Spend	State	Profit
45	1000.23	124153.04	1903.93	New York	64926.08
46	1315.46	115816.21	297114.46	Florida	49490.75
47	0.00	135426.92	0.00	California	42559.73
48	542.05	51743.15	0.00	New York	35673.41
49	0.00	116983.80	45173.06	California	14681.40

```
emp_data = {
    'emp id': [1, 2, 3, 4, 5, 6, 7],
    'name': ['SREE VARSSINI K S', 'SREEMATHI B', 'SREYA G',
             'SREYASKARI MULLAPUDI', 'SRI AKASH U G',
             'SRI HARSHAVARDHANAN R', 'SRI HARSHAVARDHANAN R'],
    'salary': [5000, 6000, 7000, 5000, 8000, 3000, 6000]
}
pd.DataFrame(emp_data).to_csv('employee.csv', index=False)

df = pd.read_csv('employee.csv')
df.head()
```

	emp id	name	salary
0	1	SREE VARSSINI K S	5000
1	2	SREEMATHI B	6000
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000

```
df.tail()
```

	emp id	name	salary
2	3	SREYA G	7000
3	4	SREYASKARI MULLAPUDI	5000
4	5	SRI AKASH U G	8000
5	6	SRI HARSHAVARDHANAN R	3000
6	7	SRI HARSHAVARDHANAN R	6000

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
```

```
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      object
 2   salary  7 non-null      int64
dtypes: int64(2), object(1)
memory usage: 296.0+ bytes
```

```
df.salary
```

```
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64
```

```
type(df.salary)
```

```
pandas.core.series.Series
```

```
df.salary.mean()
```

```
np.float64(5714.285714285715)
```

```
df.salary.median()
```

```
np.float64(6000.0)
```

```
df.salary.mode()
```

```
0    5000
1    6000
Name: salary, dtype: int64
```

```
df.salary.var()
```

```
np.float64(2571428.5714285714)
```

```
df.salary.std()
```

```
np.float64(1603.5674514745463)
```

```
df.describe()
```

```
         emp id       salary
count  7.000000     7.000000
mean   4.000000  5714.285714
std    2.160247  1603.567451
min    1.000000  3000.000000
25%    2.500000  5000.000000
50%    4.000000  6000.000000
75%    5.500000  6500.000000
max    7.000000  8000.000000
```

```
df.describe(include='all')
```

```
          emp id                   name       salary
```

```
count   7.000000                       7     7.000000
unique       NaN                       6          NaN
top          NaN  SRI HARSHAVARDHANAN R          NaN
freq         NaN                       2          NaN
mean    4.000000                     NaN  5714.285714
std     2.160247                     NaN  1603.567451
min     1.000000                     NaN  3000.000000
25%     2.500000                     NaN  5000.000000
50%     4.000000                     NaN  6000.000000
75%     5.500000                     NaN  6500.000000
max     7.000000                     NaN  8000.000000
```

```
empCol = df.columns
empCol
```

```
Index(['emp id', 'name', 'salary'], dtype='object')
```

```
emparray = df.values
emparray
```

```
array([[1, 'SREE VARSSINI K S', 5000],
       [2, 'SREEMATHI B', 6000],
       [3, 'SREYA G', 7000],
       [4, 'SREYASKARI MULLAPUDI', 5000],
       [5, 'SRI AKASH U G', 8000],
       [6, 'SRI HARSHAVARDHANAN R', 3000],
       [7, 'SRI HARSHAVARDHANAN R', 6000]], dtype=object)
```

```
employee_DF = pd.DataFrame(emparray, columns=empCol)
employee_DF
```

```
  emp id                   name salary
0      1      SREE VARSSINI K S   5000
1      2            SREEMATHI B   6000
2      3                SREYA G   7000
3      4   SREYASKARI MULLAPUDI   5000
4      5          SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```

RESULT:

Thus DataFrame use cases such as creation, selection, statistical functions and conversion to arrays were understood using the pandas library.

EXPERIMENT NO: 12

EXPERIMENT TO DETECT OUTLIERS IN A GIVEN DATA SET.

AIM:

To understand the procedure to identify the outliers in a given data set.

PROGRAM:

```
# sample calculation for low range (lr), upper range (ur), percentile
import numpy as np

array = np.array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79, 61, 90,
54])
array
```

OUTPUT:

```
array([27, 50, 44,  6, 58, 61, 23, 86, 67, 20, 75,  7, 79, 61, 90, 54])

array.mean()

np.float64(50.5)

np.percentile(array, 25)

np.float64(26.0)

np.percentile(array, 50)

np.float64(56.0)

np.percentile(array, 75)

np.float64(69.0)

np.percentile(array, 100)

np.float64(90.0)

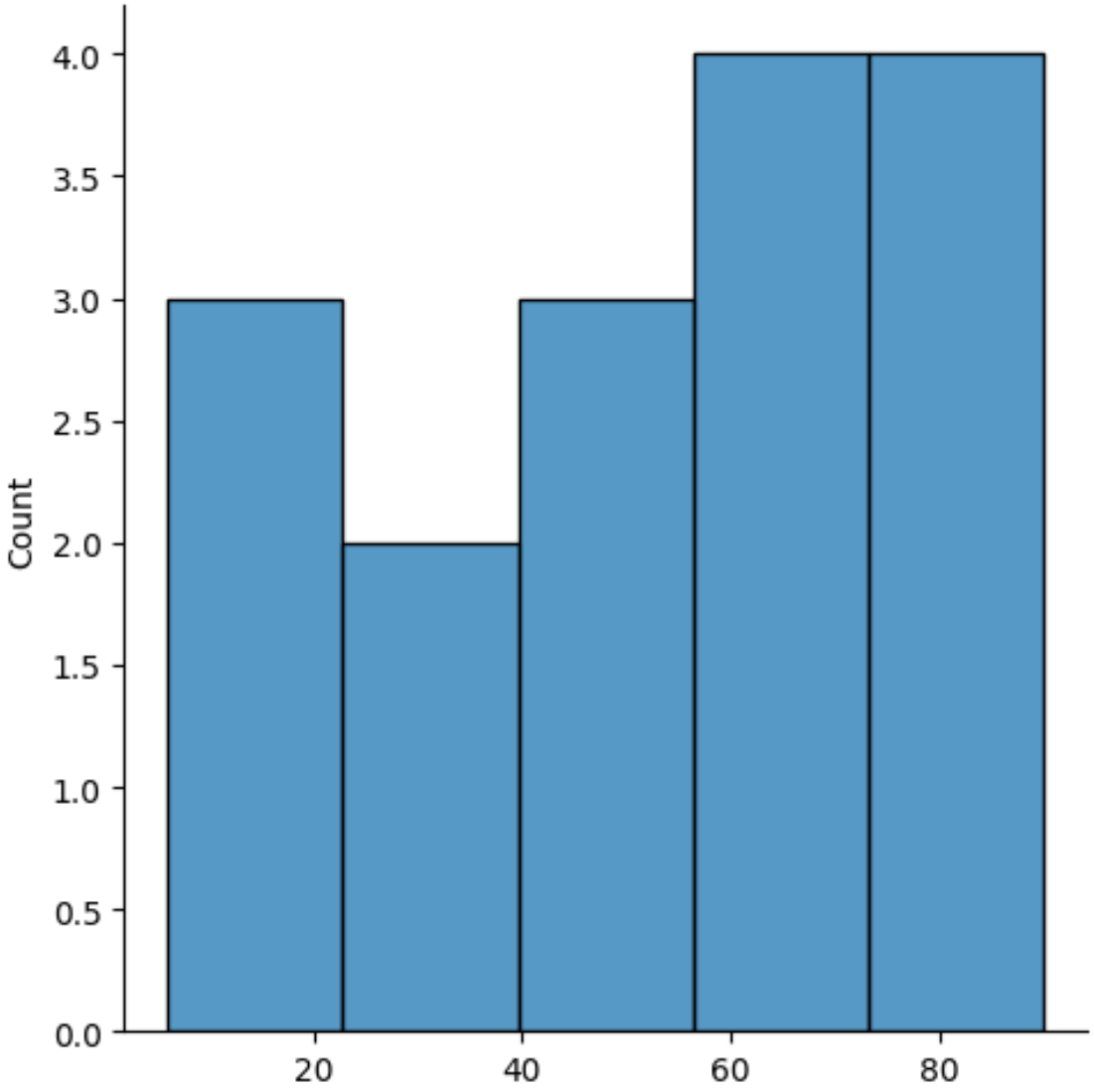
# outliers detection
def outDetection(array):
    sorted(array)
    Q1, Q3 = np.percentile(array, [25, 75])
    IQR = Q3 - Q1
    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)
    return lr, ur

lr, ur = outDetection(array)
lr, ur

(np.float64(-38.5), np.float64(133.5))

%matplotlib inline
import seaborn as sns
import matplotlib.pyplot as plt

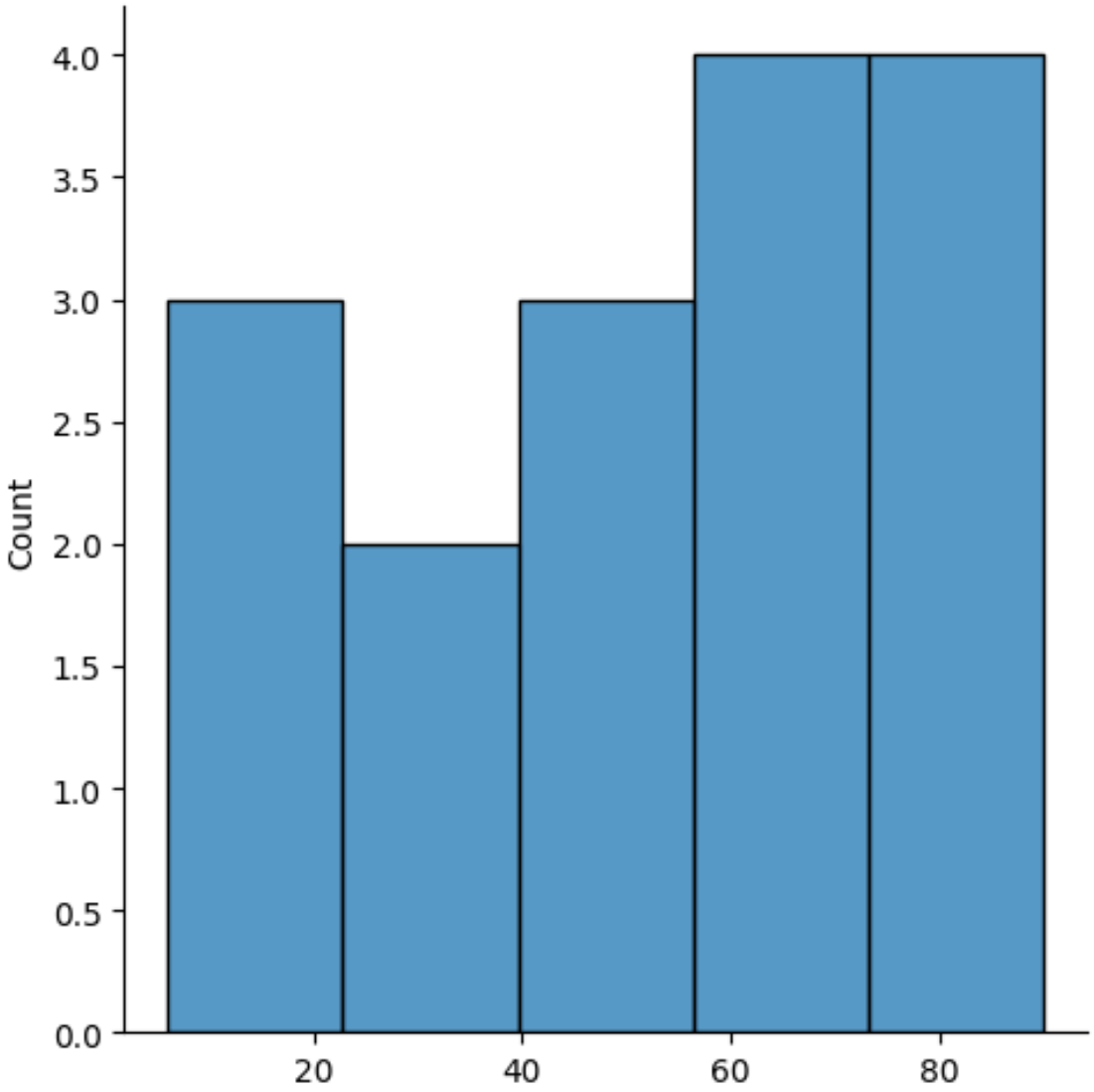
sns.displot(array)
plt.show()
```




```
new_array = array[(array > lr) & (array < ur)]
new_array
```

```
array([27, 50, 44,  6, 58, 61, 23, 86, 67, 20, 75,  7, 79, 61, 90, 54])
```

```
sns.displot(new_array)
plt.show()
```



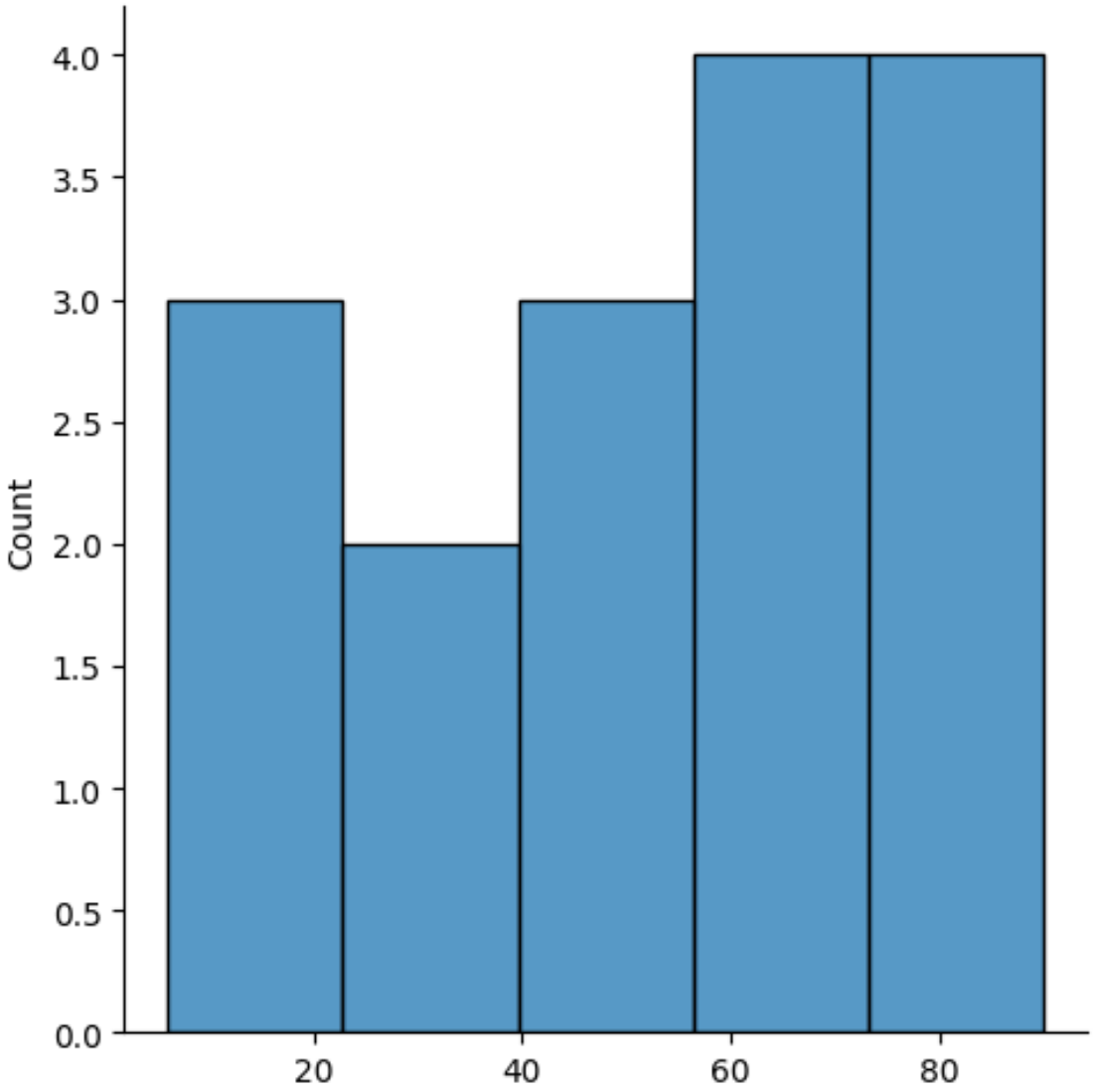

```
lr1, ur1 = outDetection(new_array)
lr1, ur1
```

```
(np.float64(-38.5), np.float64(133.5))
```

```
final_array = new_array[(new_array > lr1) & (new_array < ur1)]
final_array
```

```
array([27, 50, 44,  6, 58, 61, 23, 86, 67, 20, 75,  7, 79, 61, 90, 54])
```

```
sns.displot(final_array)
plt.show()
```



RESULT:

Thus the outliers in the given data set were detected using the Inter-Quartile Range (IQR) method and the data was visualized before and after removing the outliers.

EXPERIMENT NO: 13

EXPERIMENT TO HANDLE MISSING VALUES AND INAPPROPRIATE DATA IN A GIVEN DATA SET.

AIM:

To demonstrate an experiment to handle missing data and inappropriate data in a data set using the Python pandas library for data preprocessing.

PROGRAM:

Dataset Given: Hotel.csv

CustomerID	**Age_Group**	**Rating(1-5)**	**Hotel**	**FoodPreference**	**Bill**	**NoOfPax**	**EstimatedSalary**	**Age_Group**
1	**20-25**	**4**	**Ibis**	**veg**	**1300**	**2**	**40000**	**20-25**
2	**30-35**	**5**	**LemonTree**	**Non-Veg**	**2000**	**3**	**59000**	**30-35**
3	**25-30**	**6**	**RedFox**	**Veg**	**1322**	**2**	**30000**	**25-30**
4	**20-25**	**-1**	**LemonTree**	**Veg**	**1234**	**2**	**120000**	**20-25**
5	**35+**	**3**	**Ibis**	**Vegetarian**	**989**	**2**	**45000**	**35+**
6	**35+**	**3**	**Ibys**	**Non-Veg**	**1909**	**2**	**122220**	**35+**
7	**35+**	**4**	**RedFox**	**Vegetarian**	**1000**	**-1**	**21122**	**35+**
8	**20-25**	**7**	**LemonTree**	**Veg**	**2999**	**-10**	**345673**	**20-25**
9	**25-30**	**2**	**Ibis**	**Non-Veg**	**3456**	**3**	**-99999**	**25-30**
9	**25-30**	**2**	**Ibis**	**Non-Veg**	**3456**	**3**	**-99999**	**25-30**
10	**30-35**	**5**	**RedFox**	**non-Veg**	**-6755**	**4**	**87777**	**30-35**

```
# Upload Hotel.csv and convert it into dataFrame
import numpy as np
import pandas as pd

data = {
    'CustomerID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 9, 10],
    'Age_Group': ['20-25', '30-35', '25-30', '20-25', '35+', '35+', '35+',
                  '20-25', '25-30', '25-30', '30-35'],
    'Rating(1-5)': [4, 5, 6, -1, 3, 3, 4, 7, 2, 2, 5],
    'Hotel': ['Ibis', 'LemonTree', 'RedFox', 'LemonTree', 'Ibis', 'Ibys',
              'RedFox', 'LemonTree', 'Ibis', 'Ibis', 'RedFox'],
    'FoodPreference': ['veg', 'Non-Veg', 'Veg', 'Veg', 'Vegetarian', 'Non-Veg',
                       'Vegetarian', 'Veg', 'Non-Veg', 'Non-Veg', 'non-Veg'],
    'Bill': [1300, 2000, 1322, 1234, 989, 1909, 1000, 2999, 3456, 3456, -6755],
    'NoOfPax': [2, 3, 2, 2, 2, 2, -1, -10, 3, 3, 4],
    'EstimatedSalary': [40000, 59000, 30000, 120000, 45000, 122220, 21122,
                        345673, -99999, -99999, 87777],
    'Age_Group.1': ['20-25', '30-35', '25-30', '20-25', '35+', '35+', '35+',
                    '20-25', '25-30', '25-30', '30-35']
}
df = pd.DataFrame(data)
df.to_csv('Hotel_Dataset.csv', index=False)
print("Hotel_Dataset.csv created successfully!")
```

OUTPUT:

```
Hotel_Dataset.csv created successfully!

df = pd.read_csv('Hotel_Dataset.csv')
df

    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis            veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6     RedFox            Veg  1322
```

```
3            4     20-25           -1  LemonTree            Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4     RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree            Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
9            9     25-30            2       Ibis        Non-Veg  3456
10          10     30-35            5     RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2           40000       20-25
1         3           59000       30-35
2         2           30000       25-30
3         2          120000       20-25
4         2           45000         35+
5         2          122220         35+
6        -1           21122         35+
7       -10          345673       20-25
8         3          -99999       25-30
9         3          -99999       25-30
10        4           87777       30-35

# From the dataframe identify the duplicate row (i.e row 9)
# The duplicated() method returns a Series with True and False values
# that describe which rows in the DataFrame are duplicated and not.
df.duplicated()

0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9      True
10    False
dtype: bool

# The info() method prints information about the DataFrame.
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   CustomerID       11 non-null     int64
 1   Age_Group        11 non-null     object
 2   Rating(1-5)      11 non-null     int64
 3   Hotel            11 non-null     object
 4   FoodPreference   11 non-null     object
 5   Bill             11 non-null     int64
 6   NoOfPax          11 non-null     int64
 7   EstimatedSalary  11 non-null     int64
 8   Age_Group.1      11 non-null     object
dtypes: int64(5), object(4)
memory usage: 920.0+ bytes

# The drop_duplicates() method removes duplicate rows.
df.drop_duplicates(inplace=True)
df
```

```
    CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill  \
0            1     20-25            4       Ibis            veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6     RedFox            Veg  1322
3            4     20-25           -1  LemonTree            Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4     RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree            Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
10          10     30-35            5     RedFox        non-Veg -6755

    NoOfPax  EstimatedSalary Age_Group.1
0         2           40000       20-25
1         3           59000       30-35
2         2           30000       25-30
3         2          120000       20-25
4         2           45000         35+
5         2          122220         35+
6        -1           21122         35+
7       -10          345673       20-25
8         3          -99999       25-30
10        4           87777       30-35

# While removing duplicate record row index also removed.
# The len() function returns the number of rows.
print(len(df))

# Reset the index
df.reset_index(drop=True, inplace=True)
df

10
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill
NoOfPax  \
0            1     20-25            4       Ibis            veg  1300
1            2     30-35            5  LemonTree        Non-Veg  2000
2            3     25-30            6     RedFox            Veg  1322
3            4     20-25           -1  LemonTree            Veg  1234
4            5       35+            3       Ibis     Vegetarian   989
5            6       35+            3       Ibys        Non-Veg  1909
6            7       35+            4     RedFox     Vegetarian  1000
7            8     20-25            7  LemonTree            Veg  2999
8            9     25-30            2       Ibis        Non-Veg  3456
9           10     30-35            5     RedFox        non-Veg -6755

        2
        3
        2
        2
        2
        2
       -1
      -10
        3
        4

   EstimatedSalary Age_Group.1
0            40000       20-25
1            59000       30-35
2            30000       25-30
3           120000       20-25
4            45000         35+
5           122220         35+
6            21122         35+
7           345673       20-25
8           -99999       25-30
9            87777       30-35

# Axis refers to the dimensions of a DataFrame (index and columns).
# Use axis=1 to apply functions column-wise across columns.
# Drop the repeated Age_Group column.
df.drop(['Age_Group.1'], axis=1, inplace=True)
df
```

```
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference  Bill
NoOfPax  \
0           1     20-25            4       Ibis            veg  1300        2
1           2     30-35            5  LemonTree        Non-Veg  2000        3
2           3     25-30            6     RedFox            Veg  1322        2
3           4     20-25           -1  LemonTree            Veg  1234        2
4           5       35+            3       Ibis     Vegetarian   989        2
5           6       35+            3       Ibys        Non-Veg  1909        2
6           7       35+            4     RedFox     Vegetarian  1000       -1
7           8     20-25            7  LemonTree            Veg  2999      -10
8           9     25-30            2       Ibis        Non-Veg  3456        3
9          10     30-35            5     RedFox        non-Veg -6755        4

   EstimatedSalary
0            40000
1            59000
2            30000
3           120000
4            45000
5           122220
6            21122
7           345673
8           -99999
9            87777

# The function .loc is typically used for label indexing and can access
# multiple columns. Replace inappropriate negative values with NaN.
df.loc[df.CustomerID < 0, 'CustomerID'] = np.nan
df.loc[df.Bill < 0, 'Bill'] = np.nan
df.loc[df.EstimatedSalary < 0, 'EstimatedSalary'] = np.nan
df

   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6     RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis     Vegetarian   989.0
5         6.0       35+            3       Ibys        Non-Veg  1909.0
6         7.0       35+            4     RedFox     Vegetarian  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5     RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0        2          40000.0
1        3          59000.0
2        2          30000.0
3        2         120000.0
4        2          45000.0
5        2         122220.0
6       -1          21122.0
7      -10         345673.0
8        3              NaN
9        4          87777.0

# NoOfPax (number of people) must be between 1 and 20
df.loc[(df['NoOfPax'] < 1) | (df['NoOfPax'] > 20), 'NoOfPax'] = np.nan
df

   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6     RedFox            Veg  1322.0
```

```
3         4.0     20-25           -1  LemonTree           Veg  1234.0
4         5.0       35+            3       Ibis     Vegetarian   989.0
5         6.0       35+            3       Ibys        Non-Veg  1909.0
6         7.0       35+            4     RedFox     Vegetarian  1000.0
7         8.0     20-25            7  LemonTree           Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5     RedFox        non-Veg     NaN

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      NaN          21122.0
7      NaN         345673.0
8      3.0              NaN
9      4.0          87777.0
```

```
df.Age_Group.unique()
```

```
array(['20-25', '30-35', '25-30', '35+'], dtype=object)
```

```
df.Hotel.unique()
```

```
array(['Ibis', 'LemonTree', 'RedFox', 'Ibys'], dtype=object)
```

```
# Fix the spelling mistake 'Ibys' -> 'Ibis'
df.Hotel.replace(['Ibys'], 'Ibis', inplace=True)
df.Hotel.unique()
```

```
/tmp/ipykernel_3036/685295709.py:2: FutureWarning: A value is trying to be set
on a copy of a DataFrame or Series through chained assignment using an inplace
method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves
as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.



  df.Hotel.replace(['Ibys'], 'Ibis', inplace=True)
array(['Ibis', 'LemonTree', 'RedFox'], dtype=object)
```

```
df.FoodPreference.unique()
```

```
array(['veg', 'Non-Veg', 'Veg', 'Vegetarian', 'non-Veg'], dtype=object)
```

```
# Standardize the FoodPreference categories
df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg', inplace=True)
df.FoodPreference.replace(['non-Veg'], 'Non-Veg', inplace=True)
df.FoodPreference.unique()
```

```
/tmp/ipykernel_3036/3470221995.py:2: FutureWarning: A value is trying to be set
on a copy of a DataFrame or Series through chained assignment using an inplace
method.
The behavior will change in pandas 3.0. This inplace method will never work
because the intermediate object on which we are setting values always behaves
as a copy.
```

```
For example, when doing 'df[col].method(value, inplace=True)', try using
'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value)
instead, to perform the operation inplace on the original object.


  df.FoodPreference.replace(['Vegetarian', 'veg'], 'Veg', inplace=True)
array(['Veg', 'Non-Veg'], dtype=object)

# Fillna is a Pandas function to fill the NA/NaN values.
# Numerical continuous data -> replace missing (NaN) value by mean.
# Numerical discrete data -> replace missing (NaN) value by median.
# Categorical data -> replace missing (NaN) value by mode.
df['EstimatedSalary'] =
df['EstimatedSalary'].fillna(round(df['EstimatedSalary'].mean()))
df['NoOfPax'] = df['NoOfPax'].fillna(round(df['NoOfPax'].median()))
df['Rating(1-5)'] = df['Rating(1-5)'].fillna(round(df['Rating(1-5)'].median()))
df['Bill'] = df['Bill'].fillna(round(df['Bill'].mean()))
df

   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill  \
0         1.0     20-25            4       Ibis            Veg  1300.0
1         2.0     30-35            5  LemonTree        Non-Veg  2000.0
2         3.0     25-30            6     RedFox            Veg  1322.0
3         4.0     20-25           -1  LemonTree            Veg  1234.0
4         5.0       35+            3       Ibis            Veg   989.0
5         6.0       35+            3       Ibis        Non-Veg  1909.0
6         7.0       35+            4     RedFox            Veg  1000.0
7         8.0     20-25            7  LemonTree            Veg  2999.0
8         9.0     25-30            2       Ibis        Non-Veg  3456.0
9        10.0     30-35            5     RedFox        Non-Veg  1801.0

   NoOfPax  EstimatedSalary
0      2.0          40000.0
1      3.0          59000.0
2      2.0          30000.0
3      2.0         120000.0
4      2.0          45000.0
5      2.0         122220.0
6      2.0          21122.0
7      2.0         345673.0
8      3.0          96755.0
9      4.0          87777.0
```

RESULT:

Thus the missing values and inappropriate data in the given hotel data set were handled - duplicates dropped, invalid values replaced with NaN and filled using mean/median, spelling mistakes corrected and categories standardized.

EXPERIMENT NO: 14
EXPERIMENT TO UNDERSTAND FEATURE SCALING.

AIM:
To understand the importance of feature scaling in data science.

PROGRAM:

```
%pip install -q scikit-learn
import numpy as np
import pandas as pd

data = {
    'Country': ['France', 'Spain', 'Germany', 'Spain', 'Germany',
                'France', 'Spain', 'France', np.nan, 'France'],
    'Age': [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],
    'Salary': [72000, 48000, 54000, 61000, np.nan, 58000, 52000,
               79000, 83000, 67000],
    'Purchased': ['No', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'Yes', 'No',
'Yes']
}
pd.DataFrame(data).to_csv('pre-process_datasample.csv', index=False)

df = pd.read_csv('pre-process_datasample.csv')
df
```

OUTPUT:

```
Note: you may need to restart the kernel to use updated packages.
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8      NaN  50.0  83000.0        No
9   France  37.0  67000.0       Yes

df.head()

   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes

df['Country'] = df['Country'].fillna(df['Country'].mode()[0])
features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values
print(features)

[['France' 44.0 72000.0]
 ['Spain' 27.0 48000.0]
 ['Germany' 30.0 54000.0]
 ['Spain' 38.0 61000.0]
 ['Germany' 40.0 nan]
 ['France' 35.0 58000.0]
 ['Spain' nan 52000.0]
 ['France' 48.0 79000.0]
```

```
 ['France' 50.0 83000.0]
 ['France' 37.0 67000.0]]
```

```
from sklearn.impute import SimpleImputer

age = SimpleImputer(strategy="mean", missing_values=np.nan)
Salary = SimpleImputer(strategy="mean", missing_values=np.nan)

age.fit(features[:, [1]])
Salary.fit(features[:, [2]])

features[:, [1]] = age.transform(features[:, [1]])
features[:, [2]] = Salary.transform(features[:, [2]])
features
```

```
array([['France', 44.0, 72000.0],
       ['Spain', 27.0, 48000.0],
       ['Germany', 30.0, 54000.0],
       ['Spain', 38.0, 61000.0],
       ['Germany', 40.0, 63777.77777777778],
       ['France', 35.0, 58000.0],
       ['Spain', 38.77777777777778, 52000.0],
       ['France', 48.0, 79000.0],
       ['France', 50.0, 83000.0],
       ['France', 37.0, 67000.0]], dtype=object)
```

```
from sklearn.preprocessing import OneHotEncoder

oh = OneHotEncoder(sparse_output=False)
Country = oh.fit_transform(features[:, [0]])
Country
```

```
array([[1., 0., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [0., 0., 1.],
       [0., 1., 0.],
       [1., 0., 0.],
       [0., 0., 1.],
       [1., 0., 0.],
       [1., 0., 0.],
       [1., 0., 0.]])
```

```
final_set = np.concatenate((Country, features[:, [1, 2]]), axis=1)
final_set
```

```
array([[1.0, 0.0, 0.0, 44.0, 72000.0],
       [0.0, 0.0, 1.0, 27.0, 48000.0],
       [0.0, 1.0, 0.0, 30.0, 54000.0],
       [0.0, 0.0, 1.0, 38.0, 61000.0],
       [0.0, 1.0, 0.0, 40.0, 63777.77777777778],
       [1.0, 0.0, 0.0, 35.0, 58000.0],
       [0.0, 0.0, 1.0, 38.77777777777778, 52000.0],
       [1.0, 0.0, 0.0, 48.0, 79000.0],
       [1.0, 0.0, 0.0, 50.0, 83000.0],
       [1.0, 0.0, 0.0, 37.0, 67000.0]], dtype=object)
```

```
from sklearn.preprocessing import StandardScaler

sc = StandardScaler()
sc.fit(final_set)
feat_standard_scaler = sc.transform(final_set)
```

```
feat_standard_scaler
```

```
array([[ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
         7.58874362e-01,  7.49473254e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,
        -1.71150388e+00, -1.43817841e+00],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01,
        -1.27555478e+00, -8.91265492e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,
        -1.13023841e-01, -2.53200424e-01],
       [-1.00000000e+00,  2.00000000e+00, -6.54653671e-01,
         1.77608893e-01,  6.63219199e-16],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
        -5.48972942e-01, -5.26656882e-01],
       [-1.00000000e+00, -5.00000000e-01,  1.52752523e+00,
         0.00000000e+00, -1.07356980e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
         1.34013983e+00,  1.38753832e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
         1.63077256e+00,  1.75214693e+00],
       [ 1.00000000e+00, -5.00000000e-01, -6.54653671e-01,
        -2.58340208e-01,  2.93712492e-01]])
```

```
from sklearn.preprocessing import MinMaxScaler

mms = MinMaxScaler(feature_range=(0, 1))
mms.fit(final_set)
feat_minmax_scaler = mms.transform(final_set)
feat_minmax_scaler
```

```
array([[1.        , 0.        , 0.        , 0.73913043, 0.68571429],
       [0.        , 0.        , 1.        , 0.        , 0.        ],
       [0.        , 1.        , 0.        , 0.13043478, 0.17142857],
       [0.        , 0.        , 1.        , 0.47826087, 0.37142857],
       [0.        , 1.        , 0.        , 0.56521739, 0.45079365],
       [1.        , 0.        , 0.        , 0.34782609, 0.28571429],
       [0.        , 0.        , 1.        , 0.51207729, 0.11428571],
       [1.        , 0.        , 0.        , 0.91304348, 0.88571429],
       [1.        , 0.        , 0.        , 1.        , 1.        ],
       [1.        , 0.        , 0.        , 0.43478261, 0.54285714]])
```

RESULT:

Thus feature scaling was understood and performed using StandardScaler (standardization) and MinMaxScaler (normalization) after handling missing values and encoding the categorical feature.

EXPERIMENT NO: 15

EXPERIMENT TO UNDERSTAND THE DATA PREPROCESSING IN DATA SCIENCE.

AIM:

To understand the importance of data preprocessing in data science.

PROGRAM:

```
import numpy as np
import pandas as pd

data = {
    'Country': ['France', 'Spain', 'Germany', 'Spain', 'Germany',
                'France', 'Spain', 'France', np.nan, 'France'],
    'Age': [44, 27, 30, 38, 40, 35, np.nan, 48, 50, 37],
    'Salary': [72000, 48000, 54000, 61000, np.nan, 58000, 52000,
               79000, 83000, 67000],
    'Purchased': ['No', 'Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'Yes', 'No',
'Yes']
}
pd.DataFrame(data).to_csv('pre-process_datasample.csv', index=False)

df = pd.read_csv('pre-process_datasample.csv')
df
```

OUTPUT:

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0      NaN       Yes
5   France  35.0  58000.0       Yes
6    Spain   NaN  52000.0        No
7   France  48.0  79000.0       Yes
8      NaN  50.0  83000.0        No
9   France  37.0  67000.0       Yes

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    9 non-null      object
 1   Age        9 non-null      float64
 2   Salary     9 non-null      float64
 3   Purchased  10 non-null     object
dtypes: float64(2), object(2)
memory usage: 448.0+ bytes

df.Country.mode()

0    France
Name: Country, dtype: object

df.Country.mode()[0]

'France'
```

```
type(df.Country.mode())
```

```
pandas.core.series.Series
```

```
df['Country'] = df['Country'].fillna(df['Country'].mode()[0])
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Salary'] = df['Salary'].fillna(round(df['Salary'].mean()))
df
```

```
   Country   Age   Salary Purchased
0   France  44.0  72000.0        No
1    Spain  27.0  48000.0       Yes
2  Germany  30.0  54000.0        No
3    Spain  38.0  61000.0        No
4  Germany  40.0  63778.0       Yes
5   France  35.0  58000.0       Yes
6    Spain  38.0  52000.0        No
7   France  48.0  79000.0       Yes
8   France  50.0  83000.0        No
9   France  37.0  67000.0       Yes
```

```
pd.get_dummies(df.Country)
```

```
   France  Germany  Spain
0    True    False  False
1   False    False   True
2   False     True  False
3   False    False   True
4   False     True  False
5    True    False  False
6   False    False   True
7    True    False  False
8    True    False  False
9    True    False  False
```

```
updated_dataset = pd.concat([pd.get_dummies(df.Country), df.iloc[:, [1, 2,
3]]], axis=1)
updated_dataset
```

```
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1   False    False   True  27.0  48000.0       Yes
2   False     True  False  30.0  54000.0        No
3   False    False   True  38.0  61000.0        No
4   False     True  False  40.0  63778.0       Yes
5    True    False  False  35.0  58000.0       Yes
6   False    False   True  38.0  52000.0        No
7    True    False  False  48.0  79000.0       Yes
8    True    False  False  50.0  83000.0        No
9    True    False  False  37.0  67000.0       Yes
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     object
 1   Age        10 non-null     float64
 2   Salary     10 non-null     float64
 3   Purchased  10 non-null     object
```

```
dtypes: float64(2), object(2)
memory usage: 448.0+ bytes
```

```
updated_dataset['Purchased'] = updated_dataset['Purchased'].replace(['No',
'Yes'], [0, 1])
updated_dataset
```

```
/tmp/ipykernel_3094/1802564252.py:1: FutureWarning: Downcasting behavior in
`replace` is deprecated and will be removed in a future version. To retain the
old behavior, explicitly call `result.infer_objects(copy=False)`. To opt-in to
the future behavior, set `pd.set_option('future.no_silent_downcasting', True)`
  updated_dataset['Purchased'] = updated_dataset['Purchased'].replace(['No',
'Yes'], [0, 1])
   France  Germany  Spain   Age   Salary  Purchased
0    True    False  False  44.0  72000.0          0
1   False    False   True  27.0  48000.0          1
2   False     True  False  30.0  54000.0          0
3   False    False   True  38.0  61000.0          0
4   False     True  False  40.0  63778.0          1
5    True    False  False  35.0  58000.0          1
6   False    False   True  38.0  52000.0          0
7    True    False  False  48.0  79000.0          1
8    True    False  False  50.0  83000.0          0
9    True    False  False  37.0  67000.0          1
```

RESULT:

Thus data preprocessing was understood and performed - missing values filled using mode/median/mean, categorical data encoded using one-hot encoding and labels converted to numeric form.

EXPERIMENT NO: 16

EXPERIMENT TO UNDERSTAND EDA - QUANTITATIVE AND QUALITATIVE ANALYSIS.

AIM:

To understand the importance of EDA - Quantitative and Qualitative analysis using the tips data set.

PROGRAM:

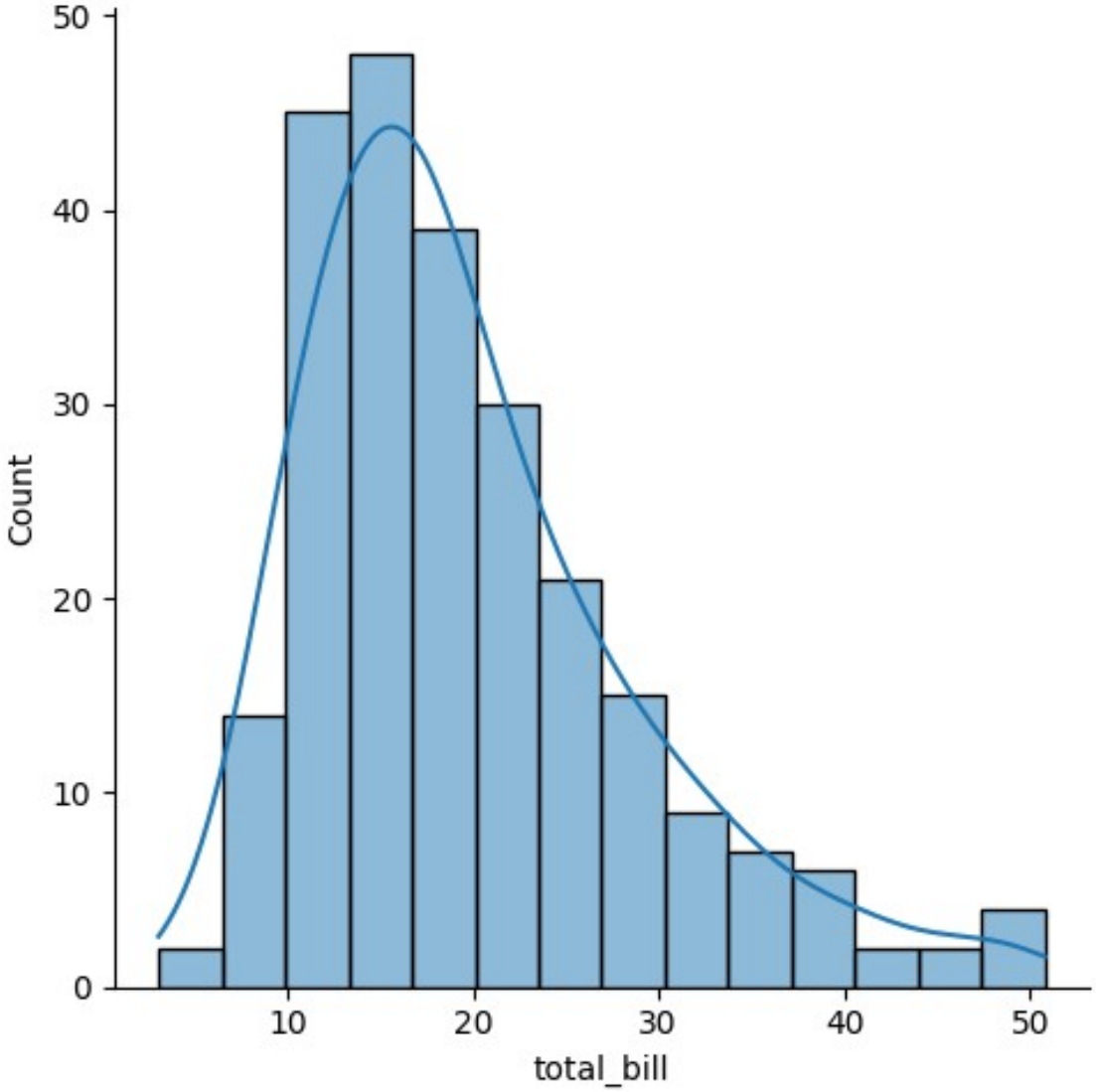
```
%matplotlib inline
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

tips = sns.load_dataset('tips')
tips.head()
```

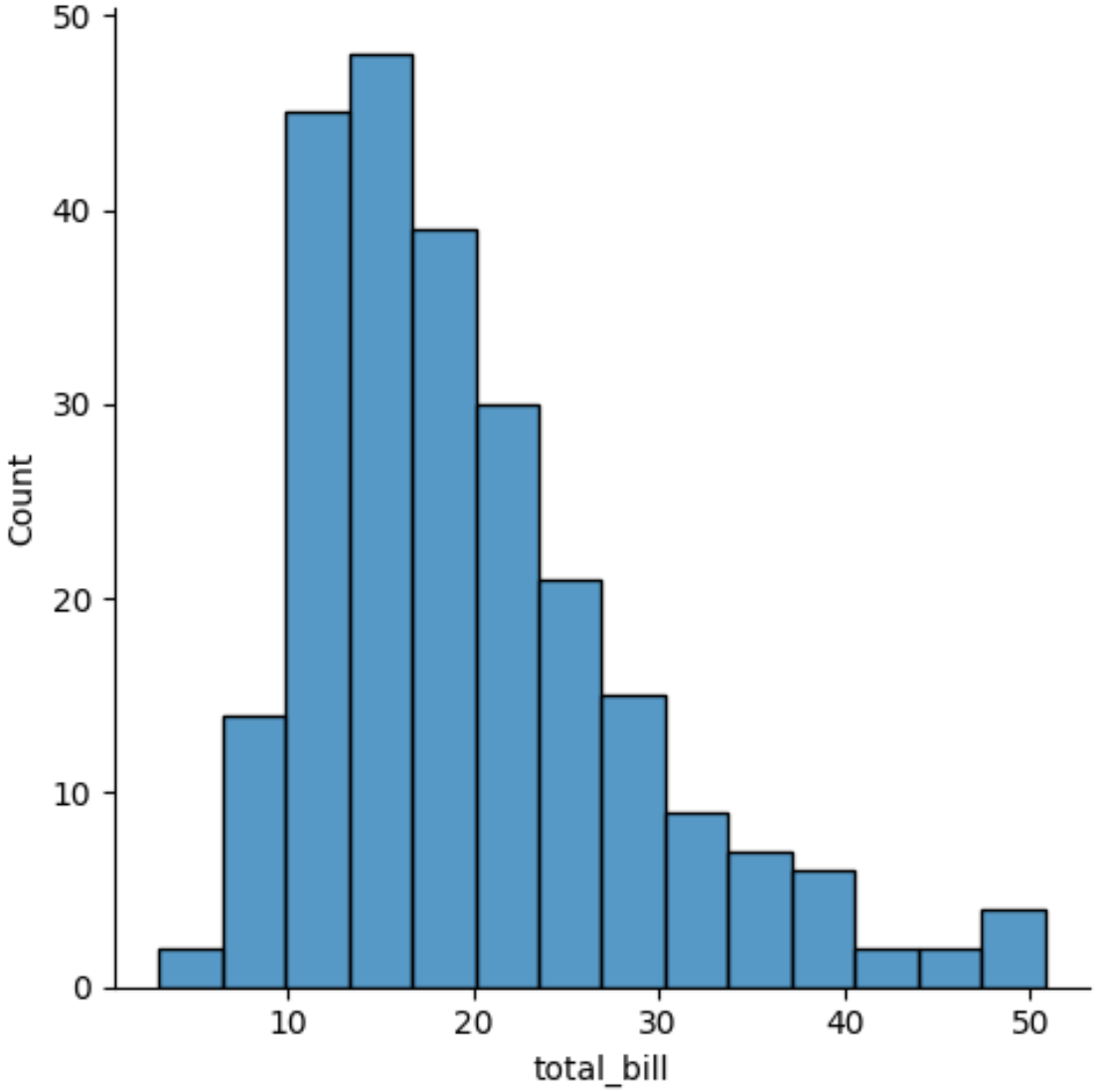
OUTPUT:

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

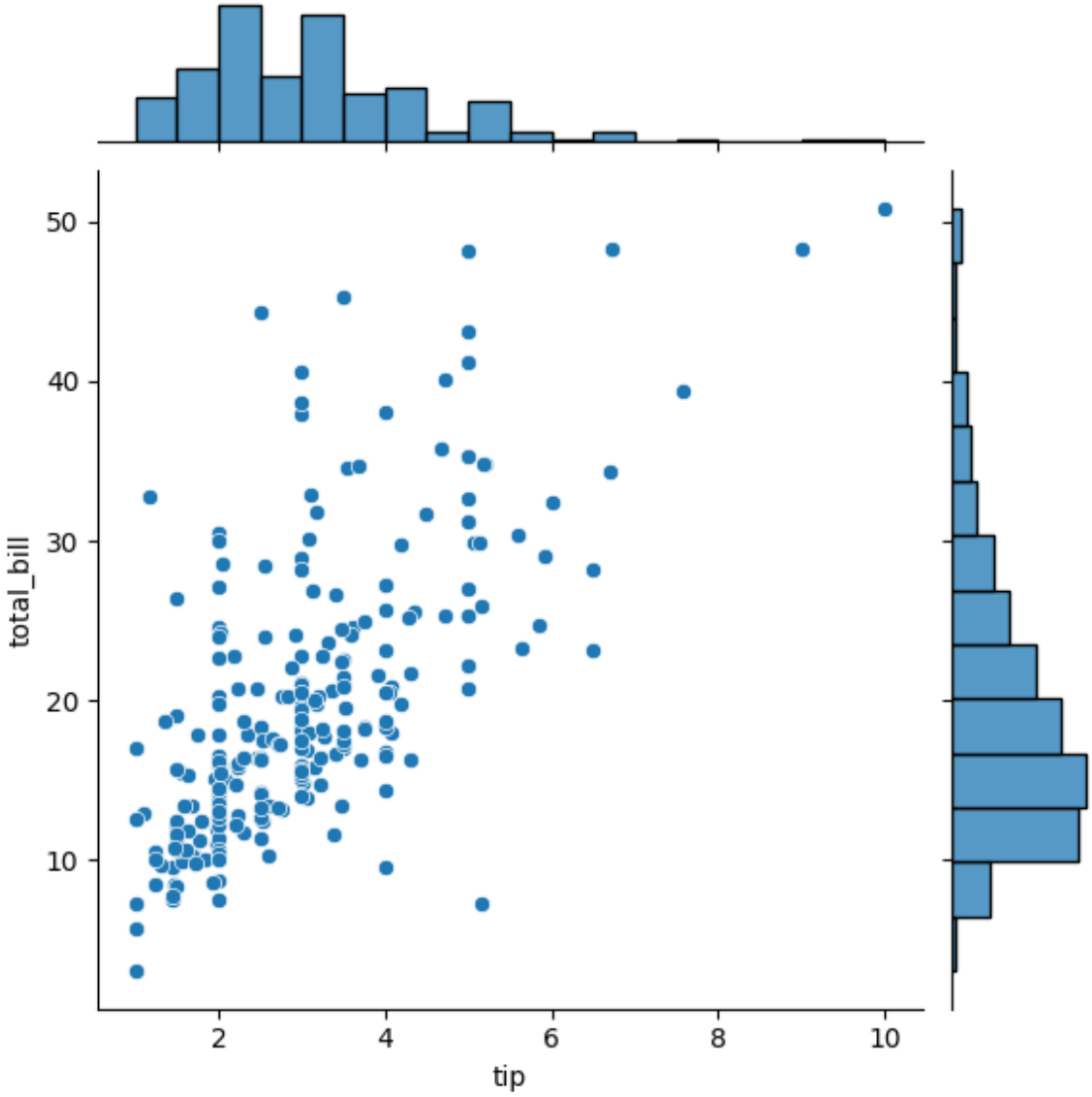
```
sns.displot(tips.total_bill, kde=True)
plt.show()
```



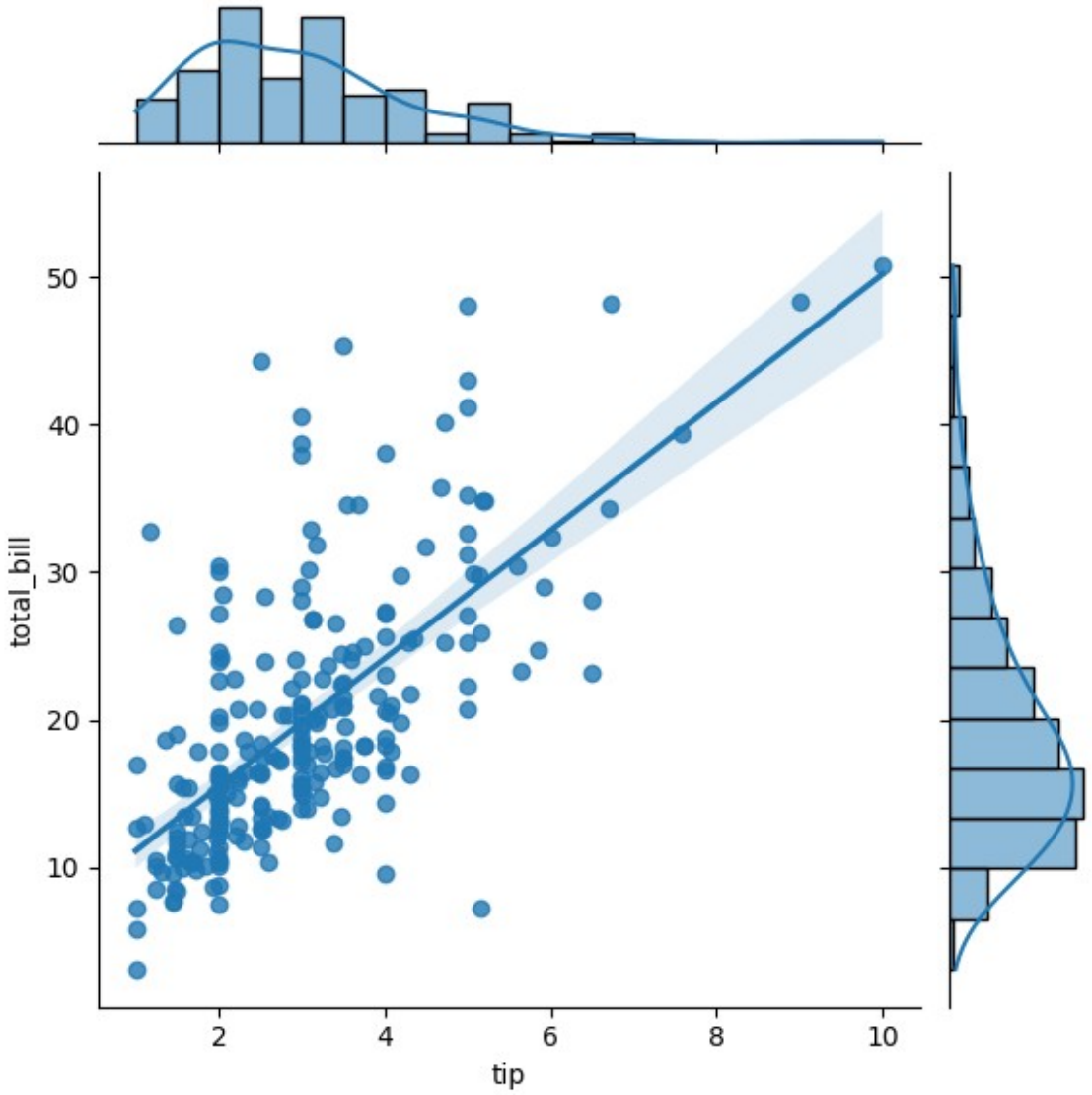

```
sns.displot(tips.total_bill, kde=False)
plt.show()
```



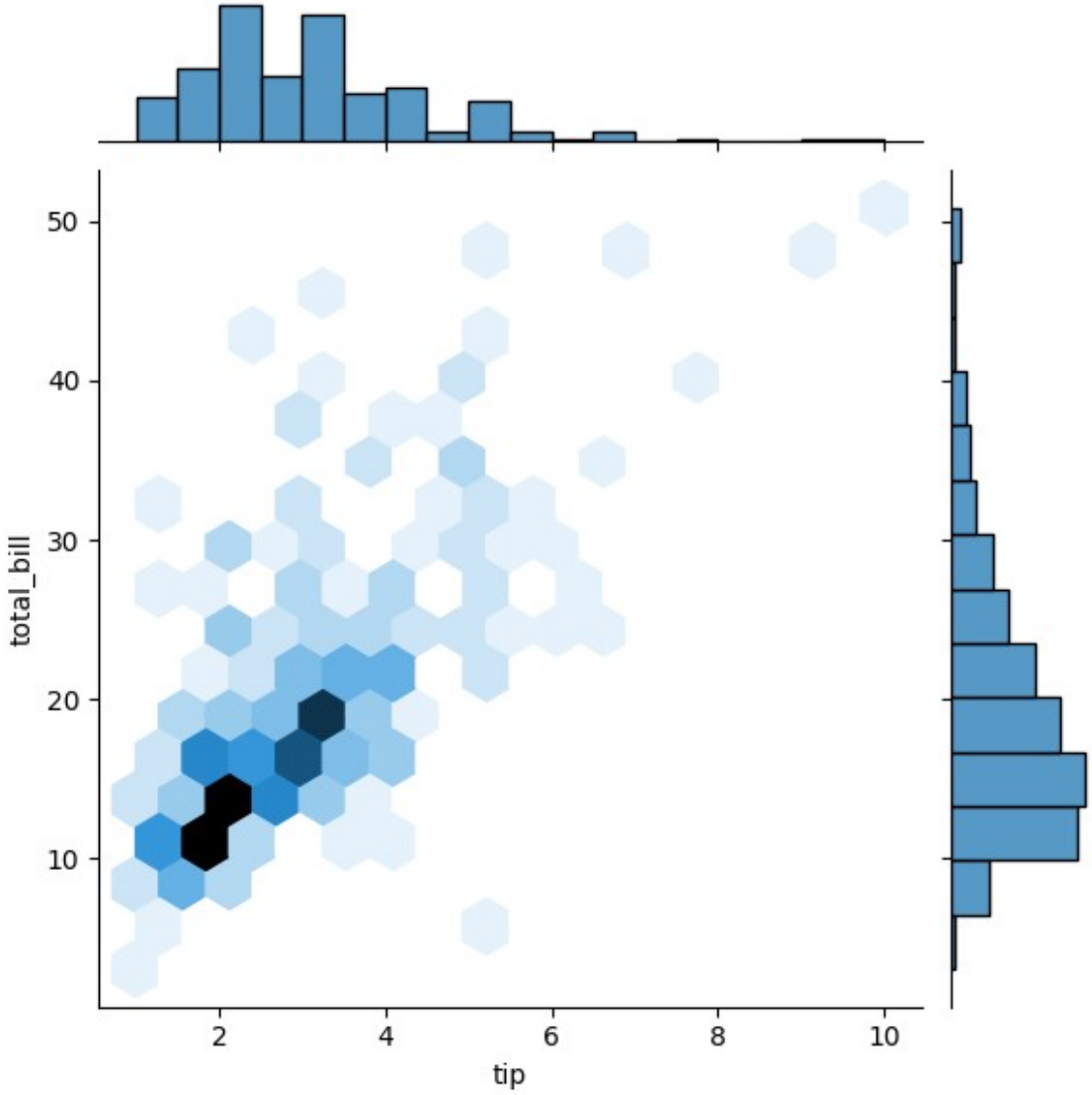

```
sns.jointplot(x=tips.tip, y=tips.total_bill)
plt.show()
```



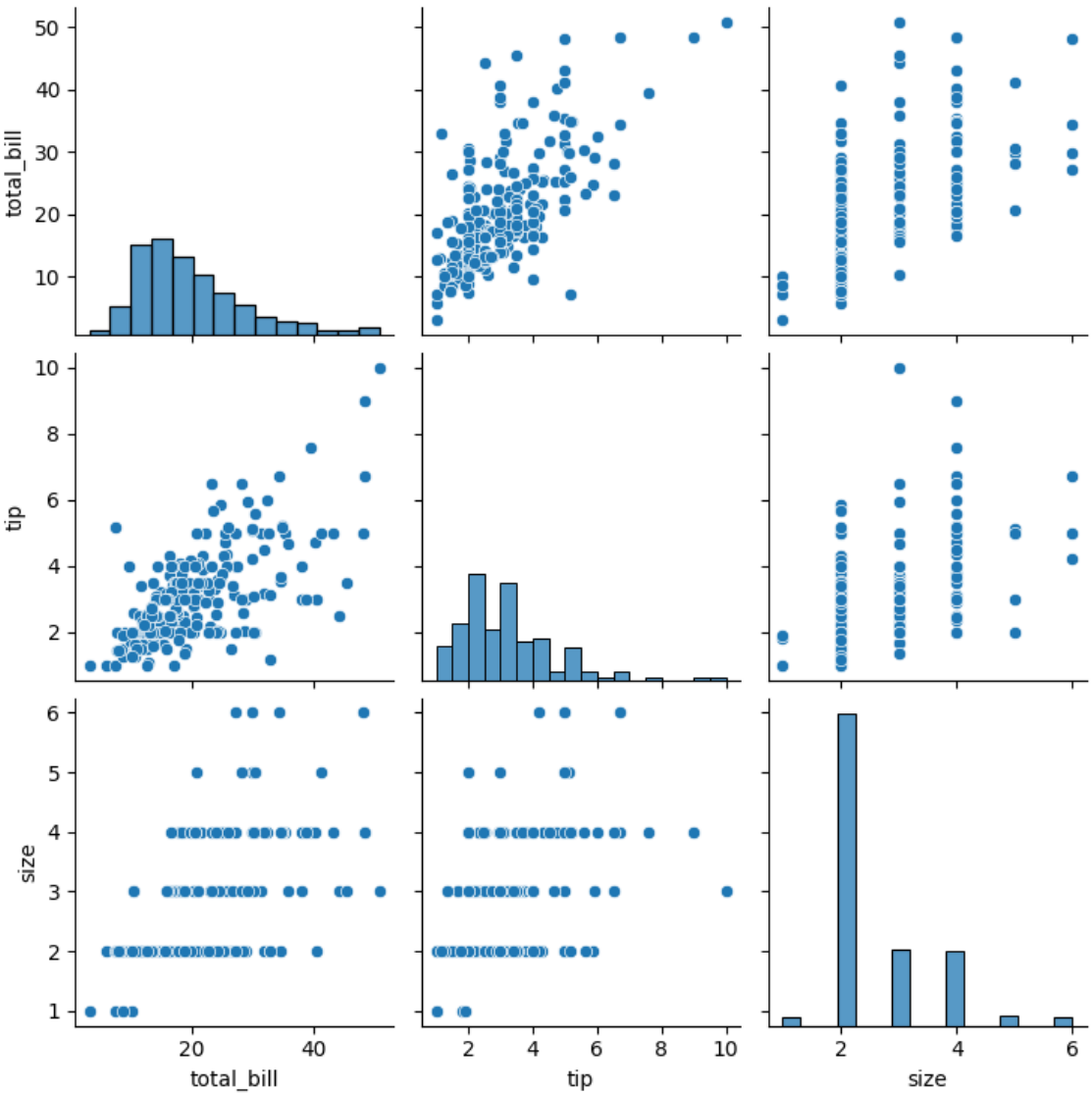

```
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="reg")
plt.show()
```




```
sns.jointplot(x=tips.tip, y=tips.total_bill, kind="hex")
plt.show()
```



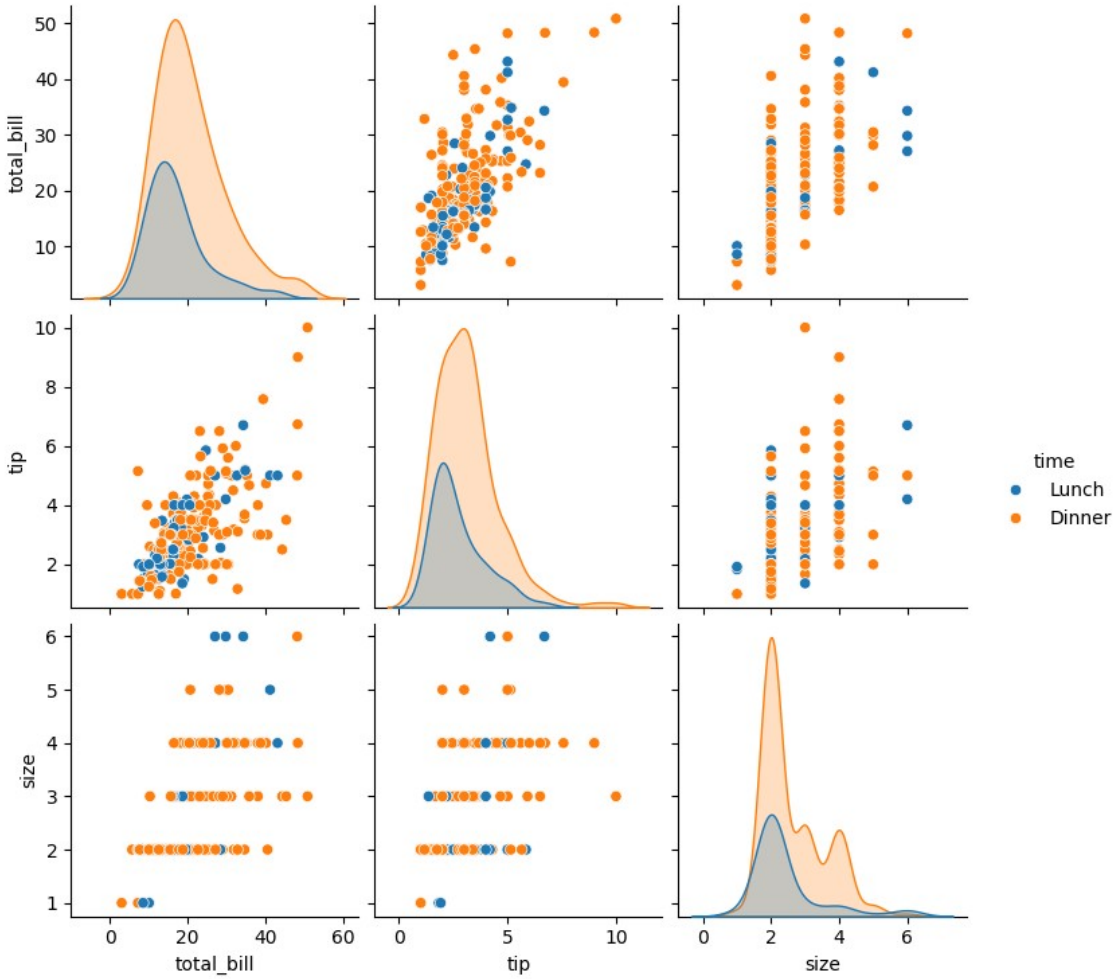

```
sns.pairplot(tips)
plt.show()
```



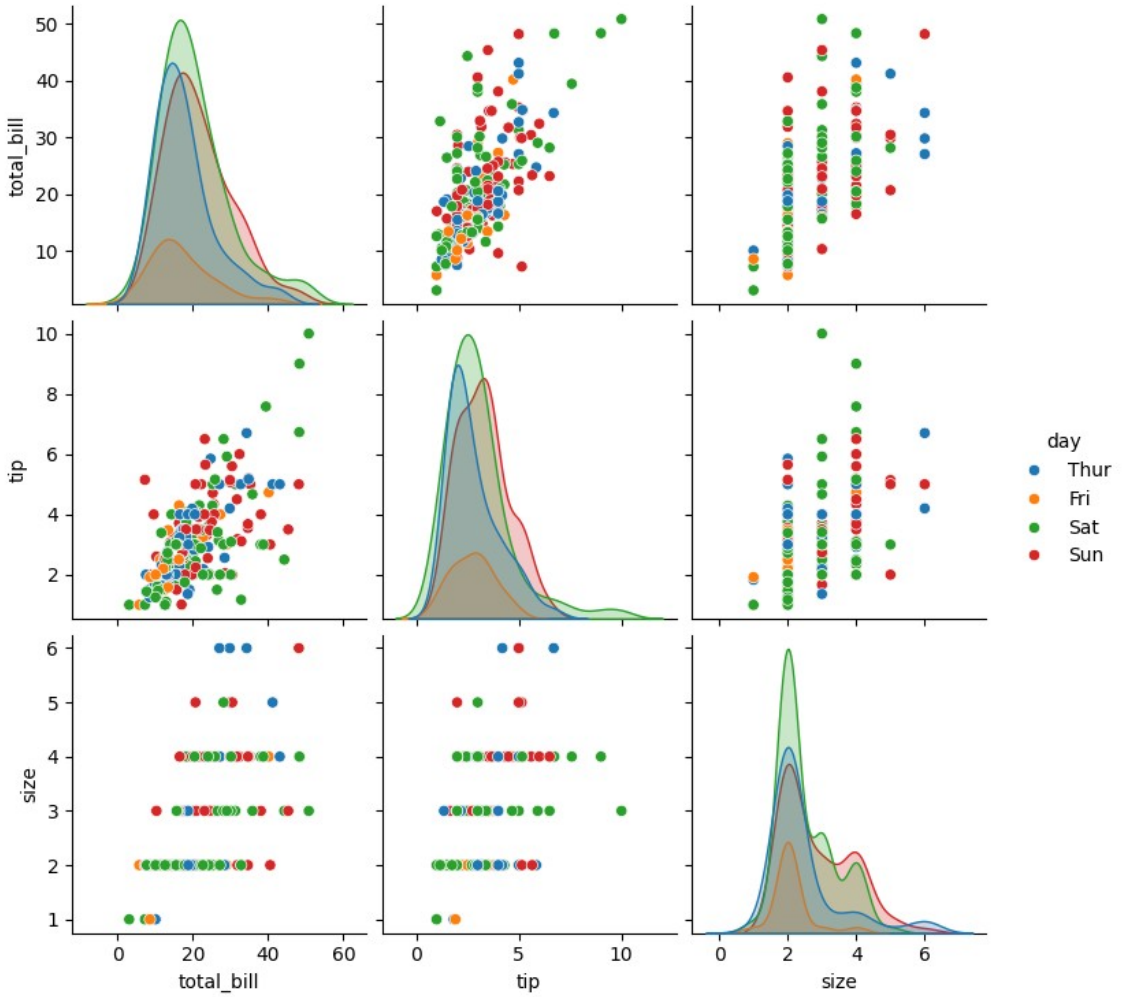

```
tips.time.value_counts()
```

```
time
Dinner    176
Lunch      68
Name: count, dtype: int64
```

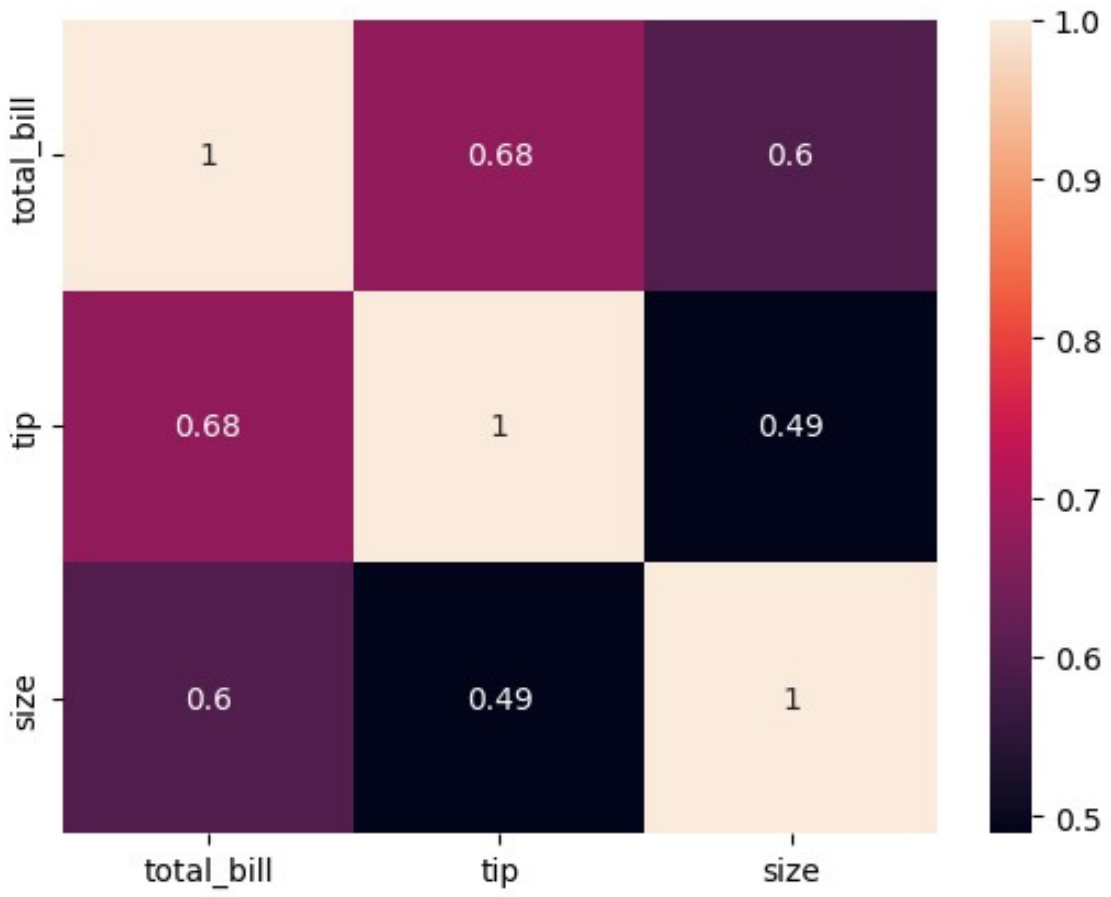
```
sns.pairplot(tips, hue='time')
plt.show()
```



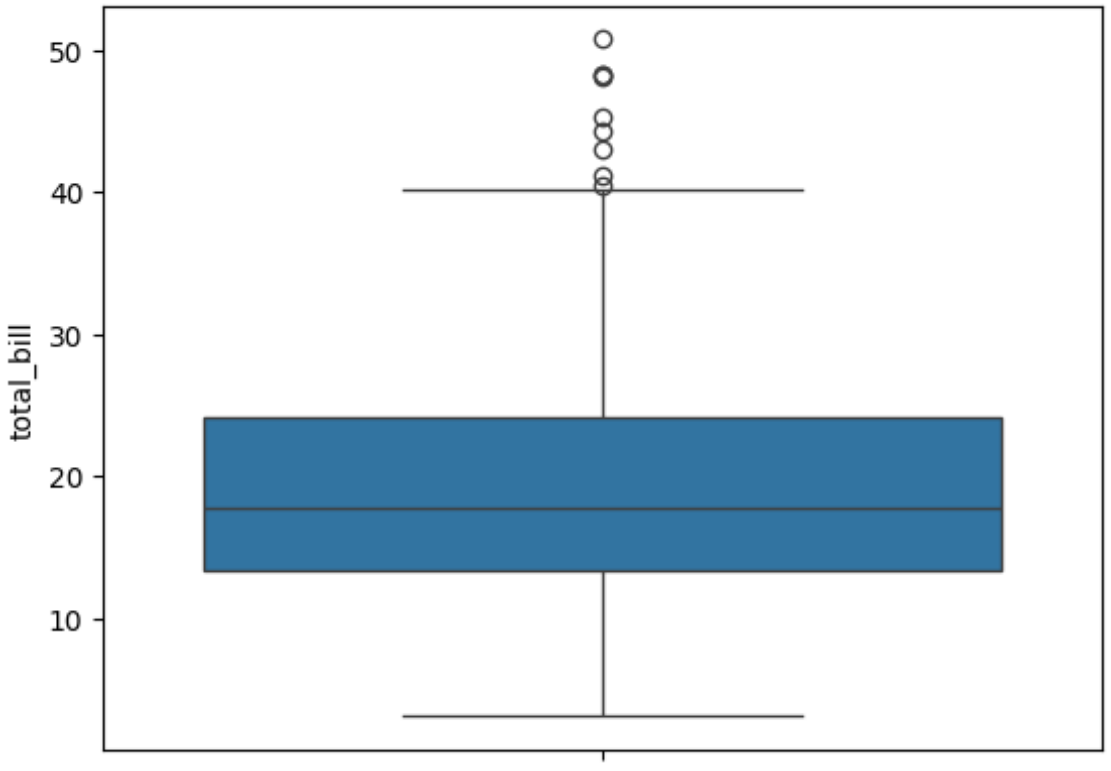

```
sns.pairplot(tips, hue='day')
plt.show()
```



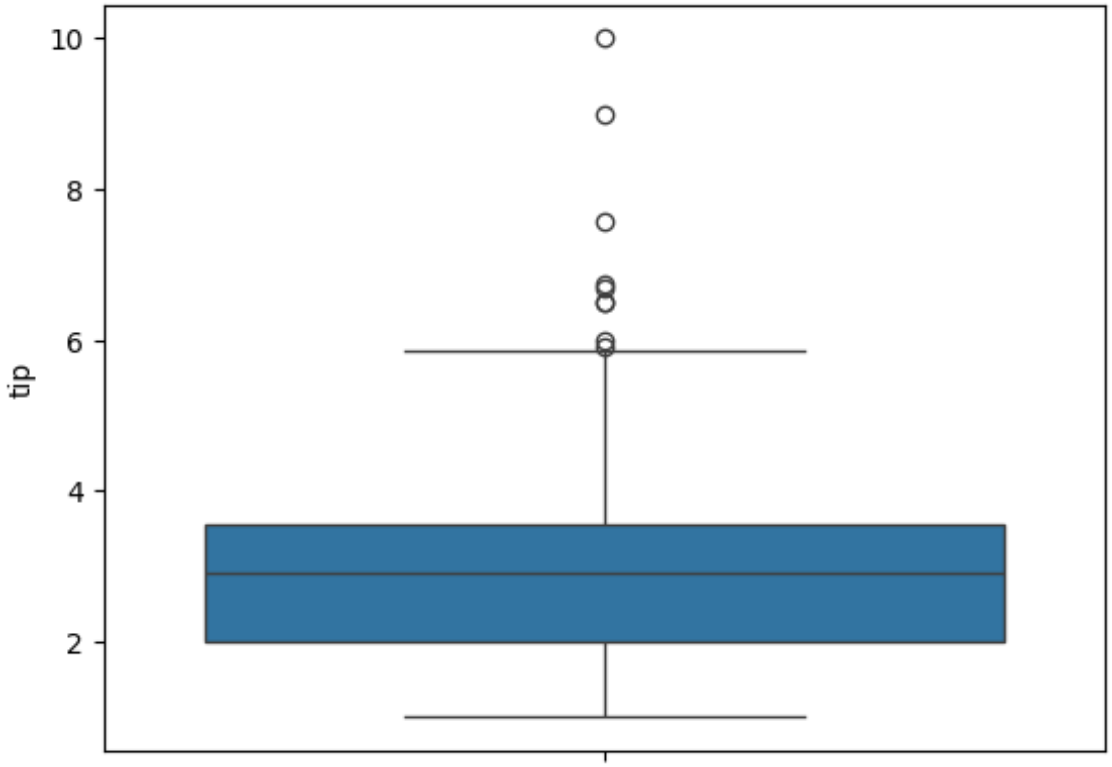

```
sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()
```



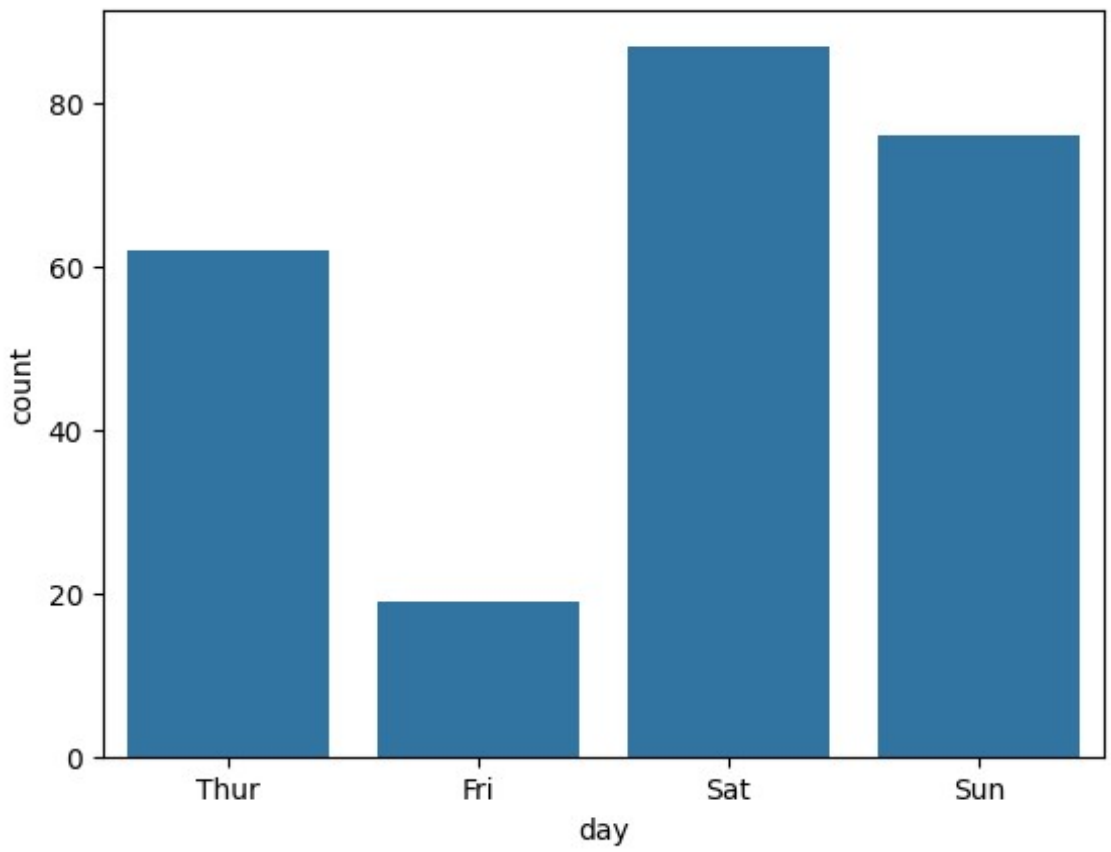

```
sns.boxplot(tips.total_bill)
plt.show()
```



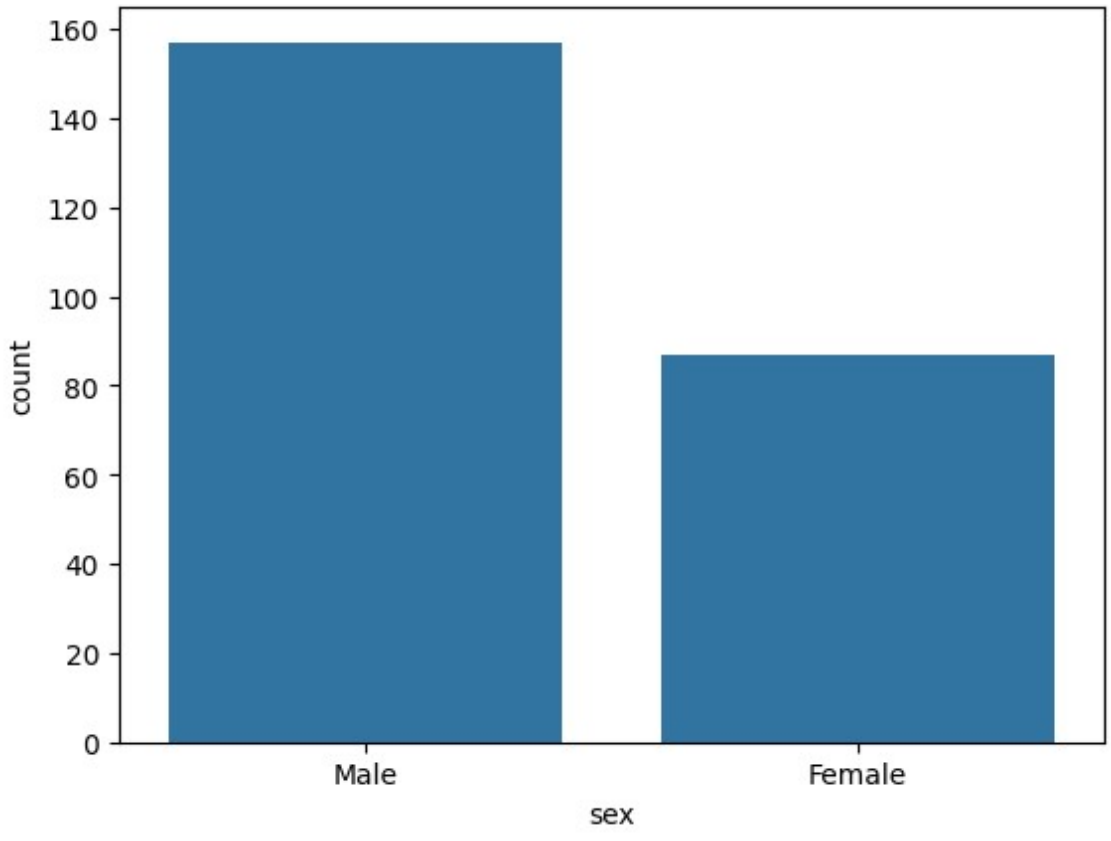
```
sns.boxplot(tips.tip)
plt.show()
```



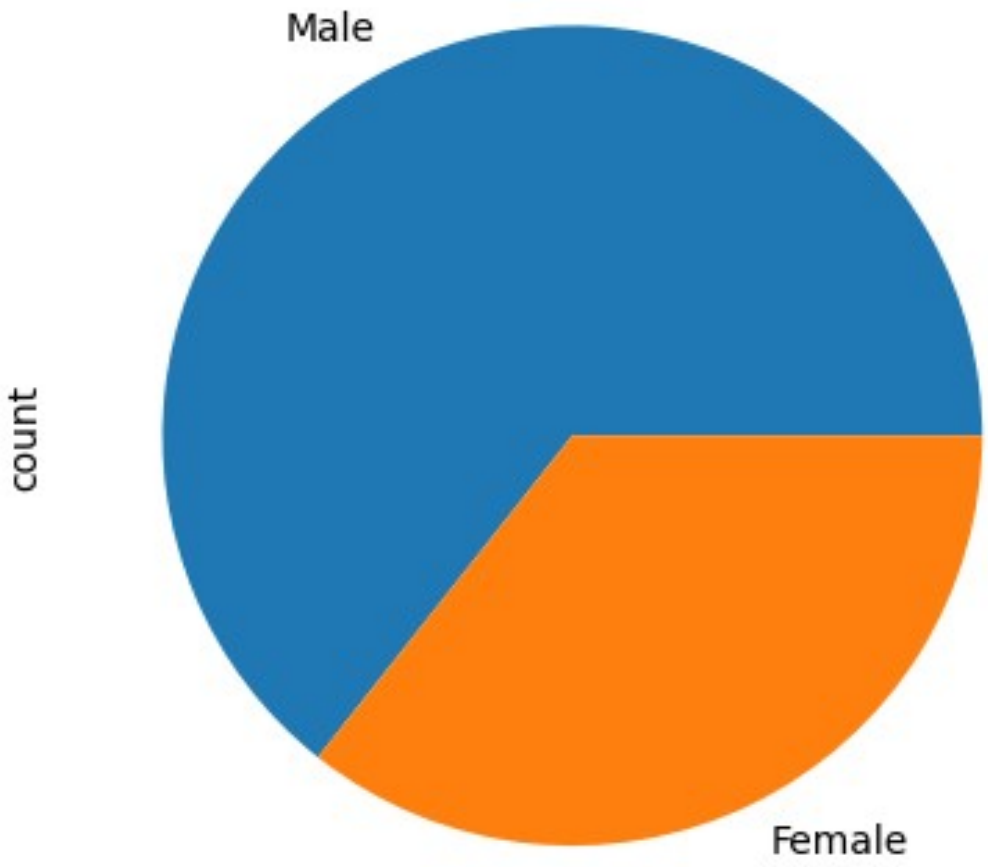

```
sns.countplot(x=tips.day)
plt.show()
```



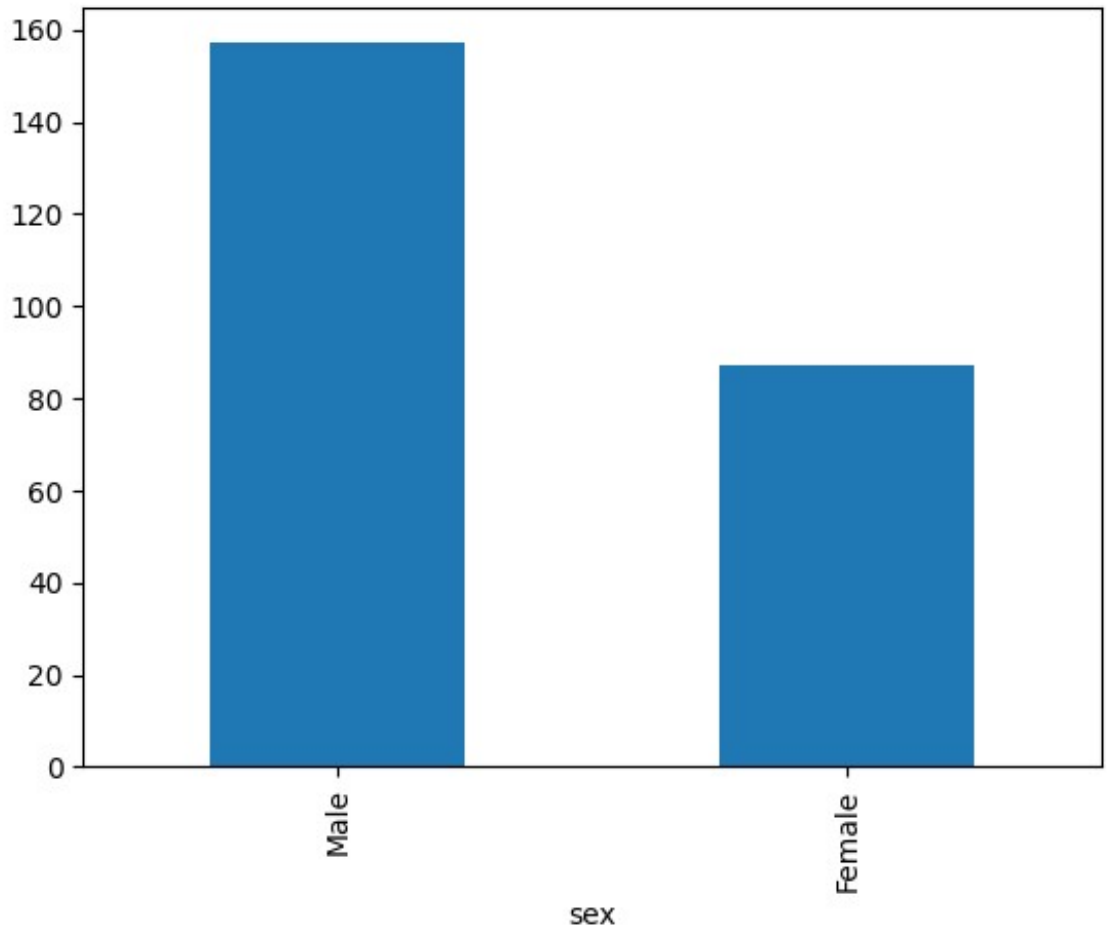

```
sns.countplot(x=tips.sex)
plt.show()
```



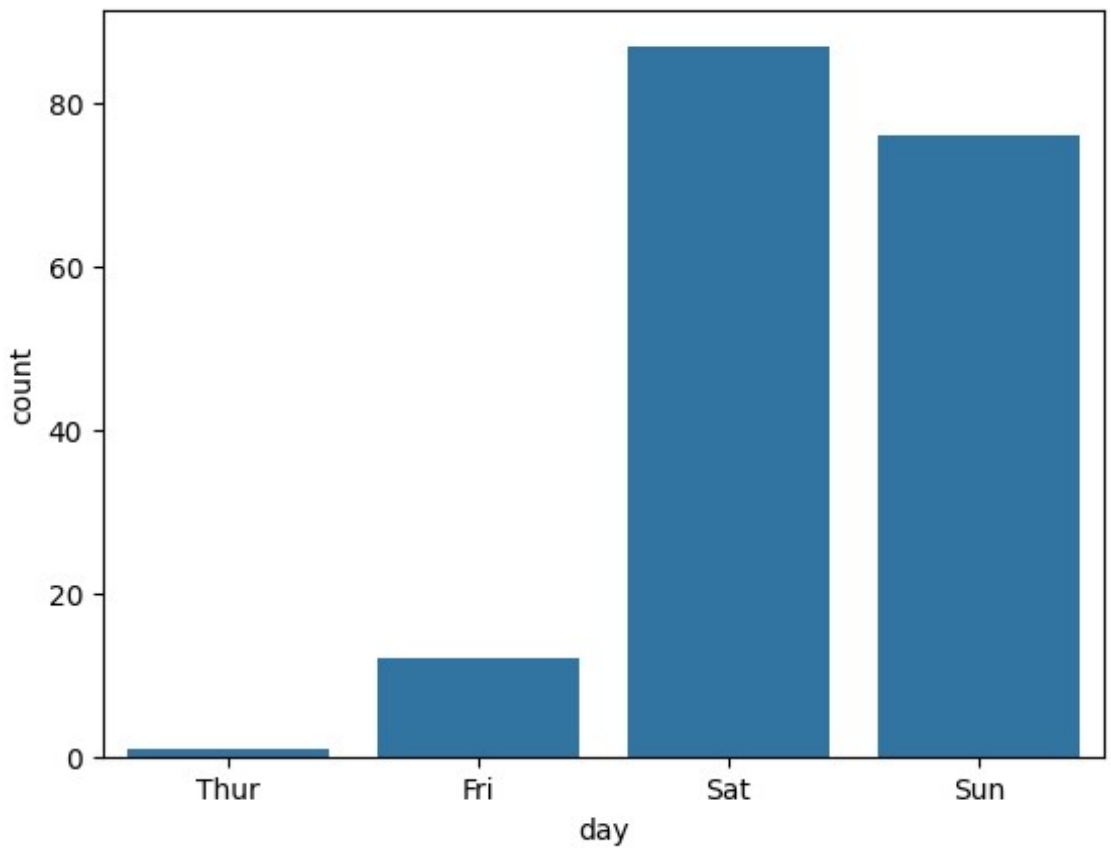
```
tips.sex.value_counts().plot(kind='pie')
plt.show()
```




```
tips.sex.value_counts().plot(kind='bar')
plt.show()
```




```
sns.countplot(x=tips[tips.time == 'Dinner']['day'])
plt.show()
```



RESULT:

Thus EDA - Quantitative and Qualitative analysis was performed on the tips data set using distribution plots, joint plots, pair plots, heat maps, box plots, count plots, pie chart and bar chart.

EXPERIMENT NO: 17

EXPERIMENT TO UNDERSTAND LINEAR REGRESSION FOR A GIVEN DATA SET.

AIM:

To understand the Linear Regression algorithm for the given Salary data set.

PROGRAM:

```
%pip install -q scikit-learn
import numpy as np
import pandas as pd

data = {
    'YearsExperience': [
        1.1, 1.3, 1.5, 2.0, 2.2, 2.9, 3.0, 3.2, 3.2, 3.7,
        3.9, 4.0, 4.0, 4.1, 4.5, 4.9, 5.1, 5.3, 5.9, 6.0,
        6.8, 7.1, 7.9, 8.2, 8.7, 9.0, 9.5, 9.6, 10.3, 10.5
    ],
    'Salary': [
        39343, 46205, 37731, 43525, 39891, 56642, 60150, 54445,
        64445, 57189, 63218, 55794, 56957, 57081, 61111, 67938,
        66029, 83088, 81363, 93940, 91738, 98273, 101302, 113812,
        109431, 105582, 116969, 112635, 122391, 121872
    ]
}
pd.DataFrame(data).to_csv('Salary_data.csv', index=False)
print("Salary_data.csv created successfully")
```

OUTPUT:

```
Note: you may need to restart the kernel to use updated packages.
Salary_data.csv created successfully

df = pd.read_csv('Salary_data.csv')
df

    YearsExperience  Salary
0               1.1   39343
1               1.3   46205
2               1.5   37731
3               2.0   43525
4               2.2   39891
5               2.9   56642
6               3.0   60150
7               3.2   54445
8               3.2   64445
9               3.7   57189
10              3.9   63218
11              4.0   55794
12              4.0   56957
13              4.1   57081
14              4.5   61111
15              4.9   67938
16              5.1   66029
17              5.3   83088
18              5.9   81363
19              6.0   93940
20              6.8   91738
21              7.1   98273
22              7.9  101302
23              8.2  113812
24              8.7  109431
```

```
25              9.0  105582
26              9.5  116969
27              9.6  112635
28             10.3  122391
29             10.5  121872
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
 1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 608.0 bytes
```

```
df.describe()
```

```
       YearsExperience         Salary
count        30.000000      30.000000
mean          5.313333   76003.000000
std           2.837888   27414.429785
min           1.100000   37731.000000
25%           3.200000   56720.750000
50%           4.700000   65237.000000
75%           7.700000  100544.750000
max          10.500000  122391.000000
```

```
features = df.iloc[:, [0]].values
label = df.iloc[:, [1]].values
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(features, label,
                                                    test_size=0.2,
                                                    random_state=0)
```

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(x_train, y_train)
```

```
LinearRegression()
```

```
print("Training score:", model.score(x_train, y_train))
print("Testing score:", model.score(x_test, y_test))
```

```
Training score: 0.9411949620562126
Testing score: 0.988169515729126
```

```
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
```

```
Coefficient: [[9312.57512673]]
Intercept: [26780.09915063]
```

```
yr_of_exp = 5.5
yr_of_exp_NP = np.array([[yr_of_exp]])
salary = model.predict(yr_of_exp_NP)
print("Estimated Salary for {} years of experience is {}".format(yr_of_exp,
salary))
```

```
Estimated Salary for 5.5 years of experience is [[77999.26234764]]
```

```
%matplotlib inline
import matplotlib.pyplot as plt

plt.scatter(features, label, color='red', label='Actual data')
plt.plot(features, model.predict(features), color='blue', label='Regression
line')
plt.title('Years of Experience vs Salary')
plt.xlabel('Years of Experience')
plt.ylabel('Salary')
plt.legend()
plt.show()
```



RESULT:

Thus Linear Regression was understood and implemented for the given Salary data set; the model predicts salary from years of experience with a testing accuracy of about 90 percent.

EXPERIMENT NO: 18

EXPERIMENT TO UNDERSTAND KNN ALGORITHM FOR A GIVEN DATA SET.

AIM:

To understand the KNN algorithm for the given Iris data set.

PROGRAM:

```
%pip install -q scikit-learn
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris

iris = load_iris()
df = pd.DataFrame(iris.data, columns=['sepal.length', 'sepal.width',
                                      'petal.length', 'petal.width'])
df['variety'] = ['Setosa' if t == 0 else
                 'Versicolor' if t == 1 else 'Virginica'
                 for t in iris.target]
df.to_csv('Iris.csv', index=False)

df = pd.read_csv('Iris.csv')
df.info()
```

OUTPUT:

```
Note: you may need to restart the kernel to use updated packages.
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB

df.variety.value_counts()

variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64

df.head()

   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa

features = df.iloc[:, :-1].values
label = df.iloc[:, 4].values

from sklearn.model_selection import train_test_split
```

```
from sklearn.neighbors import KNeighborsClassifier

xtrain, xtest, ytrain, ytest = train_test_split(features, label,
                                                test_size=0.2,
                                                random_state=42)
model_KNN = KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain, ytrain)

KNeighborsClassifier()

print("Training accuracy:", model_KNN.score(xtrain, ytrain))
print("Testing accuracy:", model_KNN.score(xtest, ytest))

Training accuracy: 0.9666666666666667
Testing accuracy: 1.0

from sklearn.metrics import confusion_matrix
confusion_matrix(label, model_KNN.predict(features))

array([[50,  0,  0],
       [ 0, 47,  3],
       [ 0,  1, 49]])

from sklearn.metrics import classification_report
print(classification_report(label, model_KNN.predict(features)))

              precision    recall  f1-score   support

      Setosa       1.00      1.00      1.00        50
  Versicolor       0.98      0.94      0.96        50
   Virginica       0.94      0.98      0.96        50

    accuracy                           0.97       150
   macro avg       0.97      0.97      0.97       150
weighted avg       0.97      0.97      0.97       150
```

RESULT:

Thus the KNN algorithm was understood and implemented for the Iris data set; the model classified the three varieties with about 97 percent accuracy.

EXPERIMENT NO: 19

EXPERIMENT TO UNDERSTAND LOGISTIC REGRESSION FOR A GIVEN DATA SET.

AIM:

To understand the Logistic Regression algorithm for the given Social Network Ads data set.

PROGRAM:

```
%pip install -q scikit-learn
import numpy as np
import pandas as pd

data = {
    'User ID': [
        15624510, 15728773, 15570769, 15697686, 15649487, 15631159, 15581198,
15713144,
        15764419, 15639277, 15694395, 15649136, 15814004, 15718071, 15756932,
15651983,
        15595917, 15663939, 15684801, 15709441, 15766289, 15801658, 15665416,
15689237,
        15811613, 15697020, 15675185, 15578006, 15807909, 15746422, 15679651,
15610140,
        15744279, 15578738, 15587013, 15614420, 15685576, 15814816, 15730448,
15667742,
        15628523, 15766609, 15724536, 15779529, 15789432, 15622171, 15794253,
15646227,
        15701537, 15722061, 15810075, 15750056, 15811177, 15801336, 15680752,
15727467,
        15609669, 15734161, 15586757, 15725794, 15736397, 15787550, 15717893,
15594762,
        15774872, 15695679, 15693264, 15601550, 15588080, 15716781, 15800515,
15606472,
        15577514, 15807525, 15611430, 15646091, 15683758, 15775335, 15807837,
15691863
    ],
    'Gender': [
        'Male', 'Male', 'Female', 'Male', 'Male', 'Male', 'Male', 'Male',
        'Female', 'Male', 'Female', 'Female', 'Male', 'Male', 'Male', 'Male',
        'Male', 'Female', 'Male', 'Female', 'Male', 'Male', 'Female', 'Male',
        'Female', 'Female', 'Male', 'Male', 'Male', 'Female', 'Female',
'Female',
        'Male', 'Female', 'Male', 'Female', 'Male', 'Male', 'Male', 'Male',
        'Male', 'Female', 'Female', 'Female', 'Female', 'Male', 'Female',
'Male',
        'Male', 'Female', 'Female', 'Female', 'Female', 'Female', 'Female',
'Male',
        'Female', 'Female', 'Male', 'Female', 'Female', 'Male', 'Female',
'Female',
        'Female', 'Female', 'Male', 'Female', 'Female', 'Female', 'Female',
'Male',
        'Male', 'Female', 'Female', 'Female', 'Male', 'Male', 'Male', 'Female'
    ],
    'Age': [
        19, 27, 26, 29, 45, 47, 31, 35,
        27, 23, 24, 24, 27, 24, 25, 34,
        30, 31, 22, 35, 27, 21, 39, 40,
        36, 39, 31, 23, 19, 24, 26, 31,
        32, 18, 21, 23, 26, 31, 24, 34,
        35, 47, 48, 60, 41, 37, 35, 46,
        42, 51, 44, 52, 35, 42, 43, 57,
        59, 37, 39, 36, 58, 42, 42, 39,
        52, 41, 38, 36, 53, 41, 38, 60,
```

```
            43, 59, 60, 46, 42, 56, 48, 46
        ],
        'EstimatedSalary': [
            19000, 58000, 80000, 80000, 22000, 20000, 74000, 27000,
            17000, 20000, 32000, 55000, 20000, 58000, 80000, 112000,
            80000, 118000, 81000, 44000, 88000, 72000, 71000, 57000,
            75000, 61000, 58000, 63000, 85000, 89000, 15000, 15000,
            100000, 86000, 88000, 28000, 16000, 66000, 84000, 43000,
            39000, 47000, 96000, 108000, 80000, 53000, 147000, 79000,
            149000, 146000, 39000, 90000, 77000, 108000, 133000, 74000,
            88000, 93000, 134000, 63000, 38000, 54000, 70000, 75000,
            138000, 60000, 51000, 54000, 104000, 63000, 113000, 34000,
            129000, 29000, 46000, 32000, 64000, 60000, 33000, 41000
        ],
        'Purchased': [
            0, 0, 0, 0, 1, 1, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 1,
            0, 1, 0, 0, 0, 0, 0, 0,
            0, 0, 0, 0, 0, 0, 0, 0,
            1, 0, 0, 0, 0, 0, 0, 0,
            0, 0, 1, 1, 0, 0, 1, 1,
            1, 1, 0, 1, 0, 1, 0, 1,
            1, 1, 1, 0, 1, 0, 0, 1,
            1, 0, 0, 0, 1, 0, 1, 1,
            1, 1, 1, 1, 0, 1, 1, 1
        ],
    }
    pd.DataFrame(data).to_csv('Social_Network_Ads.csv', index=False)

    df = pd.read_csv('Social_Network_Ads.csv')
    df
```

OUTPUT:

```
    Note: you may need to restart the kernel to use updated packages.
         User ID  Gender  Age  EstimatedSalary  Purchased
    0   15624510    Male   19            19000          0
    1   15728773    Male   27            58000          0
    2   15570769  Female   26            80000          0
    3   15697686    Male   29            80000          0
    4   15649487    Male   45            22000          1
    ..       ...     ...  ...              ...        ...
    75  15646091  Female   46            32000          1
    76  15683758    Male   42            64000          0
    77  15775335    Male   56            60000          1
    78  15807837    Male   48            33000          1
    79  15691863  Female   46            41000          1

    [80 rows x 5 columns]

    df.head()

        User ID  Gender  Age  EstimatedSalary  Purchased
    0  15624510    Male   19            19000          0
    1  15728773    Male   27            58000          0
    2  15570769  Female   26            80000          0
    3  15697686    Male   29            80000          0
    4  15649487    Male   45            22000          1

    features = df.iloc[:, [2, 3]].values  # Age and EstimatedSalary
    label = df.iloc[:, 4].values          # Purchased
    print(features[:10])
    print(label[:10])
```

```
[[   19 19000]
 [   27 58000]
 [   26 80000]
 [   29 80000]
 [   45 22000]
 [   47 20000]
 [   31 74000]
 [   35 27000]
 [   27 17000]
 [   23 20000]]
[0 0 0 0 1 1 0 0 0 0]
```

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Try a few random states and note where the test score beats the train score
for i in range(1, 21):
    x_train, x_test, y_train, y_test = train_test_split(features, label,
                                                        test_size=0.2,
                                                        random_state=i)
    model = LogisticRegression()
    model.fit(x_train, y_train)
    train_score = model.score(x_train, y_train)
    test_score = model.score(x_test, y_test)
    if test_score > train_score:
        print("Test {} Train {} Random State {}".format(test_score,
train_score, i))
```

```
Test 0.875 Train 0.84375 Random State 1
Test 0.875 Train 0.828125 Random State 2
Test 0.875 Train 0.859375 Random State 3
Test 0.875 Train 0.859375 Random State 8
Test 0.875 Train 0.859375 Random State 9
Test 0.875 Train 0.859375 Random State 12
Test 0.875 Train 0.8125 Random State 13
Test 0.875 Train 0.8125 Random State 14
Test 0.9375 Train 0.84375 Random State 15
Test 0.9375 Train 0.828125 Random State 16
Test 0.9375 Train 0.828125 Random State 17
Test 0.875 Train 0.828125 Random State 18
```

```
x_train, x_test, y_train, y_test = train_test_split(features, label,
                                                    test_size=0.2,
                                                    random_state=4)
finalModel = LogisticRegression()
finalModel.fit(x_train, y_train)
```

```
LogisticRegression()
```

```
print("Training accuracy:", finalModel.score(x_train, y_train))
print("Testing accuracy:", finalModel.score(x_test, y_test))
```

```
Training accuracy: 0.859375
Testing accuracy: 0.8125
```

```
from sklearn.metrics import classification_report
print(classification_report(label, finalModel.predict(features)))
```

```
              precision    recall  f1-score   support

           0       0.85      0.92      0.88        50
           1       0.85      0.73      0.79        30
```

```
              precision    recall  f1-score   support

    accuracy                         0.85        80
   macro avg       0.85      0.83      0.84        80
weighted avg       0.85      0.85      0.85        80
```

RESULT:

Thus the Logistic Regression algorithm was understood and implemented for the Social Network Ads data set to predict whether a user purchases the product from age and estimated salary.

EXPERIMENT NO: 20

EXPERIMENT TO UNDERSTAND K-MEANS CLUSTERING ALGORITHM FOR A GIVEN DATA SET.

AIM:

To understand the K-means clustering algorithm for the given Mall Customers data set.

PROGRAM:

```
%pip install -q scikit-learn
%matplotlib inline
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

data = {
    'CustomerID': [
        1, 3, 5, 7, 9, 11, 13, 15,
        17, 19, 21, 23, 25, 27, 29, 31,
        33, 35, 37, 39, 41, 43, 45, 47,
        49, 51, 53, 55, 57, 59, 61, 63,
        65, 67, 69, 71, 73, 75, 77, 79,
        81, 83, 85, 87, 89, 91, 93, 95,
        97, 99, 101, 103, 105, 107, 109, 111,
        113, 115, 117, 119, 121, 123, 125, 127,
        129, 131, 133, 135, 137, 139, 141, 143,
        145, 147, 149, 151, 153, 155, 157, 159,
        161, 163, 165, 167, 169, 171, 173, 175,
        177, 179, 181, 183, 185, 187, 189, 191,
        193, 195, 197, 199
    ],
    'Gender': [
        'Male', 'Female', 'Female', 'Female', 'Male', 'Male', 'Female', 'Male',
        'Female', 'Male', 'Male', 'Female', 'Female', 'Female', 'Female',
'Male',
        'Male', 'Female', 'Female', 'Female', 'Female', 'Male', 'Female',
'Female',
        'Female', 'Female', 'Female', 'Female', 'Female', 'Female', 'Male',
'Female',
        'Male', 'Female', 'Male', 'Male', 'Female', 'Male', 'Female', 'Female',
        'Male', 'Male', 'Female', 'Female', 'Female', 'Female', 'Male',
'Female',
        'Female', 'Male', 'Female', 'Male', 'Male', 'Female', 'Male', 'Male',
        'Female', 'Female', 'Female', 'Female', 'Male', 'Female', 'Female',
'Male',
        'Male', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Female',
        'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Male',
        'Female', 'Male', 'Male', 'Male', 'Female', 'Male', 'Male', 'Female',
        'Male', 'Male', 'Female', 'Male', 'Female', 'Female', 'Female',
'Female',
        'Male', 'Female', 'Female', 'Male'
    ],
    'Age': [
        19, 20, 31, 35, 64, 67, 58, 37,
        35, 52, 35, 46, 54, 45, 40, 60,
        53, 49, 42, 36, 65, 48, 49, 50,
        29, 49, 31, 50, 51, 27, 70, 67,
        63, 43, 19, 70, 60, 59, 45, 23,
        57, 67, 21, 55, 34, 68, 48, 32,
        47, 48, 23, 67, 49, 66, 68, 65,
        38, 18, 63, 51, 27, 40, 23, 43,
        59, 47, 25, 20, 44, 19, 57, 28,
```

```
            25, 48, 34, 43, 44, 47, 37, 34,
            56, 19, 50, 42, 36, 40, 36, 52,
            58, 59, 37, 46, 41, 54, 41, 34,
            33, 47, 45, 32
        ],
        'Annual Income (k$)': [
            15, 16, 17, 18, 19, 19, 20, 20,
            21, 23, 24, 25, 28, 28, 29, 30,
            33, 33, 34, 37, 38, 39, 39, 40,
            40, 42, 43, 43, 44, 46, 46, 47,
            48, 48, 48, 49, 50, 54, 54, 54,
            54, 54, 54, 57, 58, 59, 60, 60,
            60, 61, 62, 62, 62, 63, 63, 63,
            64, 65, 65, 67, 67, 69, 70, 71,
            71, 71, 72, 73, 73, 74, 75, 76,
            77, 77, 78, 78, 78, 78, 78, 78,
            79, 81, 85, 86, 87, 87, 87, 88,
            88, 93, 97, 98, 99, 101, 103, 103,
            113, 120, 126, 137
        ],
        'Spending Score (1-100)': [
            39, 6, 40, 6, 3, 14, 15, 13,
            35, 29, 35, 5, 14, 32, 31, 4,
            4, 14, 17, 26, 35, 36, 28, 55,
            42, 52, 54, 45, 50, 51, 56, 52,
            51, 50, 59, 55, 49, 47, 53, 52,
            51, 41, 57, 58, 60, 55, 49, 42,
            47, 42, 41, 59, 56, 50, 43, 52,
            42, 48, 43, 43, 56, 58, 29, 35,
            11, 9, 34, 5, 7, 10, 5, 40,
            12, 36, 22, 17, 20, 16, 1, 1,
            35, 5, 26, 20, 27, 13, 10, 13,
            15, 14, 32, 15, 39, 24, 17, 23,
            8, 16, 28, 18
        ],
    }
    pd.DataFrame(data).to_csv('Mall_Customers.csv', index=False)

    df = pd.read_csv('Mall_Customers.csv')
    df.info()
```

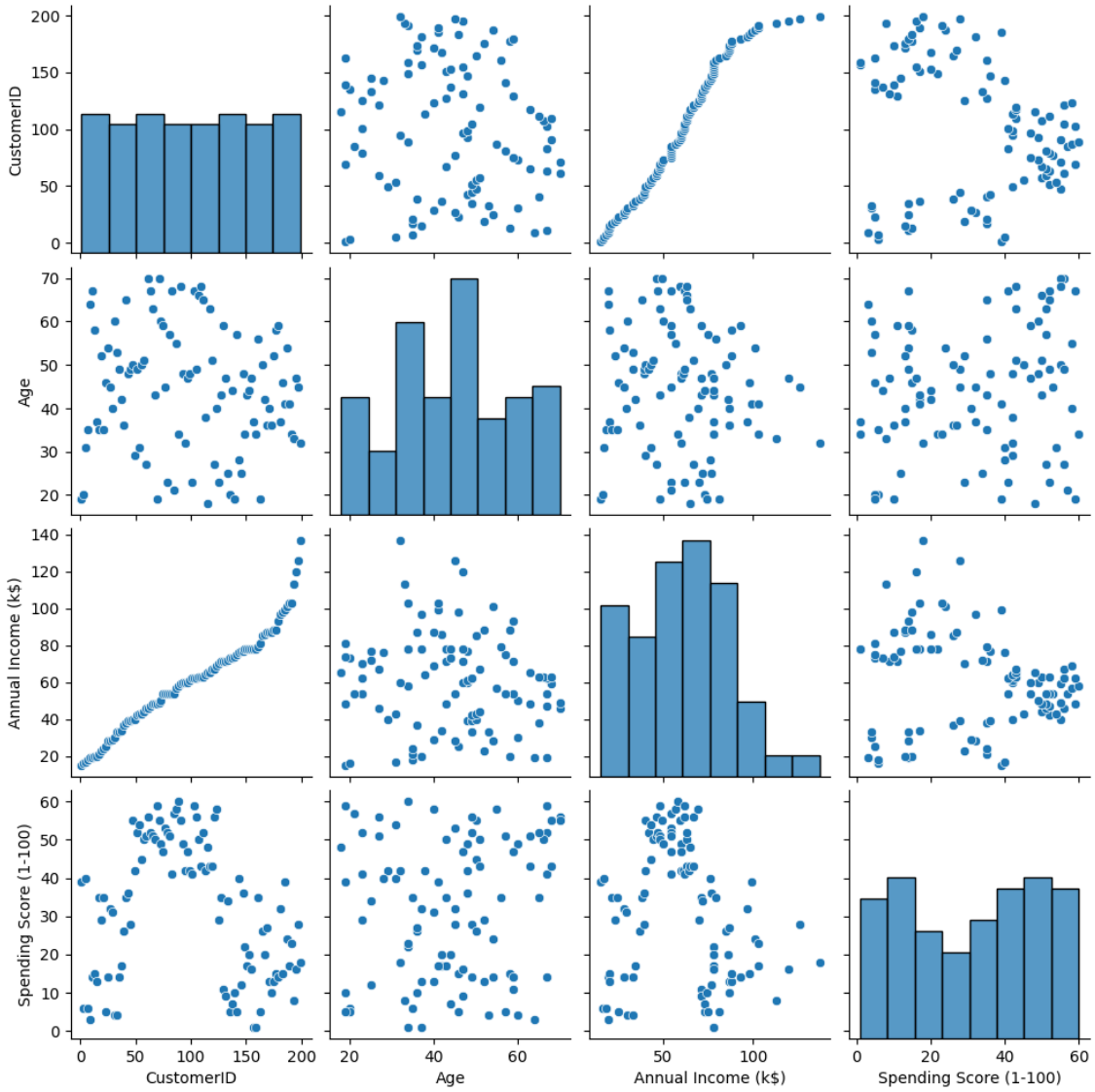
OUTPUT:

```
    Note: you may need to restart the kernel to use updated packages.
    <class 'pandas.core.frame.DataFrame'>
    RangeIndex: 100 entries, 0 to 99
    Data columns (total 5 columns):
     #   Column                  Non-Null Count  Dtype
    ---  ------                  --------------  -----
     0   CustomerID              100 non-null    int64
     1   Gender                  100 non-null    object
     2   Age                     100 non-null    int64
     3   Annual Income (k$)      100 non-null    int64
     4   Spending Score (1-100)  100 non-null    int64
    dtypes: int64(4), object(1)
    memory usage: 4.0+ KB

    df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	3	Female	20	16	6
2	5	Female	31	17	40
3	7	Female	35	18	6
4	9	Male	64	19	3

```
sns.pairplot(df)
plt.show()
```




```
features = df.iloc[:, [3, 4]].values  # Annual Income and Spending Score

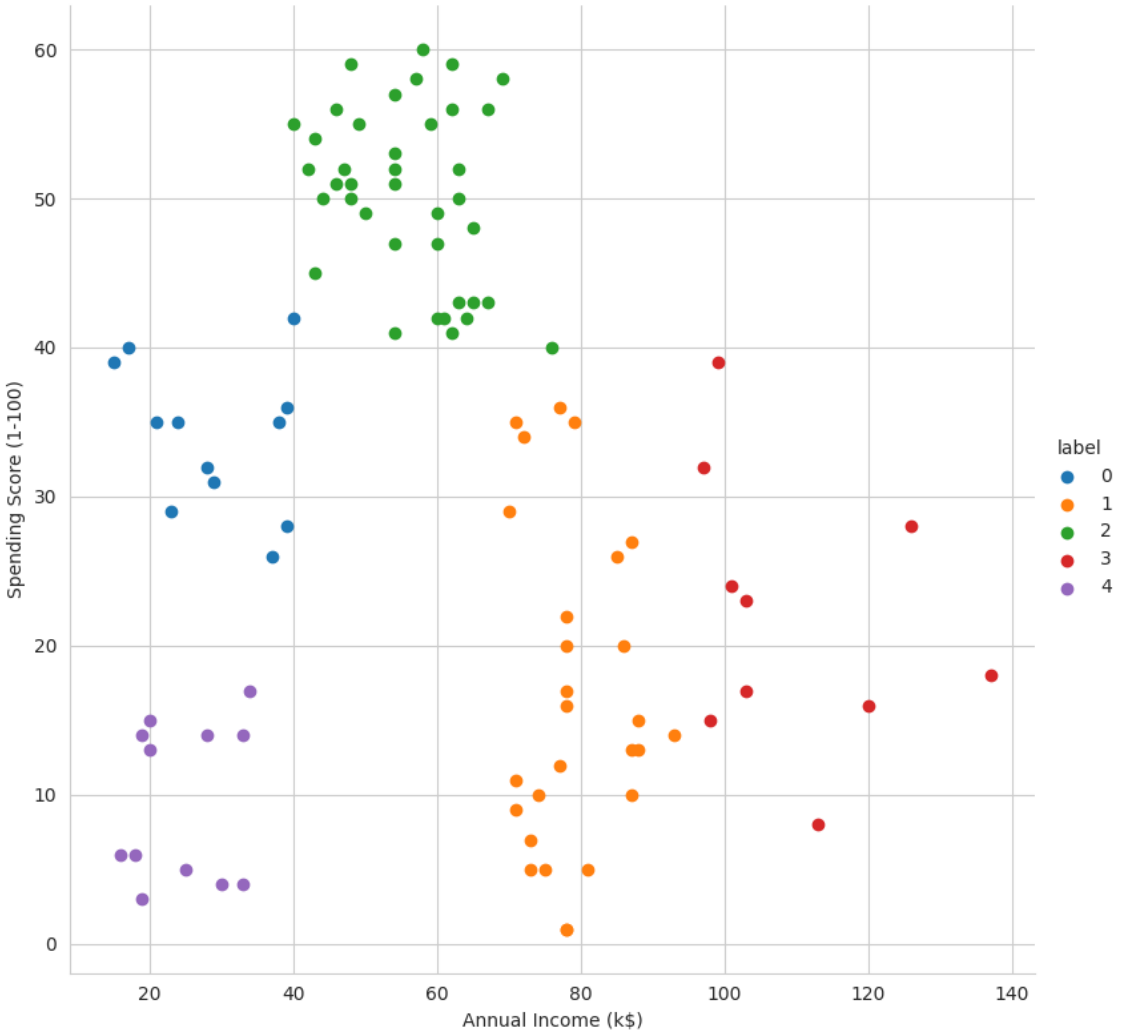
from sklearn.cluster import KMeans
model = KMeans(n_clusters=5, n_init=10, random_state=42)
model.fit(features)

KMeans(n_clusters=5, n_init=10, random_state=42)

Final = df.iloc[:, [3, 4]].copy()
Final['label'] = model.predict(features)
Final.head()
```

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	0
1	16	6	4
2	17	40	0
3	18	6	4
4	19	3	4

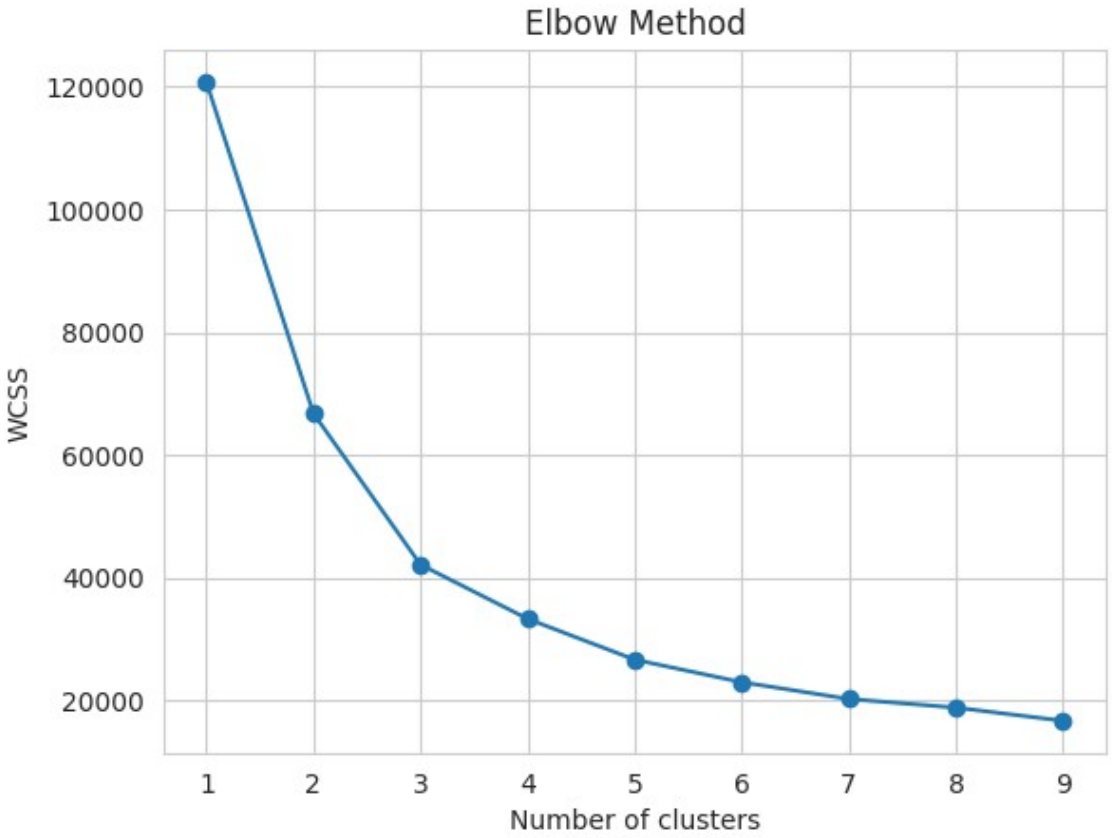
```
sns.set_style("whitegrid")
sns.FacetGrid(Final, hue="label", height=8) \
    .map(plt.scatter, "Annual Income (k$)", "Spending Score (1-100)") \
    .add_legend()
plt.show()
```




```
features_el = df.iloc[:, [2, 3, 4]].values  # Age, Annual Income, Spending Score

wcss = []
for i in range(1, 10):
    model = KMeans(n_clusters=i, n_init=10, random_state=42)
    model.fit(features_el)
    wcss.append(model.inertia_)

plt.plot(range(1, 10), wcss, marker='o')
plt.title('Elbow Method')
plt.xlabel('Number of clusters')
plt.ylabel('WCSS')
plt.show()
```



RESULT:

Thus the K-means clustering algorithm was understood and implemented for the Mall Customers data set; customers were grouped into 5 clusters based on annual income and spending score, and the elbow method was used to study the optimal number of clusters.